

Вінницький національний технічний університет  
Факультет інформаційних технологій та комп'ютерної інженерії  
Кафедра програмного забезпечення

КУРСОВА РОБОТА  
з дисципліни "Об'єктно-орієнтоване програмування"  
на тему: «РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ ДЛЯ МОДЕЛЮВАННЯ  
ОБ'ЄКТІВ "Місто музикантів. Макрооб'єкти: магазин музичних інструментів, підземний перехід,  
вокзал. Мікрооб'єкти: музикант-самоучка, випускник музичної школи, диригент"  
З ВИКОРИСТАННЯМ UML ТА МОВИ ПРОГРАМУВАННЯ JAVA»

Виконав: студент 1-го курсу групи 1ПІ-256  
спеціальності F2 Інженерія програмного  
забезпечення

Марко ВОВЧОК

*Signature*

Керівник доцент кафедри ПЗ  
к.т.н., доц. Олександр РЕШЕТНІК

Кількість балів: ##

Оцінка: ЄКТС #

Члени комісії:

\_\_\_\_\_  
(підпис)

Решетнік О.О.

\_\_\_\_\_  
(підпис)

Кательніков Д.І.

\_\_\_\_\_  
(підпис)

\_\_\_\_\_  
(прізвище та ініціали)

м. Вінниця - 2026 рік

Вінницький національний технічний університет  
Факультет інформаційних технологій та комп'ютерної інженерії  
Кафедра програмного забезпечення  
Рівень вищої освіти перший (бакалаврський)  
Спеціальність F2 Інженерія програмного забезпечення  
Освітньо-професійна програма «Інженерія програмного забезпечення»



**ЗАТВЕРДЖУЮ**

Завідувач кафедри ПЗ

д.т.н., проф., О.Н. Романюк

«10» лютого 2026 року

## ІНДИВІДУАЛЬНЕ ЗАВДАННЯ

на курсову роботу з дисципліни "Об'єктно-орієнтоване програмування"

**Вовчку Марку Івановичу**, студенту гр. 1ПІ-256

**Тема роботи:** РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ ДЛЯ МОДЕЛЮВАННЯ ОБ'ЄКТІВ "Місто музикантів."

**Макрооб'єкти:** магазин музичних інструментів, підземний перехід, вокзал. **Мікрооб'єкти:** музикант-самоучка, випускник музичної школи, диригент" з ВИКОРИСТАННЯМ UML ТА МОВИ ПРОГРАМУВАННЯ JAVA.

**Завдання:** розробити програмну систему і пояснювальну записку до процесу розробки.

**Вхідні дані до виконання проєкту:**

Мова програмування: Java.

Середовище розробки: NetBeans IDE, IntelliJ IDEA, Eclipse або Visual Studio Code.

Графічний інтерфейс: платформа JavaFX, Swing або AWT.

Зображення мікрооб'єкта: мінімум три графічних примітива (один з яких – текст). Зображення кожного макрооб'єкту повинно складатись не менше чим з трьох графічних примітивів (лінія, прямокутник, коло, арка, полігон, текст, графічний образ та інших), при цьому обов'язково один з примітивів повинен бути текст. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання. Для класу

мікрооб'єкта слід реалізувати глибинне копіювання. Макрооб'єкти повинні мати можливість містити кількість мікрооб'єктів більше двох. Повина бути реалізована ієрархія класів мікрооб'єктів, яка містить як мінімум три рівня наслідування, які можуть інстанціюватись і відображатись на екрані. Повинен бути реалізований наступний запит: знайти мікрооб'єкт з вказаними параметрами. Програма шукає об'єкт з введеними параметрами і видає на екран: його місце розташування, приналежність до макрооб'єктів (чи зазначає, що мікрооб'єкт не належить жодному макрооб'єкту). Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які належать вказаному макрооб'єкту. Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які не належать жодному макрооб'єкту. Повинен бути реалізований наступний запит: встановити критерій сортування мікрооб'єктів у запитах, які виводять переліки. Критеріїв сортування повинно бути не менше трьох, наприклад: Ім'я\_студента, Середній\_бал, Кількість\_Оцінок\_4\_та\_вище. Повинен бути реалізований автоматичний рух деяких мікрооб'єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою; 2) деякі мікрооб'єкти заходять в макрооб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Необхідно реалізувати щонайменше три запити на підрахунок мікрооб'єктів. Наприклад: а) кількість активних мікрооб'єктів; б) кількість об'єктів із рівнем життя понад 50%; в) кількість об'єктів, що розташовані у правій частині екрана тощо. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач. Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) на мінікарті повинна бути позначена видима область універсального об'єкта; 2) рух видимої області універсального об'єкта відповідно змінює мінікарту; 3) макрооб'єкти та мікрооб'єкти позначені на мінікарті; 4) рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті; 5) натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на екрані.

### **Зміст пояснювальної записки:**

АНОТАЦІЯ

ЗМІСТ

ВСТУП

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ  
ЗАВДАННЯ НА РОБОТУ

2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

4 КЕРІВНИЦТВО КОРИСТУВАЧА

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

ДОДАТОК А (обов'язковий) Лістинг програми

**Дата видачі завдання:** "11" лютого 2026 р.

**Строк подання завершеної курсової роботи:** 13 червня 2026 р.

Керівник \_\_\_\_\_

*Signature*  
(підпис)

Олександр РЕШЕТНІК

Завдання отримав \_\_\_\_\_

*Signature*  
(підпис)

**Марко ВОВЧОК**

## АНОТАЦІЯ

**Вовчок М.І.** Розробка програмної системи для моделювання об'єктів **"Місто музикантів. Макрооб'єкти: магазин музичних інструментів, підземний перехід, вокзал. Мікрооб'єкти: музикант-самоучка, випускник музичної школи, диригент."** з використанням UML та мови програмування Java. : курсова робота з дисципліни «Об'єктно-орієнтоване програмування»

У курсовій роботі здійснена розробка програмного застосунку для ОС Windows/Linux/Mac OS, виконаний за допомогою JavaFX. Програмний застосунок моделює функціонування об'єктів **"музиканти, музичні інструменти, магазини"** з використанням UML та мови програмування Java. Використано усі основні принципи об'єктно-орієнтованого програмування, а саме: абстракція, інкапсуляція, успадкування та поліморфізм. **Проаналізовано сучасний стан проблеми зменшення кількості музикантів.** Створено готовий програмний продукт.

Ключові слова: мікрооб'єкти, успадкування, макрооб'єкти, агрегація, універсальний об'єкт, композиція.

## ЗМІСТ

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| ВСТУП                                                                   | 4  |
| 1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ<br>ЗАВДАННЯ НА РОБОТУ | 7  |
| 1.1 Предметна область                                                   | 7  |
| 1.2 Розробка технічного завдання на роботу                              | 9  |
| 1.3 Обґрунтування вибору мови програмування                             | 13 |
| 1.4 Висновки                                                            | 17 |
| 2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ                           | 18 |
| 2.1 Математична модель об'єкта                                          | 18 |
| 2.2 Модель графічного відображення                                      | 20 |
| 2.3 Графічні процедури підсистеми графічного відображення               | 22 |
| 3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ                            | 24 |
| 3.1 Діаграма наслідування                                               | 24 |
| 3.2 Діаграма агрегації                                                  | 26 |
| 3.3 Діаграма композиції                                                 | 26 |
| 3.4 Діаграма асоціації                                                  | 26 |
| 3.5 Діаграма кооперації                                                 | 30 |
| 3.6 Діаграма послідовності                                              | 30 |
| 3.7 Діаграма стану                                                      | 32 |
| 4 КЕРІВНИЦТВО КОРИСТУВАЧА                                               | 34 |
| ВИСНОВКИ                                                                | 37 |
| СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ                                              | 38 |
| ДОДАТОК А (обов'язковий) Лістинг програми                               | 39 |

## ВСТУП

Сьогодні неозброєним оком видно, що сучасний світ взагалі не такий, як був раніше. Величезна кількість нових технологій, крутих винаходів, досягнень в різних областях науки, зрештою, штучний інтелект та машинне навчання – усе це робить наше повсякденне життя набагато легшим, веселішим та просто цікавим. Кожного дня якийсь професор на одному кінці світу знаходить відповідь на задачу, якою займався усе своє життя, в свою чергу на іншому кінці світу хтось розробляє план по запуску людини на Марс. У нас в Україні, наприклад, розробляють макет двигуна, що працюватиме на воді. Загалом, весь світ займається напруженою науковою діяльністю по знаходженню вакцини для корона-вірусу.

Час не стоїть на місці, змінюється і життя. Змінюється і людство, прагнучи досконалості у всьому, особливо у техніці. Дуже хочеться, витрачаючи якнайменше сил, робити все більше і більше справ. Тому люди і прагнуть, щоб сучасні технології повсякчас удосконалювалися. На ринку з'явилися технічні й електричні товари, здатні замінити будь-якого спеціаліста. Тут і електромобілі, електроплитки, автоматичні пилососи і ще безліч всього, що дає нам змогу радісно й вільно зітхнути: у нас є добрі помічники. З кожним роком кількість електричних товарів зростає, звільнюючи людину від тяжкої праці, допомагаючи в побуті [1].

Людство живе у дуже прогресивному та живому періоді, коли кожного дня може статися щось неймовірне, але на фоні усього цього колапсу з науки та техніки виникає одна «невеличка проблемка». Все рідше і рідше можна зустріти людей, які по-справжньому цікавляться мистецтвом, зокрема грою на музичних інструментах. Той факт, що все менше і менше людей можуть без проблем вночі в колі друзів біля вогню просто зіграти на гітарі, засмучує.

Напевно, більшість людей, почувши це, почнуть стверджувати, що гра на музичному інструменті, особливо сьогодні – це зовсім марна трата часу. Якщо їм вірити, цей навик ніколи не пригодиться в житті. Насправді ж, досконало відомо,

що це не так, вміння грати стає в нагоді майже завжди, а людина, яка ним володіє, ніколи про це не пожалкує.

Дуже часто музиканта запрошують зіграти на певному заході, виставці чи концерті. Також, зазвичай, їм є що робити на днях народження, коли після смачного перекусу можна взяти гітару і переспівати усі відомі пісні. Крім того, досить часто вміння грати на гітарі дуже допомагає у навчанні. Дійсно, коли, наприклад, уже три години пишеш лабораторну роботу з об'єктно-орієнтованого програмування, відбувається велике навантаження на розумову діяльність і дуже хочеться розслабитися. Тут приходить на поміч музичний інструмент: 10-15 хвилин спокійної та розслабленої гри – і людина знову в строю, готова штампувати лабораторні роботи пачками.

Окрім усього вищесказаного, досить поширеною проблемою сьогодення є те, що люди зазвичай недооцінюють важку роботу музиканта. Це зовсім не легко – взяти і з нуля навчитися віртуозно «бігати» по клавішах фортепіано, «затискати बारे» на гітарі, чи, наприклад, не фальшивити, граючи на скрипці. А як же безліч різних можливих варіацій мажорних, мінорних гамм, акордів, септакордів, домінант септакордів, та інших, незрозумілих для більшості термінів, в яких кожен, поважаючий себе музикант повинен без проблем розбиратися. На заняття музикою витрачається величезна кількість сил, енергії та обчислень: так само як в математиці, тільки замість інтегралів – секунди, терції, кварта, квінти і так далі. Усе це відкриває у тобі не лише здібності до гри, але й розвиває обидві половини головного мозку, заставляє мислити по-іншому – мислити творчо!

Повертаючись трішки до сучасних технологій, слід додати, що сьогодні існують деякі особливі види музичних інструментів, які «набиті» різними технічними покращеннями, які полегшують гру музиканта та збільшують її якість. Чого лише варта так звана «Гітара без струн» від компанії «Misa digital». Вона дійсно без струн, звук там відтворюється з вбудованого відтворюючого пристрою, а правильні ноти граються за допомогою сенсорного дисплею. Також хочеться згадати відомого ютубера – Коліна Фюрза, який створив гітару, яка окрім свого стандартного призначення уміє також випускати гігантські струмені вогню, прямо

як у одній відомій культовій франшизі. З'явилася ідея: створити на основі нейронної мережі штучний інтелект, який допомагатиме підбирати акомпанемент до музичних творів з мережі. Ця ідея унікальна тим, що вона поєднує дві, можна сказати, несумісні речі: ІТ та мистецтво. Можливо, колись у майбутньому вона буде реалізована.

Отже, у висновку хочеться сказати, що сьогодні кожна людина повинна намагатися розвиватися всебічно. Потрібно донести до усіх, що гра на музичних інструментах – це не просто розвага, це також можливість удосконалювати себе. Необхідно навчитися поєднувати заняття музикою з технічним навчанням, а в подальшому житті використовувати усе це для свого блага. Саме тому в якості теми курсової роботи було обрано розробку програмної системи моделювання життєдіяльності музикантів.



# 1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ

## 1.1 Предметна область

Музичні інструменти були створені у далекому минулому, коли ще не існувало ніяких телефонів та комп'ютерів, більше того, в ті часи не було навіть електрики.

Вже тоді людство почало ділитися на дві категорії: музиканти і не музиканти. Митці у свою чергу дотримувалися певної своєї ієрархії. Кожному музиканту притаманні такі важливі риси, як вміння слухати і любов до спокою. Окрім того кожен любитель музики чудово розбирається в піснях та видах музики, знає як сучасних, так і старих виконавців.

Існують **музиканти-самоучки (Shopper)**. Якимось так склалося, що цей тип виконавців володіє саме гітарою. Це легко зрозуміти, адже гітара – це відносно недорогий музичний інструмент (**Instrument**), і досить часто його навіть не потрібно купувати: є великий шанс знайти його на горіщі. Вони зазвичай володіють інструментом не гірше випускників музичної школи, а іноді навіть краще. Також такі люди налаштовані грати лише сучасну музику, або ту, що їм до вподоби. «Домашні» митці ніколи не гратимуть вам Бетховена чи Баха, для них це просто нестерпно.

Повна протилежність самоучок – це **випускники музичної школи (MusicianMaster)** або училища. Їх можна класифікувати за їхніми вміннями грати на певних інструментах. Існують майстри фортепіано і усіх інших клавішних інструментів. Є також віртуози гітарної гри, які з легкістю можуть самі навчитися грати на інших видах гітар. Зрідка серед випускників можна зустріти скрипачів. Такі люди скоріш за все в подальшому зв'яжуть свою кар'єру та життя з мистецтвом. Вони прекрасно володіють матеріалом і можуть навіть бути викладачем з музики.

Найвища      степінь      визнання      мають      **диригенти**      оркестру

**(OrchestraConductor)**. Такі музиканти мають багатолітній досвід та велику кількість інструментів в арсеналі. Вони з легкістю можуть творити на будь-якому інструменті, та навіть брати на себе відповідальність лідера в групі ще молодих музикантів.

Музичні інструменти також можна класифікувати. Існують клавішні, струнні, духові, ударні та інші. У свою чергу, кожен із цих типів також поділяється на багато інших. Струнними можуть бути гітари, як електро-, так і звичайні; скрипки, домбри, контрабаси та арфи. До духових інструментів можна віднести різні сопілки та труби. Серед клавішних інструментів відомі такі, як фортепіано, піаніно та рояль.

**Магазини** музичних інструментів можна зустріти нечасто, але зате вони майже завжди пропонують вам величезний асортимент, починаючи від звичайної гітари, і закінчуючи професійним фортепіано. Але потрібно розуміти, що покупка інструмента – це заняття не дешеве. Спочатку потрібно заробити кошти власними силами та працею.

Музиканти часто шукають можливостей додаткового заробітку. Наприклад, вони грають у **підземних переходах (Underpass)**. Підземні переходи не тільки дають прихисток від різних атмосферних негараздів нахвалт дощу та снігу, але й можуть виступати як зали з своєю атмосферою і публікою, яка нічого не підозрюючи прямує переходом і може долучитись до прослуховування музичних творів.

Інше місце, в якому збирається чимала аудиторія, це – **вокзал (Building)**. Тисячі людей, які їдуть залізницею у своїх справах внезапно можуть стати учасниками музичного дійства і прослухати музичний твір, очікуючи на свій потяг.

Необхідною умовою зростання майстерності музиканта є процес навчання в музичній школі **(School)**. Нічого дивного, що навчання коштує значну суму і не всім воно по кишені!

І вся ця музикальна вакханалія відбувається в універсальному об'єкті (ігровому світі) **місто (City)**. Навігація по ігровому світі полегшується завдяки

мінікарті (**MiniMap**).

Отже, було вирішено створити зручну зрозумілу програму в ігровій формі, за допомогою якої кожен зможе навчитися цінувати музикантів за їхню роботу. Більше того, програма навчить правильно розпоряджатися вільним часом, дасть можливість прослідкувати, як будуть підвищуватися навички музиканта під час його навчання.

## 1.2 Розробка технічного завдання на роботу

В програмі на мові програмування Java повинно бути створено застосунок з графічним інтерфейсом. Клас мікрооб'єкта повинен містити не менше 4 елементів змінних і не менше 4 методів (окрім геттерів та сеттерів). Як мінімум одна змінна повинна бути типу `int`, одна – типу `double` і як мінімум одна – рядок. Зображення мікрооб'єкта: мінімум три графічних примітива, один з яких – текст (навіть в неактивному стані). Білий (=невидимий) графічний примітив не рахується. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання. Для класу мікрооб'єкта слід реалізувати глибинне копіювання. Зображення мікрооб'єкта повинно містити графічні примітиви, які показують стан елемента посилального типу, який був доданий до класу мікрооб'єкта, щоб зробити необхідним глибинне копіювання при клонуванні. При натискуванні лівої кнопки миші на мікрооб'єкті він повинен ставати активним/неактивним. В програмі повинна бути реалізована активація декількох об'єктів одночасно. Зображення активного мікрооб'єкта повинно відрізнятись від зображення неактивного хоча б одним графічним примітивом. Інтерфейс програми повинен сигналізувати про те, який саме мікрооб'єкт активував користувач. По вибору студента інформація про активний мікрооб'єкт/мікрооб'єкти може виводитись у рядку статусу або робочій області програми. Якщо користувач активував декілька мікрооб'єктів, то інформація про це також має якимось чином відображатися. При натискувань клавіш-стрілок активні об'єкти/об'єкт повинні рухатись у вікні програми. При натискуванні

клавіши Delete активні об'єкти повинні знищуватись. Якщо активного об'єкта нема – клавіша, то клавіша Delete ігнорується. Клавіша Esc повинна відмінити активацію об'єкта. По певній комбінації клавіш (наприклад Ctrl-C) повинно бути реалізоване копіювання активного мікрооб'єкта. При натискуванні правої кнопки миші на мікрооб'єкті повинно з'являтися діалогове вікно, яке дозволяє змінювати параметри вказаного мікрооб'єкта. Додати до проекту три або більше макрооб'єкта. Макроб'єкти повинні мати можливість містити більше двох мікрооб'єктів одночасно. Додати команди рисування макрооб'єктів в програму. Не обов'язково мати можливості додавати/видаляти макрооб'єкти в програму. Цілком достатньо, якщо програма буде мати деяку кількість макрооб'єктів згенеровану при запуску програми і яка буде незмінною протягом роботи програми. Макроб'єкти повинні мати певні позиції в універсальному об'єкті і повинні відображатись на екрані. Позиції макрооб'єктів всередині універсального об'єкта можуть залишатись незмінними протягом роботи програми. В універсальному об'єкті повинно міститись як мінімум один екземпляр кожного оголошеного в темі макрооб'єкта. Зображення кожного макрооб'єкту повинно складатись не менше чим з трьох графічних примітивів (лінія, прямокутник, коло, арка, полігон, текст, графічний образ та інших), при цьому обов'язково один з примітивів повинен бути текст. Білий (=невидимий) графічний примітив не рахується. Мікрооб'єкти можуть або належати одному або більшій кількості макрооб'єктів або не належати жодному. Глядачу має бути зрозуміло в чому полягає приналежність мікрооб'єкта макрооб'єкту, тобто чим вигляд (або поведінка) такого мікрооб'єкта відрізняється від поведінки мікрооб'єкта, який не належить жодному макрооб'єкту. Повинна обов'язково бути реалізована можливість вилучити мікрооб'єкт з всіх макрооб'єктів, в результаті чого він не буде належати жодному макрооб'єкту. Повинна обов'язково бути реалізована можливість зробити так, щоб активний мікрооб'єкт став належати одному з наявних макрооб'єктів. В зображенні макрооб'єкта обов'язково повинно виводитись кількість мікрооб'єктів, які йому належать. Повина бути побудована ієрархія класів мікрооб'єктів, яка містить як мінімум три рівня наслідування, які можуть інстанціюватись і зображатись на

екрані. Зображення кожного рівня має відрізнятися від всіх інших хоча б одним графічним примітивом. В побудованій ієрархії класів повинні бути продемонстровані: а) виклик конструктора базового класу; б) виклик метода базового класу; в) використання динамічного поліморфізму; г) використання статичного поліморфізму. Повинно бути продемонстровано використання ключового слова `instanceof`. Повинен бути реалізований автоматичний рух деяких мікрооб'єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою (це має бути помітно візуально в тому сенсі, що або щось змінюється у їх зображенні або у характері їх руху); 2) деякі мікрооб'єкти заходять в макрооб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Під характером руху мається на увазі не просто траєкторія, а те, в який спосіб вони взаємодіють з макрооб'єктами та іншими мікрооб'єктами. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач (будь яким способом). Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) на мінікарті повинна бути позначена видима область універсального об'єкта; 2) рух видимої області універсального об'єкта відповідно змінює мінікарту; 3) макрооб'єкти та мікрооб'єкти позначені на мінікарті; 4) рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті; 5) натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на карті. При серіалізації/де-серіалізації обов'язково повинні використовуватись діалогові вікна, щоб запитати у користувача ім'я файлу та його розташування на диску (наприклад можна використовувати системне діалогове вікно `FileChooser`). Продемонструвати використання лямбда-виразу. Продемонструвати використання операція з потоком `Stream`. При натискуванні

клавiши Insert повинно з'являтися діалогове вікно, яке повинно визначати параметри створюваного мікрооб'єкта. В нього повинна бути присутня можливість обирати до якого з класів нащадків у ієрархії наслідування він належить. Крім керуючого елемента Button у діалоговому вікні також повинні бути використані як мінімум три з наступного списку: TextField, CheckBox, {ComboBox || ListView || ChoiceBox}, RadioButton.

Повинен бути реалізований наступний запит: знайти мікрооб'єкт з вказаними користувачем параметрами, в результаті якого програма шукає мікрооб'єкт і видає на екран: його місце розташування, приналежність цього мікрооб'єкта до макрооб'єктів (чи зазначає, що мікрооб'єкт не належить жодному макрооб'єкту). Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які належать вказаному макрооб'єкту. Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які не належать жодному макрооб'єкту. Повинен бути реалізований наступний запит: встановити критерій сортування мікрооб'єктів у запитах, які виводять переліки. Критеріїв сортування повинно бути не менше трьох, наприклад: Ім'я\_студента, Середній\_бал, Кількість\_Оцінок\_4\_та\_вище.

Повинні бути реалізовані як мінімум три запити по підрахуванню мікрооб'єктів. Наприклад а) кількість активних мікрооб'єктів, кількість мікрооб'єктів з рівнем життя більше половини, кількість мікрооб'єктів, які знаходяться в правій половині екрану тощо.

Для нормального виконання програми необхідно наступне апаратне забезпечення:

1. Комп'ютер серії IBM PC з частотою 2200 МГц і вище.
2. Операційні системи: Windows Vista/7/10; Linux; Mac OS.
3. 8 Gb оперативної пам'яті DDR4.
4. Графічний адаптер SVGA (Super Video Graphic Adapter).
5. Відео-карта об'ємом пам'яті не менше 2 Gb.
6. Клавіатура IBM, миша.

Розмір дискового простору, який потрібен для коректної роботи програми: 25 Mb.

Розмір оперативної пам'яті, що займає програма: 270 Мб.

### 1.3 Обґрунтування вибору мови програмування

Для виконання поставленого завдання була вибрана кроссплатформенна мова програмування Java[2]. Основними конкурентами при виборі були такі мови, як: C++[3], C#[4]. Тож розберімося, чому для виконання поставленої задачі Java являється оптимальним вибором.

Одразу хочеться наголосити, що Java – це дійсно велика та сильна мова програмування. Не дарма в рейтингу всесвітньо відомого сайту «Tiobe.com» [5] саме ця мова уже майже десять років займає першу позицію, лише зрідка на деякий час поступаючи своє місце мовам C та C++. Така велика популярність Java стає цілком зрозуміла, коли починаєш все глибше і глибше занурюватися у її вивчення. Маючи досить неважкий синтаксис, просто гігантське різноманіття різних бібліотек, можливість працювати з багатьма потоками виконання програми та чіткі стандартні принципи об'єктно-орієнтованого програмування, Java в деяких моментах просто витісняє конкурентів з поля бою, тим самим заповнює світ все більше і більше.

Але, як би того не хотілося, Java не ідеальна. Звісно, вона має свої ахіллесові п'яти, а також у неї відсутні деякі особливості, які є в мовах C++ та C#. Хоча ці усі недоліки повністю зникають на фоні переваг Java, їх все одно потрібно пам'ятати та брати до уваги, особливо, якщо ви хочете пов'язати своє майбутнє саме з цією мовою.

Отже, розглянемо основні найважливіші плюси та мінуси мови програмування Java, в порівнянні з C++ та C#.

Перелік властивостей, характерних лише Java:

1. Java є повністю кроссплатформенною мовою загального застосування. Це означає, що програми, написані на ній, працюють буквально на всьому, починаючи від суперкомп'ютерів, і закінчуючи мікро-чіпами, тоді як мова C++, наприклад, з самого початку орієнтована на компіляцію конкретно певною

платформою. Це стало можливим завдяки тому, що Java код компілюється віртуальною машиною в проміжний байт-код, який потім інтерпретується на будь-якій ОС без змін і великих зусиль.

2. Java надає широкі можливості для використання декількох потоків, що дозволяє програмам, написаним на ній працювати набагато швидше. Звісно, використання багатопоточності вимагає від програміста глибокого розуміння мови, а також високі навички програмування.

3. Java має повну зворотню сумісність з ранішніми версіями, це означає, що сьогодні чудово працюватимуть проекти, які були створені декілька років тому. Жодна мова не може похвалитися такою сумісністю, навіть платформа .NET з її мовою C#.

4. Концепція ООП набагато краще реалізована в Java, ніж в будь-яких інших мовах програмування. Це робить код, написаний на Java, більш схожим на звичайну людську мову, а робота з ним відбувається інтуїтивно та зрозуміло. До того ж, Java бере на себе усю низькорівневу роботу з пам'яттю: видаленням з пам'яті непотрібних об'єктів займається вбудований у віртуальну машину Garbage Collector.

5. Зрештою, на сьогодні Java повністю заповнила мобільний ринок. Усі програми для операційної системи андроїд пишуться на Java. Крім того, у цьому напрямку поступово набирає популярності мова Kotlin, яка ще більш спеціалізована на мобільній розробці і є послідовником Java майже у всьому.

Перелік мінусів, характерних Java, в порівнянні з C++ і C#:

1. Java повинна чимось розплачуватися за свою кросплатформенність, і це – її швидкодія. Як би там не було, виконання коду, написаного на C++ потребує набагато менше ресурсів комп'ютера, до того ж, він виконується набагато швидше. Крім того, як говорилося раніше, у Java відсутня робота напряму з пам'яттю. З одного боку це робить життя програміста легшим, але з іншого – саме використання вказівників у мовах сімейства «С» дозволяє працювати програмам максимально швидко.

2. У Java відсутня перегрузка стандартних операторів, яка притаманна мові



C#. Ця, на перший погляд, незначна особливість виявляється дуже корисною в певних нестандартних ситуаціях, а також робить програмний код зрозумілішим. Java цього позбавлена.

3. Java не надає можливостей для створення генераторів, які, наскільки відомо, є в мові C#, JavaScript та Python. Генератори допомагають економити пам'ять та змінювати план виконання певної функції прямо по ходу виконання програми.

4. У Java зовсім відсутня концепція функціонального та процедурного програмування, яка, як відомо, притаманна мовам Python, C# і JavaScript. Це досить велика тема, про яку можна говорити дуже довго, але якщо коротко, то ви ніяк не зможете передати функцію іншій функції в якості аргументу безпосередньо у точці її виклику, або ж повернути функцію з функції у Java. Відсутність саме цієї концепції є одним із найбільших мінусів і основною причиною вибору програмістом в якості основної мови, наприклад, мови C# [6].

Таблиця 1.1 - Порівняння мов програмування

| Особливості  | C++                                                            | C#                                                 | Java                                               |
|--------------|----------------------------------------------------------------|----------------------------------------------------|----------------------------------------------------|
| Парадигма    | Об'єктно-орієнтована, процедурна, з узагальненими типами даних | Об'єктно-орієнтована, з узагальненими типами даних | Об'єктно-орієнтована, з узагальненими типами даних |
| Рік випуску  | 1983                                                           | 2000                                               | 1995                                               |
| Тип системи  | Компілюємої мови програмування                                 | Компілюємої мови програмування                     | Компілюємої та інтерпретованої мови програмування  |
| Використання | Системне програмування, ігри, драйвери                         | Розробка додатків під Windows, веб-розробка        | Веб-розробка, серверні додатки, мобільна розробка  |

Продовження таблиці 1.1

| Особливості    | C++                                                             | C#                                                                  | Java                                                |
|----------------|-----------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------|
| Синтаксис      | Складний                                                        | Простіший за C++, але складніший за Java                            | Простий                                             |
| Переносимість  | Залежить від операційної системи, що використовується           | Добра переносимість між операційними системами                      | Висока переносимість між операційними системами     |
| Продуктивність | Дуже висока, володіє близькістю до машинного коду               | Висока, але менше, ніж у C++                                        | Нижча, ніж у C++ та C#                              |
| Безпека        | Низька, дуже чутлива до помилок, пов'язаних з пам'яттю          | Висока, засоби для уникнення помилок при роботі з пам'яттю          | Добра, з вбудованими засобами безпеки               |
| Розширюваність | Висока, можливість розширення за допомогою сторонніх бібліотек. | Добра, добре поєднується зі сторонніми бібліотеками та фреймворками | Добра, з великою кількістю бібліотек та фреймворків |

Таким чином, беручи до уваги результати багатокритеріального порівняння (див. таблиця 1.1) мов об'єктно-орієнтованого програмування, можна зробити висновок, що для реалізації поставленого завдання оптимальною є мова програмування Java.

#### 1.4 Висновки

Отже, була поставлена задача створити графічну програмну реалізацію життєдіяльності музикантів, вибору їхнього першого інструменту, розвитку та навчання, було розроблено технічне завдання на роботу і обгрунтовано вибір мови програмування Java для реалізації проекту.

## 2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

### 2.1 Математична модель об'єкта

У програмі реалізовані вісім можливих напрямків руху мікрооб'єктів: вгору, вниз, вправо, вліво, а також по діагоналях. Кожен напрямок має свій індивідуальний ідентифікатор – цифру від 0 до 7. Крім того, при зіткненні із краями робочої області, мікрооб'єкт відбивається у випадкову сторону, але обов'язково за напрямом, протилежним краю. Слід також знати, що у програмі мікрооб'єкти відбиваються не від глобальних країв сцени, а від видимих на конкретний момент часу, отже, є необхідність реалізувати підрахунок координат стінок в момент часу залежно від прокрутки екрану. Одна така координата складається із суми розміру вікна та поточного здвигу (рисунок 2.1) по кожній координаті X, Y. Алгоритм підрахунку здвигу по кожній координаті зображений нижче. Він виконується постійно під час роботи програми.

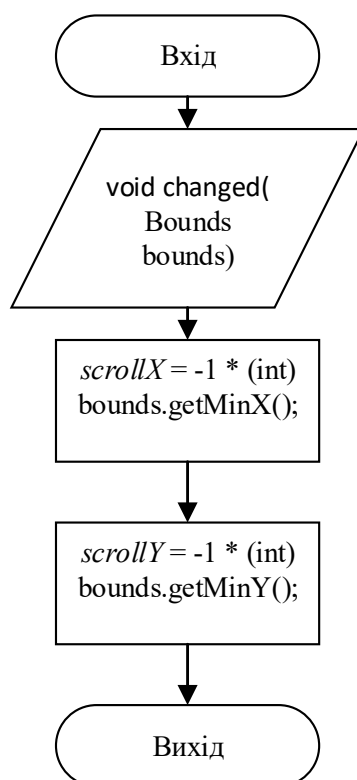


Рисунок 2.1 – Блок-схема алгоритму підрахунку здвигів по координатам

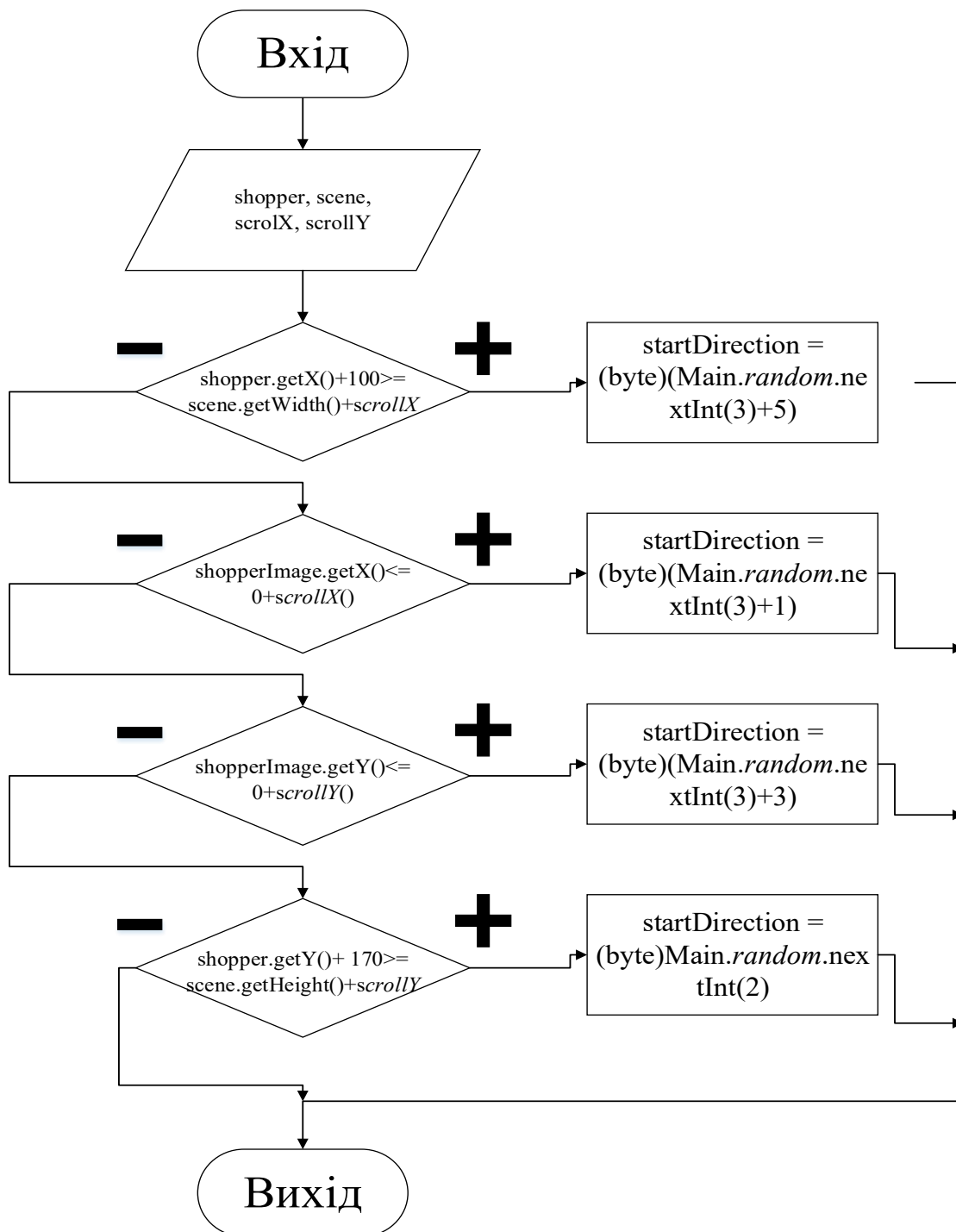


Рисунок 2.2 – Алгоритм підрахунку напрямку руху після зіткнення

Наступна блок-схема (див. рисунок 2.2) зображає алгоритм вибору напрямку руху при умові зіткнення з конкретною стінкою. На вхід приймає дані, підраховані алгоритмом на попередньому рисунку.

## 2.2 Модель графічного відображення

Кожен мікрооб'єкт складається з власне Png-картинки, текстової мітки та тіні (інструмент є окремим об'єктом, він не входить до складу елементів мікрооб'єкту та рисується окремо).

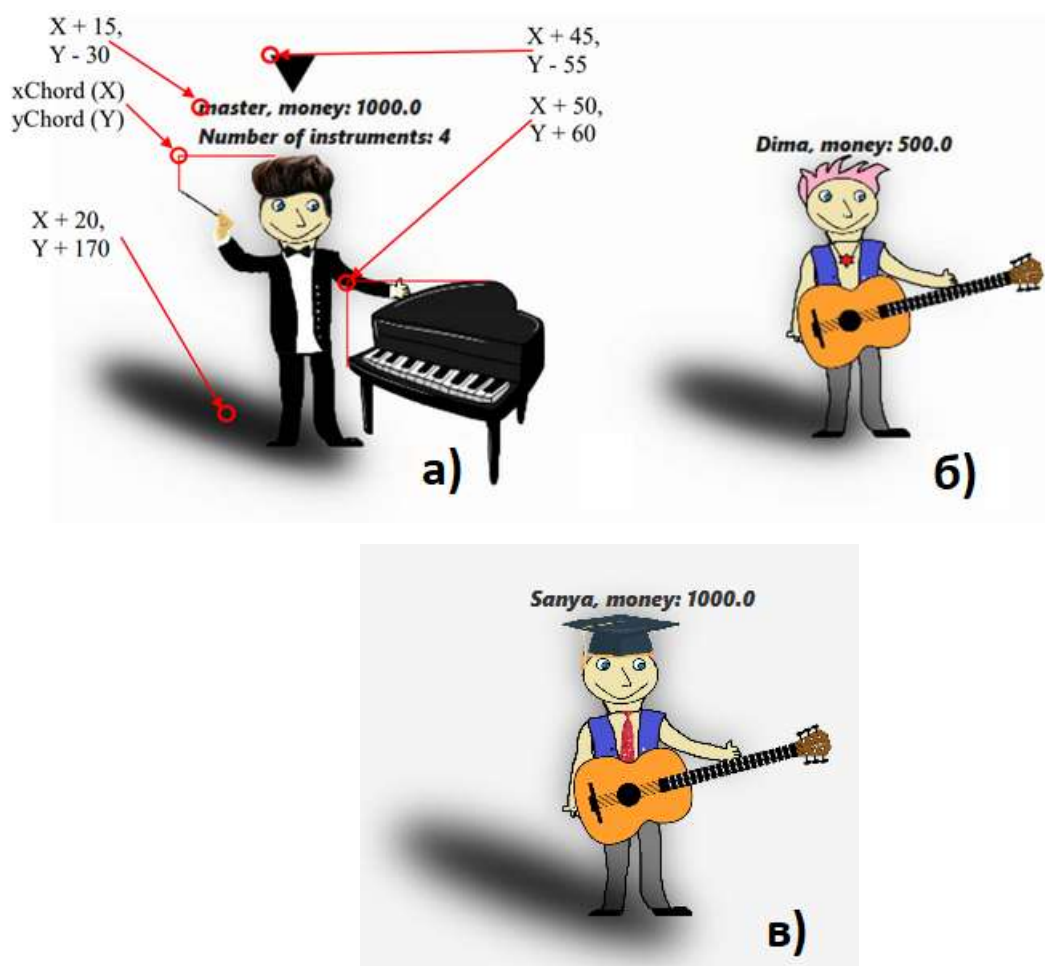


Рисунок 2.3 – Зображення мікрооб'єктів у програмі: а) диригент;  
б) музикант-самоучка; в) випускник музичної школи

Координати цих та деяких інших частин вираховуються відносно двох основних координат  $X$  та  $Y$  (див. рисунок 2.3), які є приватними полями у класі мікрооб'єкта. Коефіцієнти, які додаються щоб отримати правильну композицію картинки, були вираховані власноруч шляхом підбору та перевірки. Тінь у кожного мікрооб'єкта являє собою один з графічних примітивів, а саме еліпс з накладеними ефектами розмиття та повороту відносно головної осі.

Кожен макрооб'єкт складається з власне Png-картинки, текстової мітки та тіні. На рисунку 2.4 зображено Магазини музичних інструментів, на рисунку 2.5 зображено Вокзал, а на рисунку 2.6 - Підземний перехід.



Рисунок 2.4 – Зображення макрооб'єкту Магазин музичних інструментів

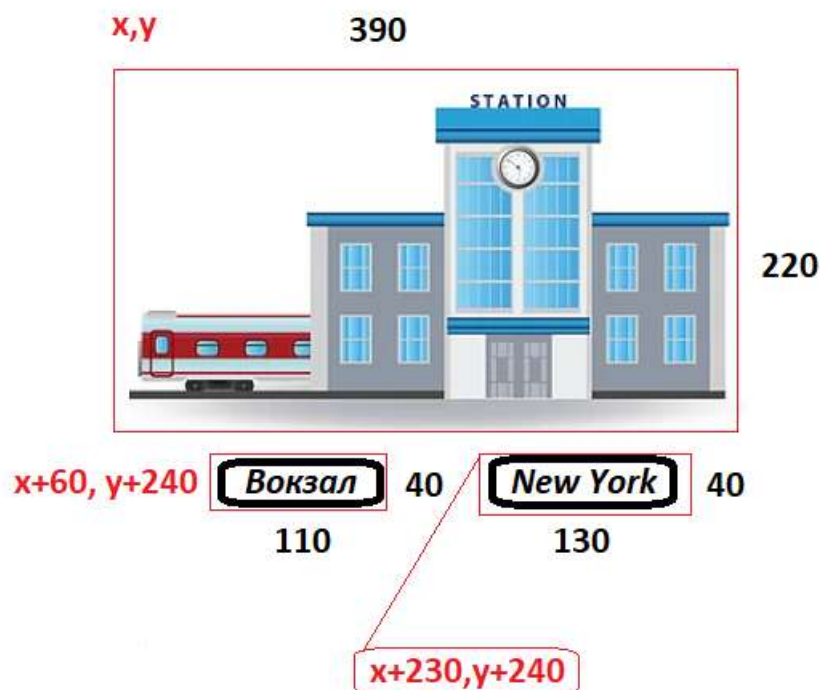
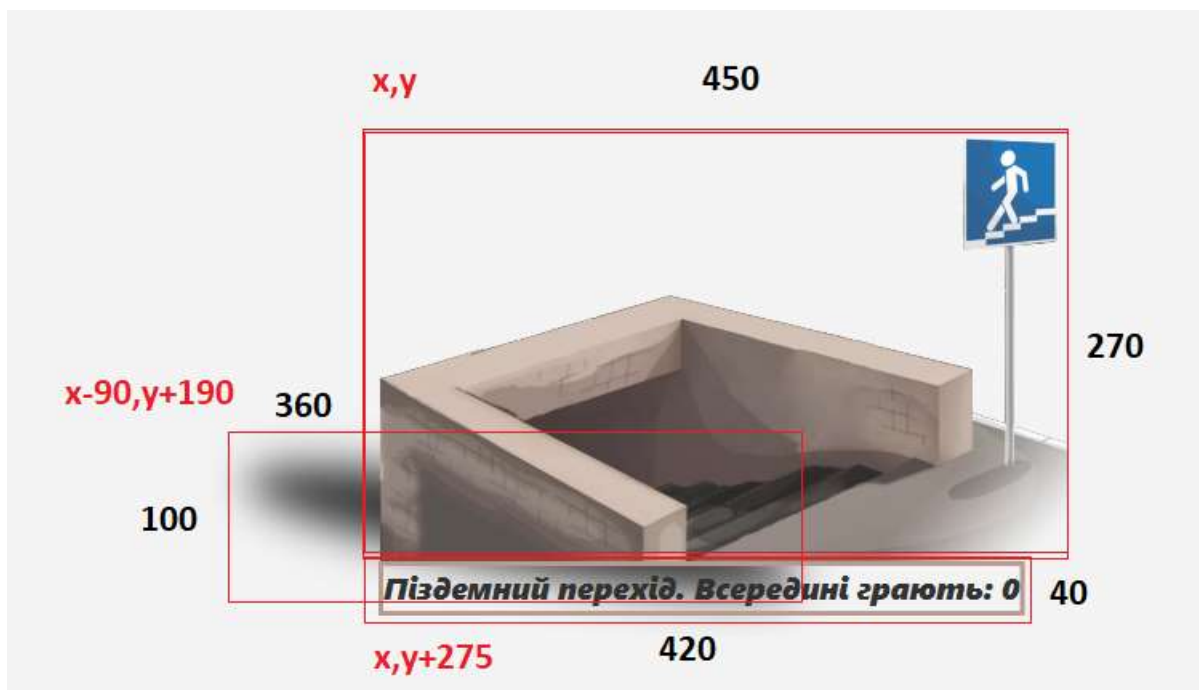


Рисунок 2.5 – Зображення макрооб'єкту Вокзал



Рисунку 2.6 - Зображення макрооб'єкту Підземний перехід

Деякі макрооб'єкти, зокрема Підземний перехід та Музична школа, в своєму зображенні містять кількість мікрооб'єктів, які їм належать в даний момент.

### 2.3 Графічні процедури підсистеми графічного відображення

Для створення зображень мікро- та макрооб'єктів у програмі було використано графічні засоби бібліотеки JavaFX [7]. Основними використаними графічними примітивами були:

`ImageView(Image image)` – конструктор елементу-картинки, який є наслідником класу `Node`, що дозволяє без проблем розміщати його на екрані.

`Image(String string)` – конструктор бібліотеки JavaFX, який завантажує картинку із переданого посилання або шляху у пам'яті комп'ютера.

`Polygon(...points)` – створює багатокутник із заданими координатами вершин, що є наслідником класу `Node` бібліотека JavaFX.

`Label(String string)` – створює текстову мітку, із заданим текстом.

`setEffect(Effect value)` – встановлює ефект для вузла, у якого був визваний цей сеттер.



`setRotation(double value)` – встановлює кут повороту елементу відносно головної осі.

`Group(...Node)` – створює групу, яка може містити у собі деяку кількість елементів класу `Node` для їх спільного розміщення на головний екран програми.

`Media(String url)` – завантажує медіа файл (mp3, mp4, wav і т. д.) і створює об'єкт, який містить цей файл.

`MediaPlayer(Media media)` – створює звуковий програвач для переданого медіа файлу, дозволяє запускати музику безпосередньо в програмі.

`setOnKeyPressed(EventHandler<KeyEvent> value)` – встановлює на вузол обробник події натискування на клавішу. Метод приймає анонімний клас, який містить переписаний метод `handle(Event event)`, у якому містяться інструкції для виконання по натискуванню на клавішу. Також може приймати лямбда-вираз.

`setOnMousePressed(EventHandler<MouseEvent> value)` – встановлює обробник події для натискування кнопки мишки.

`AnimationTimer()` – конструктор елементу, який містить методи `start()`, `stop()` та переданий нами переписаний метод `handle(long now)`. Під час виклику методу `start()` починають виконуватися інструкції, передані у метод `handle(long now)` і виконуються вони циклічно без зупинки, аж до виклику методу `stop()`.

### 3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

Діагра́ма класів (англ. class diagram) — статичне представлення структури моделі. Відображає статичні (декларативні) елементи, такі як: класи, типи даних, їх зміст та відношення.

#### 3.1 Діаграма наслідування

Наслідування – це один з принципів об'єктно-орієнтованого програмування, який дозволяє описати новий клас на основі існуючого [8]. При цьому властивості і функції базового класу успадковуються нащадком (рисунок 3.1).

У програмі серед мікрооб'єктів реалізовано наслідування трьох рівнів (рисунок 3.1). Перший рівень (Shopper) містить у собі основні поля та методи, які характеризують звичайного покупця музичних інструментів, такі як: name – ім'я; money – кількість грошей; xChord, yChord – координати мікрооб'єкта на екрані; isActive – показник того, чи музикант активний; isMan – стать музиканта; shopperImage – png-картинка покупця (перший графічний примітив); shadow – тінь покупця (другий графічний примітив); shopperText – надпис, який містить ім'я музиканта та кількість його грошей (третій графічний примітив); instrument – вказує на об'єкт інструмента; triangleAct – графічний показник того, що музикант активний; shopperPicture – згруповані графічні примітиви мікрооб'єкта; а також деякі інші технічні змінні.

«V» біля функції означає: «віртуальна». Основними функціями є: V void updateShopperChords() – бере поточні координати музиканта та переміщає в них кожен графічний примітив з певними наперед заданими коефіцієнтами для того, щоб картинка добре виглядала; void mouseClicked – виділяє/знімає виділення з мікрооб'єкта; void freeRun() – відповідає за постійний хаотичний рух мікрооб'єктів; void left(), void right(), void up(), void down() – рух у конкретні сторони при натискуванні клавіш; V void buyAnInstrument(), V void sellAnInstrument() – відповідає за покупку та продаж інструменту відповідно; V

Shopper education() – навчання музиканта; V void playAnInstrument() – гра на поточному інструменті.

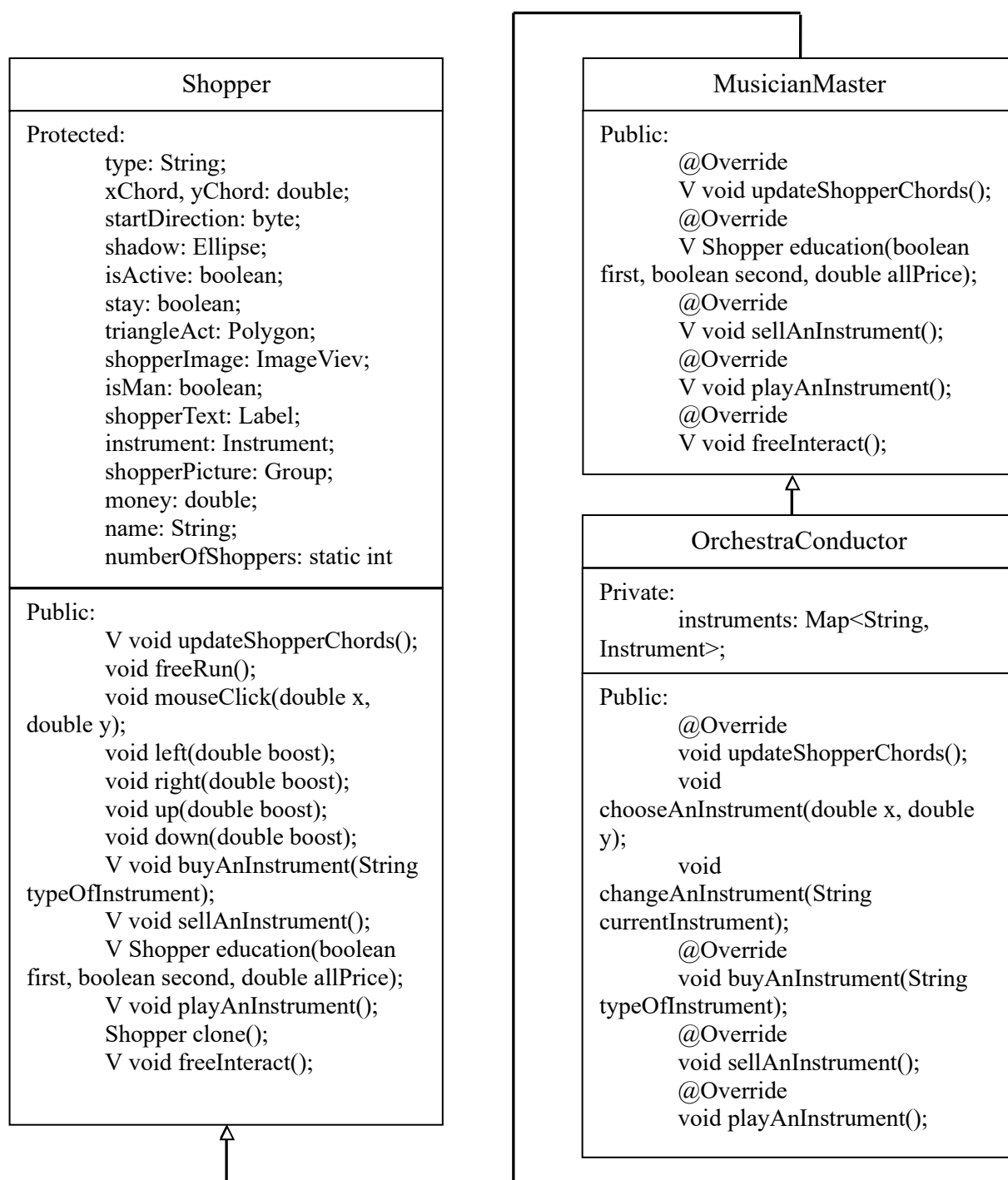


Рисунок 3.1 – Діаграма наслідування

Другий рівень (MusicianMaster) не містить у собі нових змінних та методів, але такі методи, як: V void updateShopperChords(), V Shopper education(boolean

first, boolean second, double allPrice), V void sellAnInstrument(), V void playAnInstrument() у ньому перевизначаються і набувають нового функціоналу в собі. На даному рівні ієрархії уже можна успішно продавати інструмент, а рівень гри на інструменті помітно збільшиться.

Третій рівень (OrchestraConductor) також містить перевизначені методи. У нього з'являється можливість містити у собі кілька музикальних інструментів, змінювати їх та вибирати, на якому зіграти. Рівень гри також збільшується.

### 3.2 Діаграма агрегації

Агрегація відображає співвідношення між контейнером і елементом, який в ньому може міститись, може вилучатись і додаватись. Зазвичай між контейнером і елементами співвідношення 1:N. Під час агрегації об'єкт може містити в собі посилання на інший об'єкт, але в цілому, обидва можуть існувати окремо, незалежно один від одного. У даній програмі прикладом такого зв'язку є музикант та інструмент; будівля та музикант (рисунок 3.2).

### 3.3 Діаграма композиції

Суть композиції абсолютно протилежна суті агрегації. Вона полягає у тому, що певний об'єкт є невід'ємною частиною іншого, який в свою чергу, не може існувати без попереднього. У програмі композиція реалізована у зв'язку музиканта та його тіні; музиканта та його координат X та Y; музиканта та його імені (рисунок 3.3). Неможливо створити мікрооб'єкт без імені, до того ж, кожен матиме тінь та початкові координати.

### 3.4 Діаграма асоціації

Асоціація – це вид зв'язку, при якому об'єкт містить у собі посилання на менший службовий об'єкт. Асоціація дозволяє одному об'єкту визивати та

використовувати інший об'єкт для певних цілей. У даному проекті в якості асоціації можна навести відношення між музикантом та інструментом, посилення

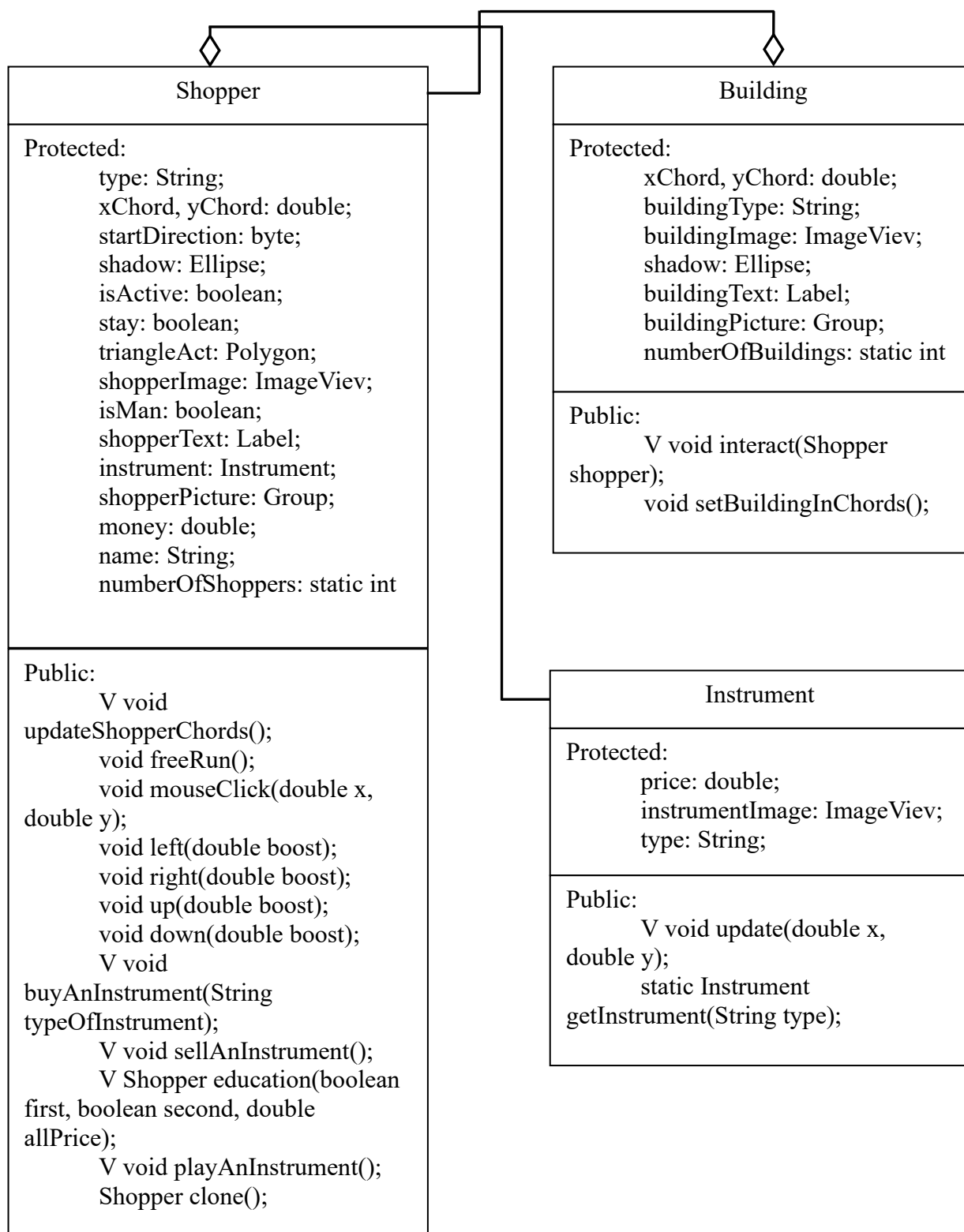


Рисунок 3.2 – Діаграми агрегації

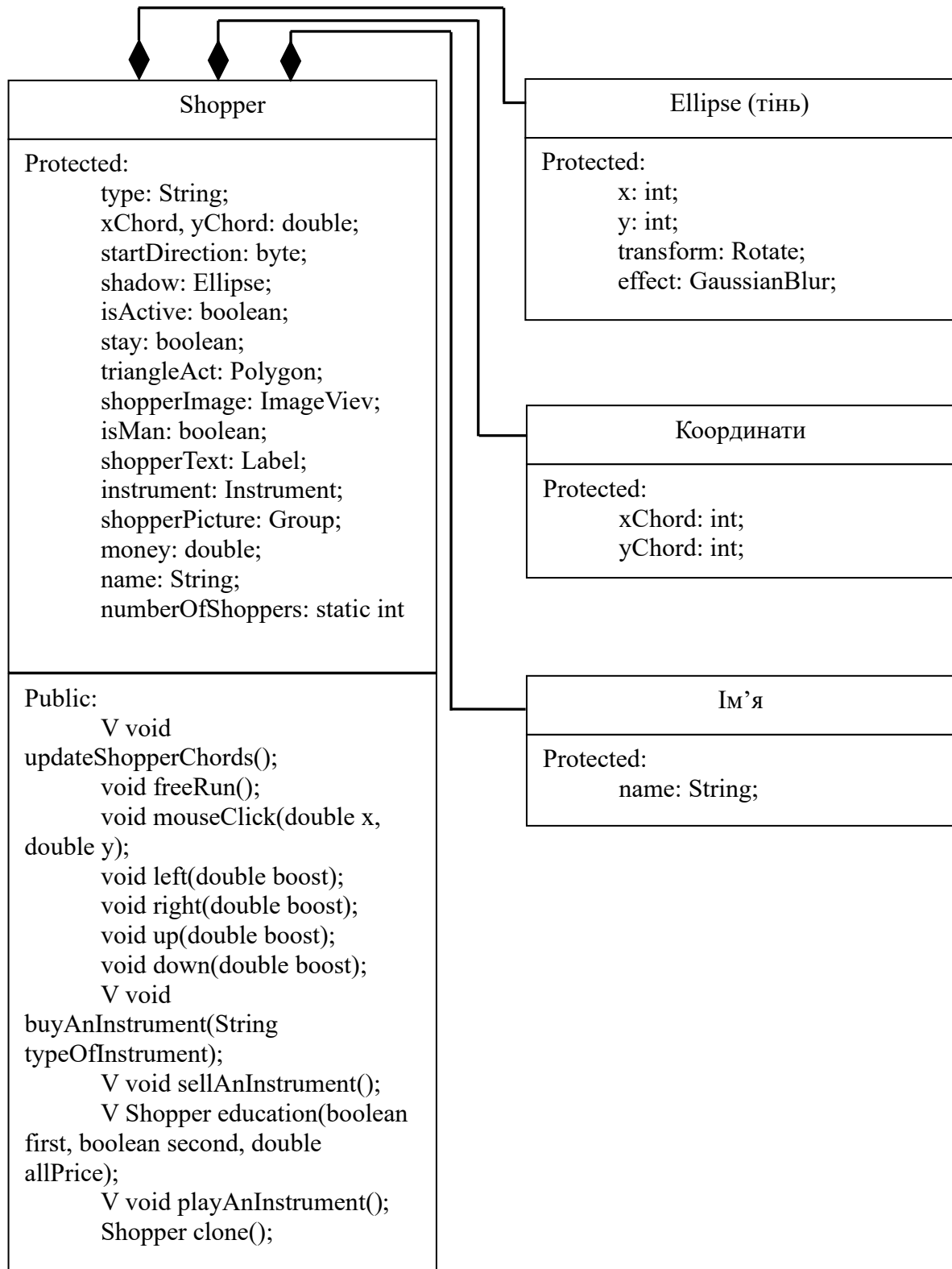


Рисунок 3.3 – Діаграма композиції

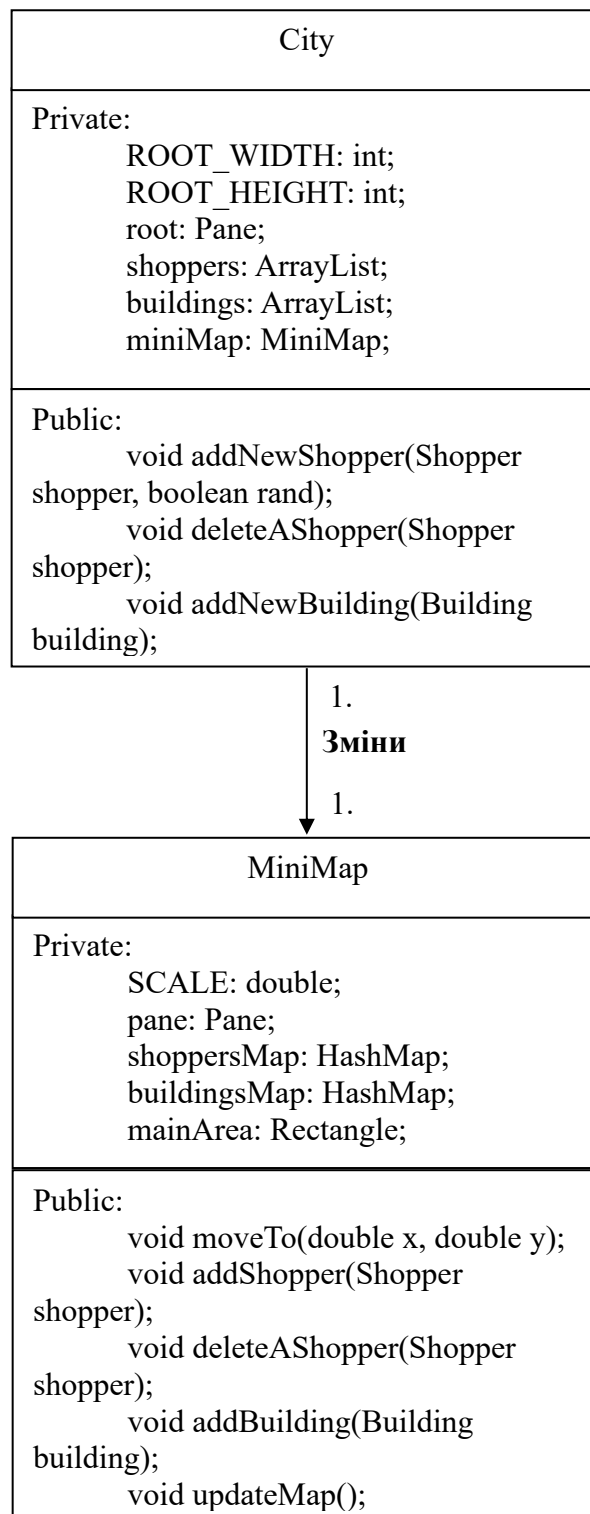


Рисунок 3.4 – Діаграма асоціації

на який зберігається в об'єкті власне музиканта, або між універсальним об'єктом місто (City) і мінікартою (MiniMap) (див. рисунок 3.4).

Універсальний об'єкт класу City взаємодіє з об'єктом класу MiniMap, забезпечуючи відображення всіх подій, які відбулись з мікро- та макрооб'єктами.

### 3.5 Діаграма кооперації

Кооперація (collaboration) - специфікація множини об'єктів, спільно взаємодіючих з метою реалізації окремих варіантів використання в загальному контексті модельованої системи. Її мета полягає в тому, щоб специфікувати особливості реалізації окремих варіантів використання або найбільш значущих операцій в системі. Кооперація визначає структуру поведінки системи в термінах взаємодії учасників цієї кооперації (рисунок 3.5).

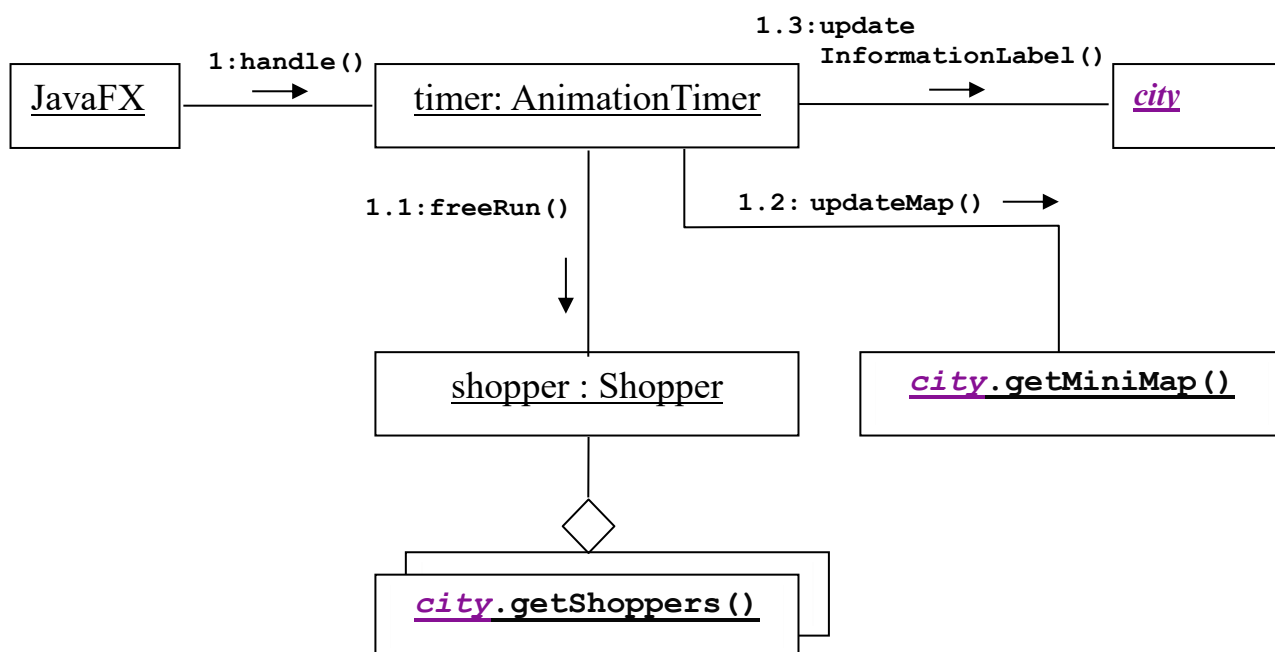


Рисунок 3.5 – Діаграма кооперації

Діаграма кооперації (див. рисунок 3.5) ілюструє обробку повідомлення таймера об'єктами універсального об'єкта **city**.

### 3.6 Діаграма послідовності

Діаграма послідовності - це тип діаграми UML, яка показує послідовність взаємодії об'єктів в системі в певному сценарії. Вона показує об'єкти та їх взаємодію один з одним через повідомлення. Діаграми послідовності



використовуються для візуалізації складних процесів та взаємодії між об'єктами в системі. Вони дозволяють розуміти, як об'єкти взаємодіють один з одним та які повідомлення вони обмінюються.

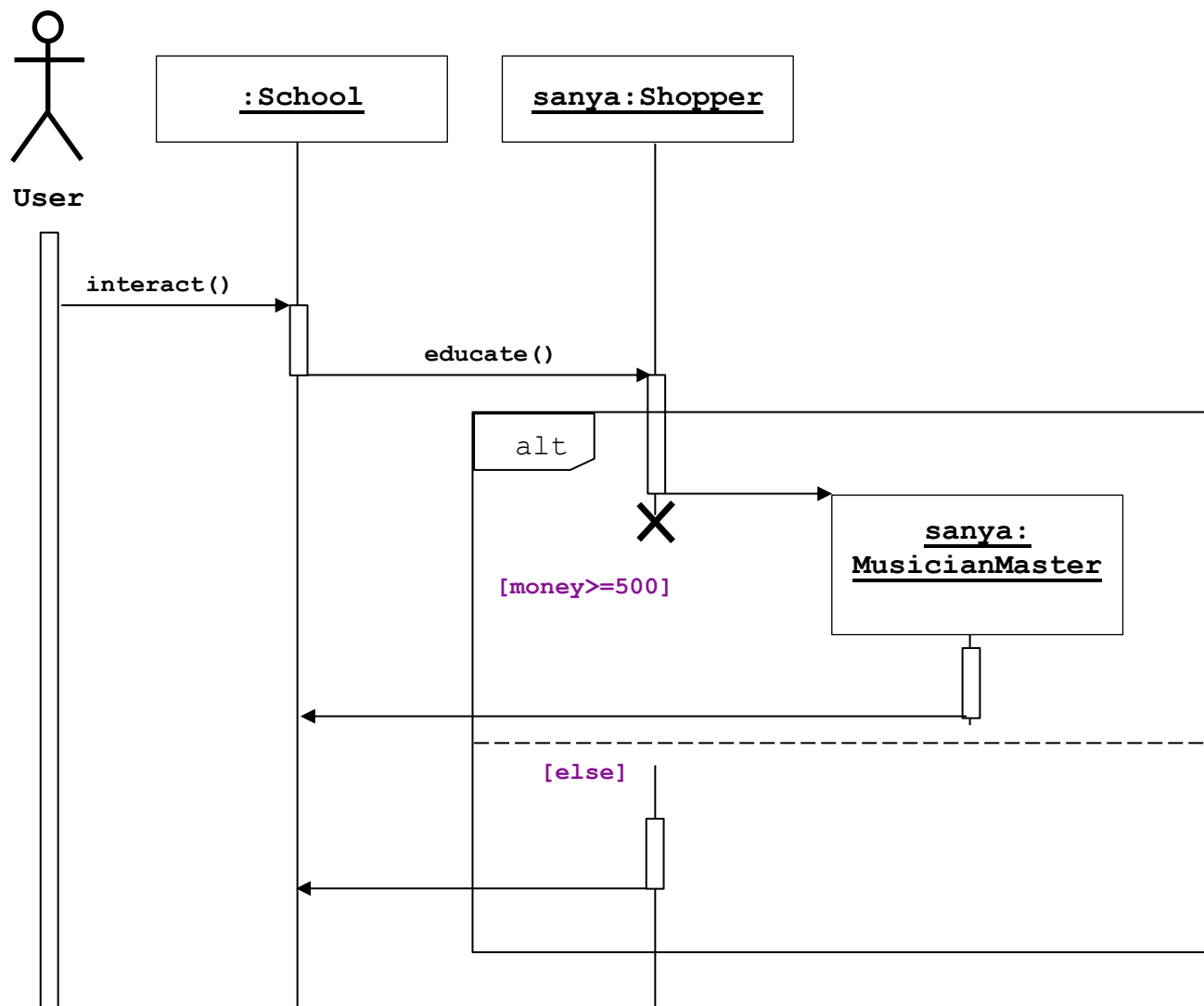


Рисунок 3.6 – Діаграма послідовності

Продемонстрована взаємодія мікрооб'єкта **Shopper** та макрооб'єкта **School** (див. рисунок 3.6), який при певних умовах може здійснити створення на його основі нового об'єкта-нащадка, який має кращі навички гри на музичних інструментах.

### 3.7 Діаграма стану

Діаграма стану (State diagram) - це діаграма UML, що використовується для візуалізації поведінки об'єкта в різних станах, та для показу переходів між цими станами. Діаграма стану складається зі станів, переходів між станами та подій, що спричиняють переходи між станами. Моделювання процесу функціонування

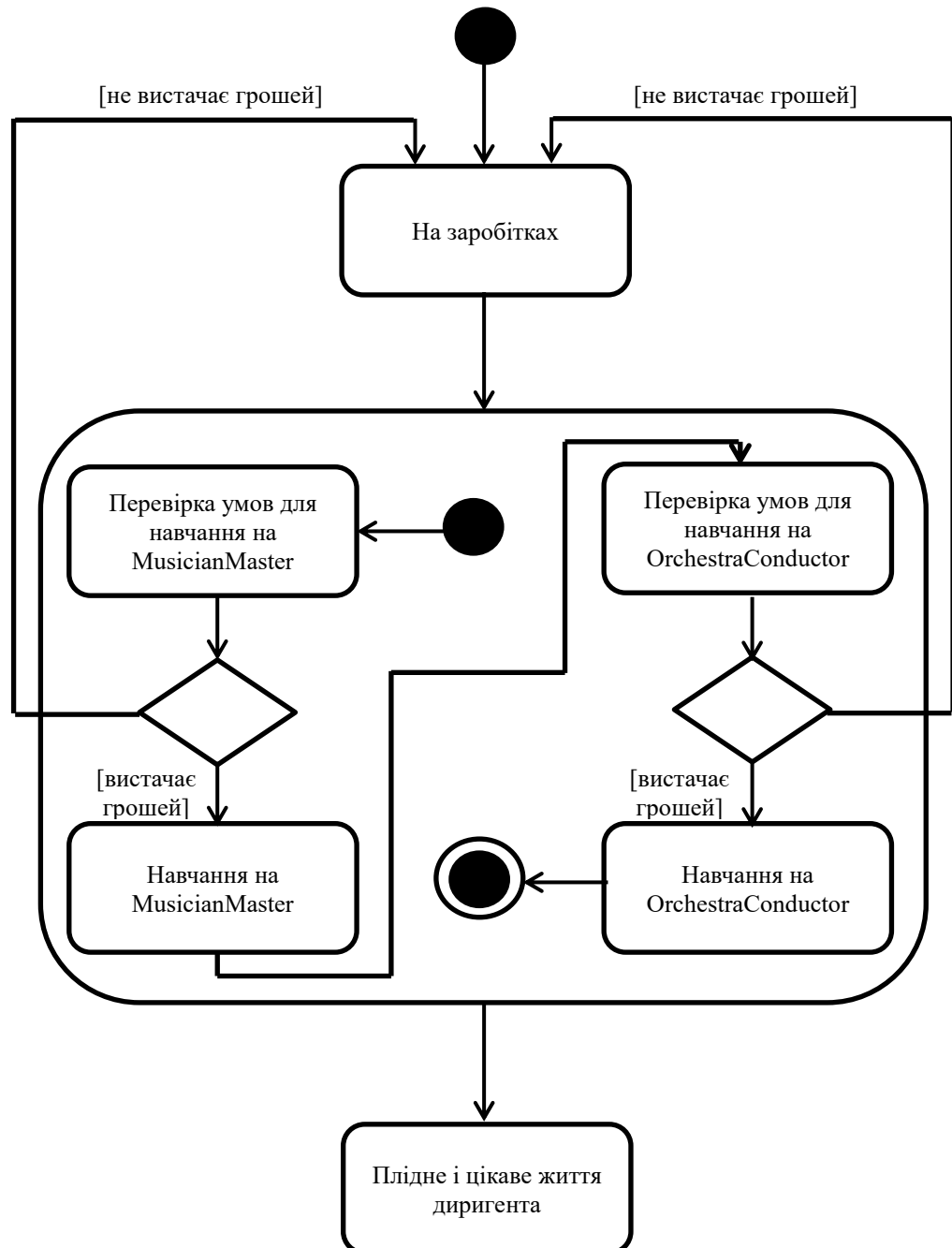


Рисунок 3.7 – Діаграма станів

складної системи, в реальному часі, зображено на рисунку 3.7 за допомогою діаграми стану.

Шлях еволюції музиканта (див. рисунок 3.7) - це шлях зростання його вміння і його наполеглива праця.

## 4 КЕРІВНИЦТВО КОРИСТУВАЧА

Створена програма не потребує інсталяції, достатньо просто скопіювати файл `name.jar` на ваш комп'ютер. Запуск здійснюється натискуванням лівої клавіші мишки по файлу двічі. Альтернативним варіантом є запуск напряму через консоль, шляхом використання скомпільованих файлів у форматі `.class`[9]. Для цього потрібно ввести в консолі команду «`java sample.Main`» (шлях до бінарних файлів `java` повинен бути доданий в змінну `PATH` операційної системи), знаходячись при цьому в каталозі, який містить пакети програми із файлами `.class`. Важливим критерієм є наявність Java актуальної версії, встановленої на вашому комп'ютері. Без неї навіть скомпільована в `jar`-архів програма не працюватиме.

Одразу після запуску програми ви побачите вікно `1920x1080`, розкрите на весь екран. Точне розташування усіх об'єктів відображає міні-карта (рисунок 4.1).

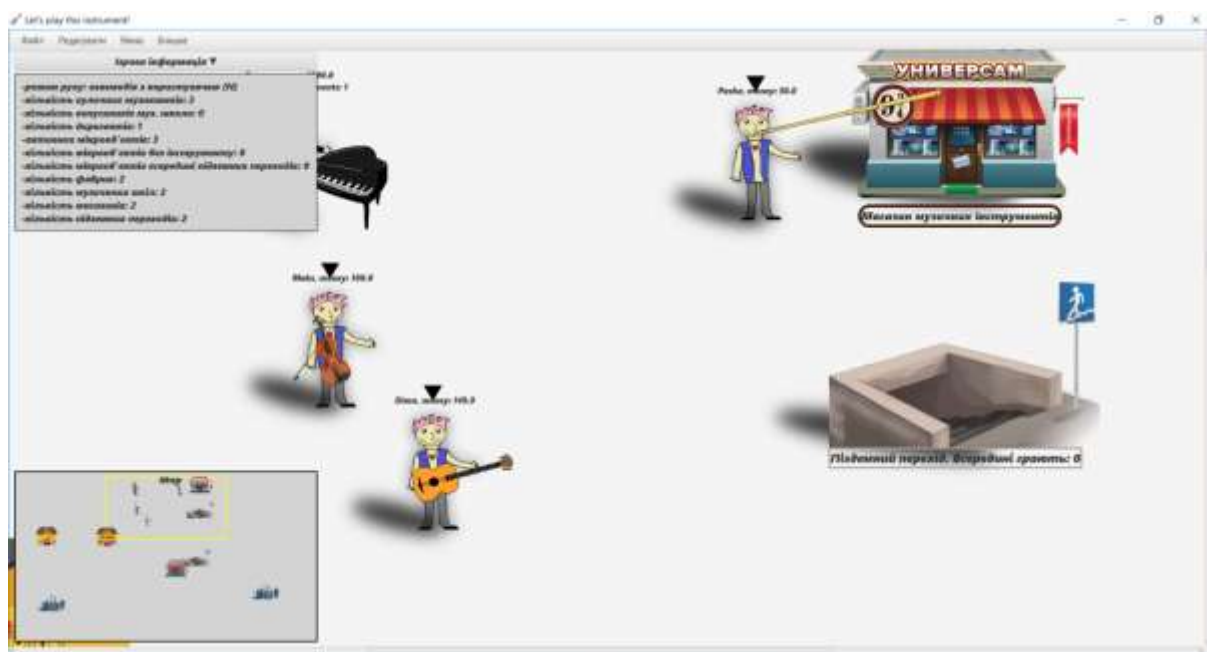


Рисунок 4.1 – Вид програми при коректному запуску

Щоб відкрити вікно для додавання нових мікрооб'єктів (рисунок 4.2), слід натиснути гарячу клавішу «`INSERT`», або ж скористатися вкладною в меню «Редагувати».



Рисунок 4.2 – Інтерфейс створення нового музиканта

Рухати активними мікрооб'єктами можна за допомогою клавіш «W», «S», «A», «D». Для взаємодії з будівлями потрібно натиснути клавішу «J», знаходячись біля потрібної будівлі. Приклад взаємодії з магазином показаний на рисунку 4.3.

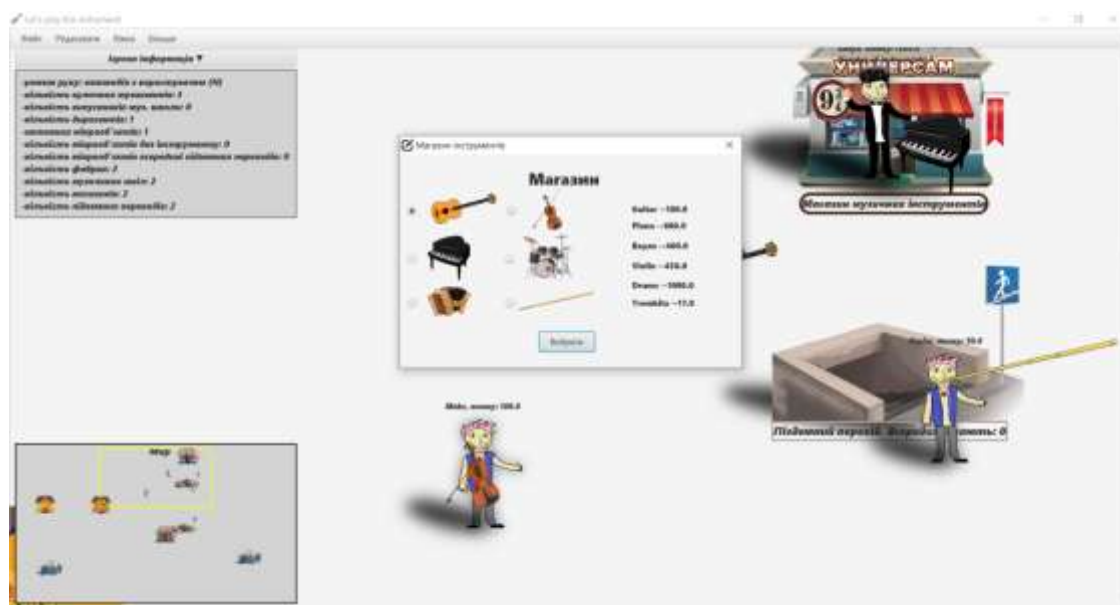


Рисунок 4.3 – Інтерфейс взаємодії з магазином

Для того, щоб мікрооб'єкт почав грати на музичному інструменті, потрібно натиснути клавішу «K», або ж скористатися меню «Редагувати». Одразу почнеться

анімація гри на, і ви почуєте музику. В залежності від рівня музиканта та його інструмента, музика буде відповідна. Також мікрооб'єкт останнього рівня може володіти декількома інструментами, та змінювати їх в будь-який момент за допомогою спеціалізованого вікна (рисунок 4.4). Для виклику цього вікна потрібно натиснути правою кнопкою мишки на останнього в ієрархії мікрооб'єкта.



Рисунок 4.4 – Зміна інструменту мікрооб'єкта

Програма також містить вікно налаштувань, де можна встановити потрібну вам гучність звуку та швидкість руху об'єктів. Також можна вимкнути відображення міні-карти та вибрати складність гри, від якої залежатимуть ціни на навчання та зарплати.

## ВИСНОВКИ

Була розроблена програма з графічним інтерфейсом на тему «Магазин музичних інструментів, музичні інструменти, покупці», яка в інтерактивному режимі відображає розвиток та становлення музиканта. В ході розробки курсової роботи було проаналізовано проблему малої зацікавленості людей мистецтвом. Були розглянуті найпопулярніші об'єктно-орієнтовані мови програмування, такі як: Java, C++, C#, висвітлені їх особливості, плюси та мінуси у порівнянні з конкурентами. Вивчені основні принципи об'єктно-орієнтованого програмування, та способи їх використання мовою Java. Була детально досліджена та вивчена бібліотека JavaFX для створення графічного інтерфейсу користувача.

В процесі роботи над курсовою роботою було отримано навички роботи у середовищі розробки «IntelliJ IDEA», вивчено основні засоби програмування мовою Java. Були набуті навички побудови та відображення основних програмних моментів за допомогою UML-діаграм. Отримано цінний досвід, необхідний при роботі з JavaFX.

## СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Розумний дім : вебсайт. URL: [https://uk.wikipedia.org/wiki/Розумний\\_дім](https://uk.wikipedia.org/wiki/Розумний_дім) (дата звернення: 25.04.2026).
2. Herbert Schildt. Java: The Complete Reference, Eleventh Edition. New York, US: McGraw Hill, 2018. 1248 p.
3. Bjarne Stroustrup. The C++ Programming Language, 4th Edition. Boston, Massachusetts: Addison-Wesley Professional, 2013. 1376 p.
4. Joseph Albahari, Ben Albahari. C# 7.0 in a Nutshell: The Definitive Reference 1st Edition. Newton, Massachusetts: O'Reilly Media, 2017. 1088 p.
5. TIOBE Index for April 2020 : вебсайт. URL: <https://www.tiobe.com/tiobe-index/> (дата звернення: 25.04.2026).
6. Порівняння C++ з мовами Java і C# та майбутній розвиток мови : вебсайт. URL:<http://cpppm12.blogspot.com/2018/05/java-i-c.html> (дата звернення: 25.04.2026).
7. Cay S. Horstmann. Core Java. Volume I. Fundamentals. 12th Edition. London: Addison-Wesley Professional, 2021. 944 p.
8. Dan Pilone, Neil Pitman. UML 2.0 in a Nutshell. Sebastopol, CA, US: O'Reilly Media, Inc., 2005. 216 p.
9. Doug Lowe. Java All-in-One For Dummies, 6th Edition. Sebastopol, CA, US: For Dummies, 2020. 912 p.



## ДОДАТОК А

### Лістинг програми (обов'язковий)

```

package sample;

import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.beans.value.ChangeListener;
import javafx.beans.value.ObservableValue;
import javafx.event.EventHandler;
import javafx.fxml.FXMLLoader;
import javafx.geometry.Bounds;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.ScrollPane;
import javafx.scene.image.Image;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.stage.Stage;
import objects.firstMacro.Guitar;
import objects.firstMacro.Piano;
import objects.firstMacro.Trembita;
import objects.firstMacro.Violin;
import objects.micro.OrchestraConductor;
import objects.micro.Shopper;
import objects.secondMacro.Factory;
import objects.secondMacro.School;
import objects.secondMacro.Shop;
import objects.secondMacro.Underpass;
import objects.thirdMacro.City;

import java.util.Arrays;
import java.util.Date;
import java.util.Random;

public class Main extends Application {

    final public static int SCENE_WIDTH = 1920;
    final public static int SCENE_HEIGHT = 1080;

    public static Random random = new Random(new Date().getTime());

    private static City city = new City();
    private static ScrollPane scrollPane = new ScrollPane(city.getRoot());
    private static Scene scene = new Scene(scrollPane, Main.SCENE_WIDTH, Main.SCENE_HEIGHT);

    private static double scrollX;
    private static double scrollY;

```

```

public static City getCity() {
    return city;
}
public static ScrollPane getScrollPane() {
    return scrollPane;
}
public static Scene getScene() {
    return scene;
}

public static double getScrollX() {
    return scrollX;
}
public static double getScrollY() {
    return scrollY;
}

@Override
public void start(Stage primaryStage) throws Exception {

    scrollPane.setFitToHeight(true);
    scrollPane.setFitToWidth(true);

    city.addNewBuilding(new Factory());
    city.addNewBuilding(new Factory());
    city.addNewBuilding(new Shop());
    city.addNewBuilding(new Shop());
    city.addNewBuilding(new School());
    city.addNewBuilding(new School());
    city.addNewBuilding(new Underpass());
    city.addNewBuilding(new Underpass());

    city.addNewShopper(new Shopper(new Violin(),false, "Maks", 100, true),true);
    city.addNewShopper(new Shopper(new Trembita(),false, "Pasha", 50, true),true);
    city.addNewShopper(new Shopper(new Guitar(),false, "Dima", 149, true),true);
    city.addNewShopper(new OrchestraConductor(new Piano(), false, "Sanya", 1000),true);

    Parent parent = FXMLLoader.load(getClass().getResource("sample.fxml"));
    city.getRoot().getChildren().add(parent);

    scrollPane.viewportBoundsProperty().addListener(new ChangeListener<Bounds>() {
        @Override
        public void changed(ObservableValue<? extends Bounds>
            observable, Bounds oldBounds,
            Bounds bounds) {
            double scrollWidth;
            double scrollHeight;
            Main.scrollX = -1 * (int) bounds.getMinX();
            scrollWidth = -1 * (int) bounds.getMinX() + (int) bounds.getWidth();
            Main.scrollY = -1 * (int) bounds.getMinY();
            scrollHeight = -1 * (int) bounds.getMinY() + bounds.getHeight();
        }
    });

```

```

        //постійний зсув стрічки меню при прокручуванні
        parent.setLayoutX(scrollX);
        parent.setLayoutY(scrollY);

        // постійни зсув карти при прокручуванні
        city.getMiniMap().getPane().setLayoutX(scrollX + 10);
        city.getMiniMap().getPane().setLayoutY(scrollY + scene.getHeight() - city.getMiniMap().getPane().getHeight() - 25);
        city.getMiniMap().getMainArea().setLayoutX(scrollX*MiniMap.getSCALE());
        city.getMiniMap().getMainArea().setLayoutY(scrollY*MiniMap.getSCALE());

        city.getInformationGroup().setLayoutX(scrollX);
        city.getInformationGroup().setLayoutY(scrollY);
        /* city.getInteractWithPlayerModeLabel().setLayoutX(scrollX+10);
        city.getInteractWithPlayerModeLabel().setLayoutY(scrollY+scene.getHeight()-city.getMiniMap().getPane().getHeight() - 50);
*/

        /*//просто показує координати в даний момент
        System.out.println(" X from " + Main.scrollX + " to " +
            scrollWidth + "; Y from " + Main.scrollY + " to " +
            scrollHeight);*/
    }
});

primaryStage.setMaximized(true);
primaryStage.setScene(scene);
primaryStage.setTitle("Let's play this instrument!");
primaryStage.getIcons().add(new Image("assets/icon.png"));
scene.setOnKeyPressed(new KeyPressedHandler());

AnimationTimer timer = new AnimationTimer() {
    @Override
    public void handle(long now) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.freeRun();
            shopper.setShopperInChords();
        }
        city.getMiniMap().updateMap();
        city.updateInformationLabel();
    }
};
timer.start();
primaryStage.show();
}

private class KeyPressedHandler implements EventHandler<KeyEvent> {

    //З ПИТАННЯМИ, ЧОМУ ТУТ ПОЛОВИНА КОДУ ВИКЛЮЧЕНА, ЗВЕРТАТИСЯ ДО МЕНЕ!!!
    @Override
    public void handle(KeyEvent event) {
        /*if (event.getCode().equals(KeyCode.M)) {
            try {
                new CreateShopperWindow().display("Let's create new Shopper!");
            } catch (IOException e) {

```

```

        e.printStackTrace();
    }
}
if (event.getCode().equals(KeyCode.DELETE)) {
    for (int i = 0; i < shoppers.size(); i++) {
        Shopper shopper = shoppers.get(i);
        if (shopper.isActive()) {
            Main.deleteAShopper(shopper);
            i--;
        }
    }
    System.out.println(Shopper.getNumberOfShoppers() + "----");
    System.out.println(shoppers.size());
}
if (event.getCode().equals(KeyCode.ESCAPE)) {
    for (Shopper shopper : shoppers) {
        if (shopper.isActive()) {
            shopper.setActive(false);
        }
    }
}
if (event.getCode().equals(KeyCode.J)) {
    for (Shopper shopper : shoppers) {
        if (shopper.isActive()) {
            for (Building b : buildings) {
                b.interact(shopper, event.isShiftDown());
            }
        }
    }
}
if (event.getCode().equals(KeyCode.K)) {
    for (Shopper shopper : shoppers) {
        if (shopper.isActive() && shopper.getInstrument() != null) {
            shopper.playAnInstrument();
            break;
        }
    }
}
}*/
if (event.getCode().equals(KeyCode.X)){
    Shopper[] shoppersArray = Main.getCity().getShoppers().toArray(new Shopper[0]);
    System.out.println("Відсортовані за кількістю грошей:");
    Arrays.sort(shoppersArray, (o1,o2) -> (int) (o1.getMoney()-o2.getMoney()));

    for (Shopper shopper: shoppersArray){
        System.out.println(shopper.getName());
    }
    System.out.println("\nВідсортовані за ім'ям:");
    Arrays.sort(shoppersArray, (o1, o2) -> o1.getName().compareTo(o2.getName()));
    for (Shopper shopper: shoppersArray){
        System.out.println(shopper.getName());
    }
    System.out.println(Arrays.toString(shoppersArray));
}

```

```

    }
    if (event.getCode().equals(KeyCode.W) && !event.isShiftDown()) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.up(1);
        }
    }
    } else if (event.getCode().equals(KeyCode.S) && !event.isShiftDown()) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.down(1);
        }
    }
    } else if (event.getCode().equals(KeyCode.D) && !event.isShiftDown()) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.right(1);
        }
    }
    } else if (event.getCode().equals(KeyCode.A) && !event.isShiftDown()) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.left(1);
        }
    }
    } else if (event.getCode().equals(KeyCode.W)) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.up(2);
        }
    }
    } else if (event.getCode().equals(KeyCode.S)) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.down(2);
        }
    }
    } else if (event.getCode().equals(KeyCode.D)) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.right(2);
        }
    }
    } else if (event.getCode().equals(KeyCode.A)) {
        for (Shopper shopper : city.getShoppers()) {
            shopper.left(2);
        }
    }
    }
}

public static void main(String[] args) {
    launch(args);
}
}

```

## Лістинг модуля Controller.java

```
package sample;

import javafx.application.Platform;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.ButtonType;
import javafx.scene.control.MenuItem;
import javafx.scene.control.RadioMenuItem;
import javafx.scene.input.KeyCombination;
import javafx.stage.FileChooser;
import objects.micro.OrchestraConductor;
import objects.micro.Shopper;
import objects.secondMacro.Building;
import sample.windows.aboutWindow.AboutWindow;
import sample.windows.createShopperWindow.CreateShopperWindow;
import sample.windows.preferencesWindow.PreferencesWindow;
import java.io.*;
import java.util.Optional;

public class Controller {

    @FXML
    private MenuItem menuFileOpen;

    @FXML
    private MenuItem menuFileSave;

    @FXML
    private MenuItem menuFileExit;

    @FXML
    private RadioMenuItem menuEditRadioFastRun;

    @FXML
    private MenuItem menuEditInteract;

    @FXML
    private MenuItem menuEditPlay;

    @FXML
    private MenuItem menuEditSell;

    @FXML
    private MenuItem menuEditCreate;
```

```

@FXML
private MenuItem menuEditClone;

@FXML
private MenuItem menuEditDelete;

@FXML
private MenuItem menuEditCancel;

@FXML
private MenuItem menuEditProperties;

@FXML
private MenuItem menuWindowsCreate;

@FXML
private MenuItem menuWindowsProperties;

@FXML
private MenuItem menuMoreAbout;

@FXML
void initialize() {
    EventHandler<ActionEvent> menuHandler = new EventHandler<ActionEvent>() {
        @Override
        public void handle(ActionEvent event) {
            String buttonName = ((MenuItem)event.getTarget()).getText();
            if (buttonName.equals("Зберегти")){
                FileChooser fileChooser = new FileChooser();
                fileChooser.setTitle("Виберіть місце для збереження");
                fileChooser.getExtensionFilters().addAll(new FileChooser.ExtensionFilter("XML-збереження", "*.xml"),
                    new FileChooser.ExtensionFilter("TXT-збереження", "*.txt"),
                    new FileChooser.ExtensionFilter("Усі файли", "*.*"));
                File file = fileChooser.showSaveDialog(Main.getScene().getWindow());
                if (file != null)
                    Serialization.serializeNow(file);
            } else if (buttonName.equals("Відкрити")){

                Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
                alert.setHeaderText("Ви дійсно хочете це зробити?");
                alert.setContentText("Переконайтеся, що поточна гра збережена");

                Optional<ButtonType> option = alert.showAndWait();
                if (option.get() == ButtonType.OK) {
                    FileChooser fileChooser = new FileChooser();
                    fileChooser.setTitle("Виберіть файл");
                    fileChooser.getExtensionFilters().addAll(new FileChooser.ExtensionFilter("XML-збереження", "*.xml"),
                        new FileChooser.ExtensionFilter("TXT-збереження", "*.txt"),
                        new FileChooser.ExtensionFilter("Усі файли", "*.*"));
                    File file = fileChooser.showOpenDialog(Main.getScene().getWindow());
                    if (file != null)
                        Serialization.deserializeNow(file);
                }
            }
        }
    };
}

```

```

    }

    else if (buttonName.equals("Вийти")){
        Platform.exit();
    } else if (buttonName.equals("Взаємодія з будівлею")){
        for (Shopper shopper : Main.getCity().getShoppers()) {
            if (shopper.isActive()) {
                for (Building b : Main.getCity().getBuildings()) {
                    b.interact(shopper, false);
                }
            }
        }
    }
    } else if (buttonName.equals("Продати (тільки магазин)")) {
        for (Shopper shopper : Main.getCity().getShoppers()) {
            if (shopper.isActive()) {
                for (Building b : Main.getCity().getBuildings()) {
                    b.interact(shopper, true);
                }
            }
        }
    }
    } else if (buttonName.equals("Грати на інструменті")){
        for (Shopper shopper : Main.getCity().getShoppers()) {
            if (shopper.isActive()) {
                if (shopper.getInstrument() != null || (shopper instanceof OrchestraConductor)) {
                    shopper.playAnInstrument();
                    break;
                } else {
                    Alert alert = new Alert(Alert.AlertType.INFORMATION);
                    alert.setHeaderText("УВАГА");
                    alert.setContentText("Переконайтеся у наявності інструменту");
                    alert.showAndWait();
                }
            }
        }
    }
    } else if (buttonName.equals("Створити персонажа")) {
        try {
            new CreateShopperWindow().display();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    } else if (buttonName.equals("Клонувати одного з вибраних")){
        for (Shopper shopper : Main.getCity().getShoppers()) {
            if (shopper.isActive()) {
                try {
                    Shopper clone = shopper.clone();
                    clone.setXChord(shopper.getXChord()+200);
                    Main.getCity().addNewShopper(clone, false);
                } catch (CloneNotSupportedException e) {
                    Alert alert = new Alert(Alert.AlertType.INFORMATION);
                    alert.setContentText("Помилка клонування");
                    alert.showAndWait();
                }
            }
        }
    }

```



```

        break;
    }
}
} else if (buttonName.equals("Видалити персонажей")){
    for (int i = 0; i < Main.getCity().getShoppers().size(); i++) {
        Shopper shopper = Main.getCity().getShoppers().get(i);
        if (shopper.isActive()) {
            Main.getCity().deleteAShopper(shopper);
            i--;
        }
    }
    System.out.println(Shopper.getNumberOfShoppers()+"----");
    System.out.println(Main.getCity().getShoppers().size());
} else if (buttonName.equals("Відмінити вибір")){
    for(Shopper s: Main.getCity().getShoppers()){
        if (s.isActive()) {
            s.setActive(false);
        }
    }
} else if (buttonName.equals("Налаштування")){
    try {
        new PreferencesWindow().display();
    } catch (IOException e) {
        e.printStackTrace();
    }
} else if (buttonName.equals("Створення нового персонажа")){
    try {
        new CreateShopperWindow().display();
    } catch (IOException e) {
        e.printStackTrace();
    }
} else if (buttonName.equals("Вікно налаштувань")){
    try {
        new PreferencesWindow().display();
    } catch (IOException e) {
        e.printStackTrace();
    }
} else if (buttonName.equals("Про програму")){
    try {
        new AboutWindow().display();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
};
menuEditRadioFastRun.setOnAction(event -> {
    if (menuEditRadioFastRun.isSelected()){
        Main.getCity().setInteractWithPlayerMode(true);
    } else {
        Main.getCity().setInteractWithPlayerMode(false);
    }
}

```

```

});
menuEditRadioFastRun.setSelected(true);

menuFileOpen.setOnAction(menuHandler);
menuFileOpen.setAccelerator(KeyCombination.keyCombination("Ctrl+O"));
menuFileSave.setOnAction(menuHandler);
menuFileSave.setAccelerator(KeyCombination.keyCombination("Ctrl+S"));
menuFileExit.setOnAction(menuHandler);
menuFileExit.setAccelerator(KeyCombination.keyCombination("Ctrl+E"));
menuEditRadioFastRun.setAccelerator(KeyCombination.keyCombination("N"));

menuEditInteract.setOnAction(menuHandler);
menuEditInteract.setAccelerator(KeyCombination.keyCombination("J"));
menuEditSell.setOnAction(menuHandler);
menuEditSell.setAccelerator(KeyCombination.keyCombination("J+shift"));
menuEditPlay.setOnAction(menuHandler);
menuEditPlay.setAccelerator(KeyCombination.keyCombination("K"));
menuEditCreate.setOnAction(menuHandler);
menuEditCreate.setAccelerator(KeyCombination.keyCombination("M"));
menuEditClone.setOnAction(menuHandler);
menuEditClone.setAccelerator(KeyCombination.keyCombination("C"));
menuEditDelete.setOnAction(menuHandler);
menuEditDelete.setAccelerator(KeyCombination.keyCombination("Delete"));
menuEditCancel.setOnAction(menuHandler);
menuEditCancel.setAccelerator(KeyCombination.keyCombination("Esc"));
menuEditProperties.setOnAction(menuHandler);
menuEditProperties.setAccelerator(KeyCombination.keyCombination("Ctrl+Alt+S"));
menuWindowsCreate.setOnAction(menuHandler);
menuWindowsCreate.setAccelerator(KeyCombination.keyCombination("M"));
menuWindowsProperties.setOnAction(menuHandler);
menuWindowsProperties.setAccelerator(KeyCombination.keyCombination("Ctrl+Alt+S"));
menuMoreAbout.setOnAction(menuHandler);
menuMoreAbout.setAccelerator(KeyCombination.keyCombination("f1"));
}
}

```

## Лістинг модуля sample.fxml

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Menu?>
<?import javafx.scene.control.MenuBar?>
<?import javafx.scene.control.MenuItem?>
<?import javafx.scene.control.RadioMenuItem?>
<?import javafx.scene.layout.BorderPane?>

<BorderPane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="25.0" prefWidth="1920.0"
xmlns="http://javafx.com/javafx/11.0.1" xmlns:fx="http://javafx.com/fxml/1" fx:controller="sample.Controller">
  <top>
    <MenuBar BorderPane.alignment="CENTER">
      <menus>
        <Menu mnemonicParsing="false" text="Файл">
          <items>
            <MenuItem fx:id="menuFileOpen" mnemonicParsing="false" text="Відкрити" />
            <MenuItem fx:id="menuFileSave" mnemonicParsing="false" text="Зберегти" />
            <MenuItem fx:id="menuFileExit" mnemonicParsing="false" text="Вийти" />
          </items>
        </Menu>
        <Menu mnemonicParsing="false" text="Редагувати">
          <items>
            <RadioMenuItem fx:id="menuEditRadioFastRun" mnemonicParsing="false" text="Режим взаємодії з користувачем" />
            <MenuItem fx:id="menuEditInteract" mnemonicParsing="false" text="Взаємодія з будівлею" />
            <MenuItem fx:id="menuEditSell" mnemonicParsing="false" text="Продати (тільки магазин)" />
            <MenuItem fx:id="menuEditPlay" mnemonicParsing="false" text="Грати на інструменті" />
            <MenuItem fx:id="menuEditCreate" mnemonicParsing="false" text="Створити персонажа" />
            <MenuItem fx:id="menuEditClone" mnemonicParsing="false" text="Клонувати одного з вибраних" />
            <MenuItem fx:id="menuEditDelete" mnemonicParsing="false" text="Видалити персонажей" />
            <MenuItem fx:id="menuEditCancel" mnemonicParsing="false" text="Відмінити вибір" />
            <MenuItem fx:id="menuEditProperties" mnemonicParsing="false" text="Налаштування" />
          </items>
        </Menu>
        <Menu mnemonicParsing="false" text="Вікна">
          <items>
            <MenuItem fx:id="menuWindowsCreate" mnemonicParsing="false" text="Створення нового персонажа" />
            <MenuItem fx:id="menuWindowsProperties" mnemonicParsing="false" text="Вікно налаштувань" />
          </items>
        </Menu>
        <Menu mnemonicParsing="false" text="Більше">
          <items>
            <MenuItem fx:id="menuMoreAbout" mnemonicParsing="false" text="Про програму" />
          </items>
        </Menu>
      </menus>
    </MenuBar>
  </top>
</BorderPane>

```

## Лістинг модуля MiniMap.java

```

package sample;

import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import objects.micro.Shopper;
import objects.secondMacro.Building;
import objects.thirdMacro.City;
import sample.windows.preferencesWindow.Preferences;

import java.util.HashMap;

public class MiniMap {
    final static private double SCALE = 0.1;
    private Pane pane;
    private HashMap<Shopper, ImageView> shoppersMap;
    private HashMap<Building, ImageView> buildingsMap;
    private Rectangle mainArea;

    public MiniMap() {
        this.pane = new Pane();
        this.pane.setMinWidth(City.ROOT_WIDTH * MiniMap.SCALE);
        this.pane.setMinHeight(City.ROOT_HEIGHT * MiniMap.SCALE);
        shoppersMap = new HashMap<>();
        buildingsMap = new HashMap<>();

        Rectangle rectangle = new Rectangle(0, 0, pane.getMinWidth(), pane.getMinHeight());
        rectangle.setFill(Color.LIGHTGREY);
        rectangle.setStroke(Color.BLACK);
        rectangle.setStrokeWidth(2);
        Label label = new Label("Map");
        label.setFont(new Font("Segoe UI Black Italic", 16));
        label.setLayoutX(pane.getMinWidth() / 2.1);

        mainArea = new Rectangle(0, 0, Main.SCENE_WIDTH * MiniMap.SCALE, Main.SCENE_HEIGHT * MiniMap.SCALE);
        mainArea.setFill(Color.TRANSPARENT);
        mainArea.setStrokeWidth(2);
        mainArea.setStroke(Color.YELLOW);
        this.pane.getChildren().addAll(rectangle, label, mainArea);

        this.pane.setOnMousePressed(event -> {
            this.moveTo(event.getX(), event.getY());
        });
    }
}

```

```

public void moveTo(double x, double y) {
    if (x < mainArea.getWidth() / 2) {
        Main.getScrollPane().setHvalue(0);
    } else if (x > pane.getWidth() - mainArea.getWidth() / 2) {
        Main.getScrollPane().setHvalue(1);
    } else Main.getScrollPane().setHvalue(x / pane.getWidth());

    if (y < mainArea.getHeight() / 2) {
        Main.getScrollPane().setVvalue(0);
    } else if (y > pane.getHeight() - mainArea.getHeight() / 2) {
        Main.getScrollPane().setVvalue(1);
    } else Main.getScrollPane().setVvalue(y / pane.getHeight());
}

public Pane getPane() {
    return pane;
}

public Rectangle getMainArea() {
    return mainArea;
}

public static double getSCALE() {
    return SCALE;
}

public void addShopper(Shopper shopper) {
    ImageView imageView = new ImageView(shopper.getShopperImage().getImage());
    imageView.setLayoutX(shopper.getXChord() * MiniMap.SCALE);
    imageView.setLayoutY(shopper.getYChord() * MiniMap.SCALE);
    imageView.setPreserveRatio(true);
    imageView.setFitHeight(190 * MiniMap.SCALE);
    shoppersMap.put(shopper, imageView);
    pane.getChildren().add(imageView);
}

public void deleteAShopper(Shopper shopper) {
    pane.getChildren().remove(shoppersMap.get(shopper));
    shoppersMap.remove(shopper);
}

public void addBuilding(Building building) {
    ImageView imageView = new ImageView(building.getBuildingImage().getImage());
    imageView.setPreserveRatio(true);
    imageView.setFitHeight(building.getBuildingImage().getFitHeight() * MiniMap.SCALE);
    imageView.setLayoutX(building.getXChord() * MiniMap.SCALE);
    imageView.setLayoutY(building.getYChord() * MiniMap.SCALE);

    buildingsMap.put(building, imageView);
    pane.getChildren().add(imageView);
}

```

```
public void deleteABuilding(Building building) {
    pane.getChildren().remove(buildingsMap.get(building));
    buildingsMap.remove(building);
}

public void updateMap() {
    if (Preferences.isMAP()) {
        pane.setOpacity(1);
        mainArea.setWidth(Main.getScene().getWidth()* MiniMap.SCALE);
        mainArea.setHeight(Main.getScene().getHeight()*MiniMap.SCALE);
    } else pane.setOpacity(0);
    for (Shopper shopper : Main.getCity().getShoppers()) {
        ImageView imageView = shoppersMap.get(shopper);
        imageView.setLayoutX(shopper.getXChord() * MiniMap.SCALE);
        imageView.setLayoutY(shopper.getYChord() * MiniMap.SCALE);
    }
}
}
```

## Лістинг модуля Serialization.java

```

package sample;
import objects.micro.Shopper;
import objects.secondMacro.Building;
import java.beans.XMLDecoder;
import java.beans.XMLEncoder;

public class Serialization {

    public static void serializeNow(File file) {
        XMLEncoder encoder;
        try {
            encoder = new XMLEncoder(new BufferedOutputStream(new FileOutputStream(file)));

            HashMap<String, Object> hashMap = new HashMap<>();
            hashMap.put("shoppers", Main.getCity().getShoppers());
            hashMap.put("buildings", Main.getCity().getBuildings());
            encoder.writeObject(hashMap);
            encoder.close();
        } catch (FileNotFoundException e) {
            System.out.println("Помилка відкриття файлу");
        }
    }

    public static void deserializeNow(File file){
        XMLDecoder decoder;
        try {
            decoder=new XMLDecoder(new BufferedInputStream(new FileInputStream(file)));

            for (int i = 0; i<Main.getCity().getShoppers().size(); i++){
                Shopper shopper = Main.getCity().getShoppers().get(i--);
                Main.getCity().deleteAShopper(shopper);
            }
            for (int i = 0; i<Main.getCity().getBuildings().size(); i++){
                Building building = Main.getCity().getBuildings().get(i--);
                Main.getCity().deleteABuilding(building);
            }
            HashMap<String, Object> hashMap = (HashMap<String, Object>)decoder.readObject();

            for (Building building: (ArrayList<Building>)hashMap.get("buildings")){
                Main.getCity().addNewBuilding(building);
            }
            for (Shopper shopper:(ArrayList<Shopper>)hashMap.get("shoppers")){
                Main.getCity().addNewShopper(shopper, false);
            }
            decoder.close();
        } catch (FileNotFoundException e) {
            System.out.println("Помилка відкриття файлу");
        }
    }
}

```

## Лістинг модуля Sprite.java

```

package sample;

import javafx.animation.Transition;
import javafx.geometry.Rectangle2D;
import javafx.scene.image.ImageView;
import javafx.util.Duration;

public class Sprite extends Transition {
    private boolean loadReady = false;
    final private ImageView imageView;
    final private int count;
    final private int columns;
    final private int width;
    final private int height;

    public Sprite(ImageView imageView, Duration duration, int count, int columns, int width, int height) {
        this.imageView = imageView;
        this.imageView.setOpacity(0);
        this.count = count;
        this.columns = columns;
        this.width = width;
        this.height = height;
        setCycleDuration(duration);
    }

    //Виглядає складно))))))
    @Override
    protected void interpolate(double frac) {
        int index = Math.min((int) Math.floor(frac * count), count - 1);
        int x = (index % columns) * width;
        int y = (index / columns) * height;
        imageView.setViewport(new Rectangle2D(x, y, width, height));
        if (!loadReady) {
            this.imageView.setOpacity(1); //це для того, щоб не було багу з повною картинкою спрайту
            loadReady = !loadReady;
        }
    }
}

```



## Лістинг модуля Shopper.java

```
package objects.micro;

import javafx.animation.Animation;
import javafx.geometry.Point2D;
import javafx.scene.Group;
import javafx.scene.control.Alert;
import javafx.scene.control.Label;
import javafx.scene.effect.*;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.media.AudioClip;
import javafx.scene.media.Media;
import javafx.scene.media.MediaException;
import javafx.scene.paint.Color;
import javafx.scene.shape.Ellipse;
import javafx.scene.shape.Polygon;
import javafx.scene.text.Font;
import javafx.scene.transform.Rotate;
import javafx.util.Duration;
import objects.firstMacro.Instrument;
import objects.secondMacro.Building;
import sample.Main;
import sample.Sprite;
import sample.windows.preferencesWindow.Preferences;

import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;

public class Shopper implements Cloneable{

    protected double playVariable;
    protected String type;
    protected double xChord, yChord;
    protected byte startDirection;

    protected Ellipse shadow;

    protected boolean isActive;
    protected boolean stay;
    protected Polygon triangleAct;

    protected ImageView shopperImage;
    protected boolean isMan;
    protected Label shopperText;
```

```

protected Instrument instrument = null;
protected Group shopperPicture;

protected double money;
protected String name;
protected static int numberOfShoppers = 0;

public Shopper(Instrument instrument, boolean isActive, String name, double money, boolean isMan) {
    Shopper.numberOfShoppers++;
    this.type = "Shopper";
    this.playVariable = 0.1;
    this.name = name;
    this.money = money;
    this.isMan = isMan;
    this.startDirection = (byte) Main.random.nextInt(8);

    if (isMan) {
        this.shopperImage = new ImageView(new Image("assets/shopper.png"));
    } else this.shopperImage = new ImageView(new Image("assets/girlShopper.png"));

    this.isActive = isActive;
    this.stay = false;
    this.shopperImage.setPreserveRatio(true);
    this.shopperImage.setFitHeight(190);
    this.shopperImage.setEffect(new DropShadow(30, 0, 0, Color.GRAY));

    this.instrument = instrument;

    this.shopperText = new Label(/*this.name + ", money: " + this.money*/);
    this.shopperText.setFont(new Font("Segoe UI Black Italic", 13));

    this.shadow = new Ellipse(75, 20);
    this.shadow.setFill(Color.BLACK);
    this.shadow.setOpacity(0.8);
    this.shadow.getTransforms().add(new Rotate(15));
    this.shadow.setEffect(new GaussianBlur(40));

    this.triangleAct = new Polygon();
    triangleAct.getPoints().addAll(0.0, 0.0,
        30.0, 0.0,
        15.0, 25.0);
    this.triangleAct.setFill(Color.BLACK);

    if (this.instrument != null) {
        this.shopperPicture = new Group(shopperImage, shopperText, shadow, triangleAct, this.instrument.getInstrumentImage());
    } else {
        this.shopperPicture = new Group(shopperImage, shopperText, shadow, triangleAct);
    }
}

```

```
public Shopper(boolean isActive, String name, double money, boolean isMan) {
    this(null, isActive, name, money, isMan);
}
```

```
public Shopper(Instrument instrument, boolean isActive, String name) {
    this(instrument, isActive, name, 0, true);
}
```

```
public Shopper(boolean isActive, String name) {
    this(null, isActive, name, 0, true);
}
```

```
public Shopper(boolean isActive, String name, double money) {
    this(null, isActive, name, money, true);
}
```

```
public Shopper(String name) {
    this(null, true, name, 0, true);
}
```

```
public Shopper() {
    this(null, false, null, 0, true);
}
```

```
public void setShopperInChords() {
    this.shopperImage.setX(this.xChord);
    this.shopperImage.setY(this.yChord);

    this.shopperText.setLayoutX(this.xChord - 15);
    this.shopperText.setLayoutY(this.yChord - 25);
    this.shopperText.setText(this.name + ", money: " + this.money);
}
```

```
this.shadow.setLayoutX(this.xChord - 10);
this.shadow.setLayoutY(this.yChord + 170);
```

```
this.triangleAct.setLayoutX(this.xChord + 30);
this.triangleAct.setLayoutY(this.yChord - 40);
```

```
if (this.isActive) {
    this.triangleAct.setOpacity(1);
} else {
    this.triangleAct.setOpacity(0);
}
if (this.instrument != null) {
    this.instrument.update(this.xChord, this.yChord);
}
}
```

```
public void freeRun() {
    if (!this.isActive() && !this.stay) {
```

```

double speed = Main.getCity().isInteractWithPlayerMode() ? Preferences.getSPEED()/3 : Preferences.getSPEED();
if (((this instanceof OrchestraConductor) && !Main.getCity().isInteractWithPlayerMode()) && Main.getCity().getShoppers().size() >= 2)
    speed = 0;
switch (this.startDirection) {
    case 0:
        this.yChord -= speed;
        break;
    case 1:
        this.yChord -= speed;
        this.xChord += speed;
        break;
    case 2:
        this.xChord += speed;
        break;
    case 3:
        this.xChord += speed;
        this.yChord += speed;
        break;
    case 4:
        this.yChord += speed;
        break;
    case 5:
        this.yChord += speed;
        this.xChord -= speed;
        break;
    case 6:
        this.xChord -= speed;
        break;
    case 7:
        this.xChord -= speed;
        this.yChord -= speed;
        break;
}
if (Main.getCity().isInteractWithPlayerMode()) {
    if (this.shopperImage.getX() + 100 >= Main.getScene().getWidth() + Main.getScrollX()) {
        this.startDirection = (byte) (Main.random.nextInt(3) + 5);
    } else if (this.shopperImage.getX() <= 0 + Main.getScrollX()) {
        this.startDirection = (byte) (Main.random.nextInt(3) + 1);
    } else if (this.shopperImage.getY() <= 0 + Main.getScrollY()) {
        this.startDirection = (byte) (Main.random.nextInt(3) + 3);
    } else if (this.shopperImage.getY() + 170 >= Main.getScene().getHeight() + Main.getScrollY()) {
        this.startDirection = (byte) Main.random.nextInt(2);
    }
} else {
    if (this.shopperImage.getX() + 100 >= Main.getCity().getRoot().getWidth()) {
        this.startDirection = (byte) (Main.random.nextInt(3) + 5);
    } else if (this.shopperImage.getX() <= 0) {
        this.startDirection = (byte) (Main.random.nextInt(3) + 1);
    } else if (this.shopperImage.getY() <= 0) {
        this.startDirection = (byte) (Main.random.nextInt(3) + 3);
    } else if (this.shopperImage.getY() + 170 >= Main.getCity().getRoot().getHeight()) {
        this.startDirection = (byte) Main.random.nextInt(2);
    }
}

```

```

        this.freeInteract();
    }
}

public void mouseClicked(double x, double y) {
    if (this.shopperImage.getBoundsInParent().contains(new Point2D(x, y)) && !this.stay) {
        this.isActive = !this.isActive;
    }
}

public void left(double boost) {
    if (isActive && xChord >= 0) {
        xChord -= Preferences.getSPEED() * boost;
    }
}

public void right(double boost) {
    if (isActive && xChord <= Main.getCity().getRoot().getWidth() - 100) {
        xChord += Preferences.getSPEED() * boost;
    }
}

public void up(double boost) {
    if (isActive && yChord >= 0) {
        yChord -= Preferences.getSPEED() * boost;
    }
}

public void down(double boost) {
    if (isActive && yChord <= Main.getCity().getRoot().getHeight() - 200) {
        yChord += Preferences.getSPEED() * boost;
    }
}

public void setStay(boolean stay) {
    this.isActive = false;
    this.stay = stay;
}

public boolean isStay() {
    return stay;
}

public void buyAnInstrument(String typeOfInstrument) {
    if (this.money >= Instrument.getInstrument(typeOfInstrument).getPrice()) {
        this.instrument = Instrument.getInstrument(typeOfInstrument);
        this.money -= this.instrument.getPrice();
        this.shopperPicture.getChildren().add(this.instrument.getInstrumentImage());
    } else {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle("Увага!");
        alert.setHeaderText("Приходьте пізніше");
    }
}

```

```

        alert.setContentText("Недостатньо коштів для покупки даного інструмента, прямуйте заробляти");
        alert.showAndWait();
    }
}

public void sellAnInstrument() {
    if (this.getInstrument() != null) {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setHeaderText("Приходьте пізніше");
        alert.setContentText("Будучи початківцем, продавати ваш єдиний інструмент дуже ризиковано.");
        alert.showAndWait();
    } else {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setContentText("У вас немає що продавати");
        alert.showAndWait();
    }
}

public Shopper education(boolean first, boolean second, double allPrice) {
    if (first && second && this.money >= allPrice) { // провырку на жынку не забути
        if (this.isMan) {
            return new OrchestraConductor(Instrument.getInstrument(this.instrument.getType()), true, this.name, this.money - allPrice);
        } else {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);
            alert.setContentText("Жінки не можуть бути дирижорами");
            alert.showAndWait();
        }
    } else if (first && !second && this.money >= allPrice) {
        return new MusicianMaster(Instrument.getInstrument(this.instrument.getType()), true, this.name, this.money - allPrice, this.isMan);
    } else if (second && !first && this.money >= allPrice) {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setContentText("Спочатку потрібно пройти початкове навчання, а вже потім учитися на дирижора, або ж можна на обох одразу");
        alert.showAndWait();
    } else {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setContentText("Недостатньо коштів");
        alert.showAndWait();
    }
    return this;
}

public void playAnInstrument() {

    this.shopperImage.setOpacity(0);

    ImageView shopperPlaySprite;
    if (this.isMan) {
        shopperPlaySprite = new ImageView(new Image("assets/shopperPlaySprite.png"));
    } else {
        shopperPlaySprite = new ImageView(new Image("assets/girlShopperPlaySprite.png"));
    }
    shopperPlaySprite.setX(this.shopperImage.getX());
}

```

```
shopperPlaySprite.setY(this.shopperImage.getY());
shopperPlaySprite.setPreserveRatio(true);
shopperPlaySprite.setFitHeight(this.shopperImage.getFitHeight());
```

```
this.getShopperPicture().getChildren().add(4, shopperPlaySprite);
this.setStay(true);
```

```
Animation playAnimation = new Sprite(shopperPlaySprite, Duration.millis(2000), 4, 4, 431, 683);
playAnimation.setCycleCount(10); // як довго буде грати
```

```
String musicPath = "";
switch (this.instrument.getType()) {
    case "Guitar":
        musicPath = "src/assets/music/witcher.mp3";
        break;
    case "Bayan":
        musicPath = "src/assets/music/bayanBad.mp3";
        break;
    case "Drums":
        musicPath = "src/assets/music/drumsBad.mp3";
        break;
    case "Piano":
        musicPath = "src/assets/music/dogWaltz.mp3";
        break;
    case "Trembita":
        musicPath = "src/assets/music/trembitaBad.mp3";
        break;
    case "Violin":
        musicPath = "src/assets/music/violinBad.mp3";
        break;
}
```

```
AudioClip mediaPlayer = null;
try {
    Media hit = new Media(Paths.get(musicPath).toUri().toString());
    mediaPlayer = new AudioClip(hit.getSource());
    mediaPlayer.setVolume(Preferences.getVOLUME());
    mediaPlayer.play();
} catch (MediaException m) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setHeaderText("Помилка загрузки аудіо!!");
    alert.setContentText(m.toString());
    alert.show();
}
AudioClip finalMediaPlayer = mediaPlayer;
```

```
playAnimation.play();
```

```
playAnimation.setOnFinished(event -> {
    this.shopperImage.setOpacity(1);
    this.getShopperPicture().getChildren().remove(shopperPlaySprite);
    this.setStay(false);
});
```

```

        if (finalMediaPlayer != null) {
            finalMediaPlayer.stop();
        }
    });

}

public void freeInteract(){
    if (this.instrument != null && this.money<200){
        for (Building b : (ArrayList<Building>) Main.getCity().getObjectsHashMap().get("Underpass")) {
            if (b.interact(this, false))
                break;
        }
    }

    if (this.isMan && this.money<500) {
        for (Building b : (ArrayList<Building>) Main.getCity().getObjectsHashMap().get("Factory")) {
            if (b.interact(this, false))
                break;
        }
    }

    if (this.instrument != null){
        if (Main.random.nextDouble()>0.99995){
            this.playAnInstrument();
        }
    }
}

public double getPlayVariable() {
    return playVariable;
}

public String getType() {
    return type;
}

public double getXChord() {
    return xChord;
}

public void setXChord(double xChord) {
    this.xChord = xChord;
}

public double getYChord() {
    return yChord;
}

public void setYChord(double yChord) {
    this.yChord = yChord;
}

public byte getStartDirection() {
    return startDirection;
}

```



```

public void setStartDirection(byte startDirection) {
    this.startDirection = startDirection;
}

public boolean isActive() {
    return isActive;
}

public void setActive(boolean active) {
    isActive = active;
}

public ImageView getShopperImage() {
    return shopperImage;
}

public boolean isMan() {
    return isMan;
}

public void setMan(boolean man) {
    isMan = man;
    if (man)
        this.shopperImage.setImage(new Image("assets/shopper.png"));
    else this.shopperImage.setImage(new Image("assets/girlShopper.png"));
}

public Instrument getInstrument() {
    return instrument;
}

public void setInstrument(Instrument instrument) {
    this.instrument = instrument;
    this.shopperPicture.getChildren().add(this.instrument.getInstrumentImage());
}

public Group getShopperPicture() {
    return shopperPicture;
}

public double getMoney() {
    return money;
}

public void setMoney(double money) {
    this.money = money;
}

public String getName() {
    return name;
}

```

```

public void setName(String name) {
    this.name = name;
}

public static int getNumberOfShoppers() {
    return numberOfShoppers;
}

public static void setNumberOfShoppers(int numberOfShoppers) {
    Shopper.numberOfShoppers = numberOfShoppers;
}

@Override
public Shopper clone() throws CloneNotSupportedException {
    Shopper temp = (Shopper) super.clone();
    if (this.instrument != null)
        temp.instrument = this.instrument.clone();
    temp.shadow = new Ellipse(75,20);
    temp.shadow.setFill(this.shadow.getFill());
    temp.shadow.setOpacity(this.shadow.getOpacity());
    temp.shadow.getTransforms().addAll(this.shadow.getTransforms());
    temp.shadow.setEffect(this.shadow.getEffect());
    temp.triangleAct = new Polygon();
    temp.triangleAct.getPoints().addAll(this.triangleAct.getPoints());
    temp.triangleAct.setFill(this.triangleAct.getFill());
    temp.shopperImage = new ImageView();
    temp.shopperImage.setPreserveRatio(this.shopperImage.isPreserveRatio());
    temp.shopperImage.setFitHeight(this.shopperImage.getFitHeight());
    temp.shopperImage.setEffect(this.shopperImage.getEffect());
    temp.shopperImage.setImage(this.shopperImage.getImage());
    temp.shopperText = new Label();
    temp.shopperText.setFont(this.shopperText.getFont());
    if (temp instanceof OrchestraConductor && this instanceof OrchestraConductor){
        ((OrchestraConductor) temp).setInstruments(new HashMap<>());
        ((OrchestraConductor) this).getInstruments().forEach((k,v)->{
            try {
                ((OrchestraConductor) temp).getInstruments().put(k,v.clone());
            } catch (CloneNotSupportedException e) {
                Alert alert = new Alert(Alert.AlertType.INFORMATION);
                alert.setContentText("Помилка клонування в інструментах диригента");
                alert.showAndWait();
            }
        });
    }
    if (temp.instrument != null) {
        temp.shopperPicture = new Group(temp.shopperImage, temp.shopperText, temp.shadow, temp.triangleAct,
temp.instrument.getImage());
    } else {
        temp.shopperPicture = new Group(temp.shopperImage, temp.shopperText, temp.shadow, temp.triangleAct);
    }
    return temp;
}

/*public static Comparator<Shopper> moneyComparator = new Comparator<Shopper>() {

```

```
@Override
public int compare(Shopper o1, Shopper o2) {
    return (int) (o1.money-o2.money);
}
};
public static Comparator<Shopper> nameComparator = new Comparator<Shopper>() {
    @Override
    public int compare(Shopper o1, Shopper o2) {
        return o1.name.compareTo(o2.name);
    }
};*/
}
```

## Лістинг модуля MusicianMaster.java

```

package objects.micro;

import javafx.animation.Animation;
import javafx.scene.Group;
import javafx.scene.control.Alert;
import javafx.scene.control.ButtonType;
import javafx.scene.control.Label;
import javafx.scene.effect.DropShadow;
import javafx.scene.effect.GaussianBlur;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.media.AudioClip;
import javafx.scene.media.Media;
import javafx.scene.paint.Color;
import javafx.scene.shape.Ellipse;
import javafx.scene.shape.Polygon;
import javafx.scene.text.Font;
import javafx.scene.transform.Rotate;
import javafx.util.Duration;
import objects.firstMacro.Instrument;
import sample.Sprite;
import sample.windows.preferencesWindow.Preferences;

import java.nio.file.Paths;
import java.util.Optional;

public class MusicianMaster extends Shopper {

    public MusicianMaster(Instrument instrument, boolean isActive, String name, double money, boolean isMan){
        super(instrument, isActive, name, money, isMan);
        this.type = "Master";
        this.playVariable = 1;

        if (isMan){
            this.shopperImage = new ImageView(new Image("assets/master.png"));
        } else this.shopperImage = new ImageView(new Image("assets/girlMaster.png"));

        this.shopperImage.setPreserveRatio(true);
        this.shopperImage.setFitHeight(190);
        this.shopperImage.setEffect(new DropShadow(30,0,0, Color.GRAY));

        this.instrument = instrument;

        this.shopperText = new Label(this.name + ", money: " + this.money);
        this.shopperText.setFont(new Font("Segoe UI Black Italic", 13));

        this.shadow = new Ellipse(95, 20);
        this.shadow.setFill(Color.BLACK);

```

```

this.shadow.setOpacity(0.9);
this.shadow.getTransforms().add(new Rotate(20));
this.shadow.setEffect(new GaussianBlur(40));

this.triangleAct = new Polygon();
triangleAct.getPoints().addAll(0.0, 0.0,
    30.0, 0.0,
    15.0, 25.0);
this.triangleAct.setFill(Color.BLACK);

if (this.instrument != null) {
    this.shopperPicture = new Group(shopperImage, shopperText, shadow, triangleAct, this.instrument.getInstrumentImage());
} else {
    this.shopperPicture = new Group(shopperImage, shopperText, shadow, triangleAct);
}
}

public MusicianMaster(boolean isActive, String name, double money, boolean isMan) {
    this(null, isActive, name, money, isMan);
}

public MusicianMaster( Instrument instrument, boolean isActive, String name) {
    this(instrument, isActive, name, 0, true);
}

public MusicianMaster( boolean isActive, String name) {
    this(null, isActive, name, 0, true);
}

public MusicianMaster(boolean isActive, String name, double money){
    this(null, isActive, name, money, true);
}

public MusicianMaster(String name){
    this(null, true, name, 0, true);
}

public MusicianMaster(){
    this(null, false, null, 0, true);
}

@Override
public void setShopperInChords() {

    this.shopperImage.setX(this.xChord);
    this.shopperImage.setY(this.yChord);

    this.shopperText.setLayoutX(this.xChord-15);
    this.shopperText.setLayoutY(this.yChord -17);
    this.shopperText.setText(this.name + ", money: "+ this.money);

    this.shadow.setLayoutX(this.xChord -10);
    this.shadow.setLayoutY(this.yChord + 170);

    this.triangleAct.setLayoutX(this.xChord+30);
    this.triangleAct.setLayoutY(this.yChord-40);

    if (this.isActive) {

```

```

        this.triangleAct.setOpacity(1);
    } else {
        this.triangleAct.setOpacity(0);
    }
    if (this.instrument != null) {
        this.instrument.update(this.xChord, this.yChord);
    }
}

@Override
public Shopper education(boolean first, boolean second, double allPrice) {
    if (second && !first && this.money >= allPrice)
        if (this.isMan)
            return new OrchestraConductor(Instrument.getInstrument(this.instrument.getType()), true, this.name, this.money-allPrice);
        else
        {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);
            alert.setContentText("Жінки не можуть бути дирижорами");
            alert.showAndWait();
        }
    else if (second && !first && this.money <= allPrice)
    {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setContentText("Недостатньо коштів");
        alert.showAndWait();
    } else {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setContentText("Ви уже пройшли цю програму!");
        alert.showAndWait();
    }
    return this;
}

@Override
public void sellAnInstrument() {
    if (this.instrument != null){
        Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
        alert.setContentText("Ви дійсно хочете продати поточний інструмент?");
        Optional<ButtonType> option = alert.showAndWait();
        if (option.get() == ButtonType.OK){
            this.shopperPicture.getChildren().remove(this.instrument.getInstrumentImage());
            this.money+= this.instrument.getPrice()* 0.5;
            this.instrument = null;
        }
    } else {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setContentText("У вас немає що продавати");
        alert.showAndWait();
    }
}

@Override

```

```

public void playAnInstrument() {

    this.shopperImage.setOpacity(0);
    ImageView masterPlaySprite;
    if (this.isMan) {
        masterPlaySprite = new ImageView(new Image("assets/masterPlaySprite.png"));
    } else {
        masterPlaySprite = new ImageView(new Image("assets/girlMasterPlaySprite.png"));
    }
    masterPlaySprite.setX(this.shopperImage.getX());
    masterPlaySprite.setY(this.shopperImage.getY());
    masterPlaySprite.setPreserveRatio(true);
    masterPlaySprite.setFitHeight(this.shopperImage.getFitHeight());

    this.getShopperPicture().getChildren().add(4 ,masterPlaySprite);
    this.setStay(true);

    Animation playAnimation = new Sprite(masterPlaySprite , Duration.millis(2000),4,4,431,683);
    playAnimation.setCycleCount(10);//як довго буде грати

    String musicPath = "";
    switch (this.instrument.getType()){
        case "Guitar":
            musicPath = "src/assets/music/guitarGood.mp3";
            break;
        case "Bayan":
            musicPath = "src/assets/music/bayanGood.mp3";
            break;
        case "Drums":
            musicPath = "src/assets/music/drumsGood.mp3";
            break;
        case "Piano":
            musicPath = "src/assets/music/pianoGood.mp3";
            break;
        case "Trembita":
            musicPath = "src/assets/music/trembitaGood.mp3";
            break;
        case "Violin":
            musicPath = "src/assets/music/despasito.mp3";
            break;
    }
    Media hit = new Media(Paths.get(musicPath).toUri().toString());
    AudioClip mediaPlayer = new AudioClip(hit.getSource());
    mediaPlayer.setVolume(Preferences.getVOLUME());
    playAnimation.play();
    mediaPlayer.play();

    playAnimation.setOnFinished(event -> {
        this.shopperImage.setOpacity(1);
        this.getShopperPicture().getChildren().remove(masterPlaySprite);
    });
}

```

```
        this.setStay(false);
        mediaPlayer.stop();
    });

}

@Override
public void setMan(boolean man) {
    isMan = man;
    if (man)
        this.shopperImage.setImage(new Image("assets/master.png"));
    else this.shopperImage.setImage(new Image("assets/girlMaster.png"));
}
}
```



## Лістинг модуля OrchestraConductor.java

```

package objects.micro;

import javafx.animation.Animation;
import javafx.geometry.Point2D;
import javafx.scene.Group;
import javafx.scene.control.Alert;
import javafx.scene.control.ButtonType;
import javafx.scene.control.Label;
import javafx.scene.effect.DropShadow;
import javafx.scene.effect.GaussianBlur;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.media.AudioClip;
import javafx.scene.media.Media;
import javafx.scene.paint.Color;
import javafx.scene.shape.Ellipse;
import javafx.scene.shape.Polygon;
import javafx.scene.text.Font;
import javafx.scene.transform.Rotate;
import javafx.util.Duration;
import objects.firstMacro.Instrument;
import sample.Main;
import sample.Sprite;
import sample.windows.chooseAnInstrumentWindow.ChooseAnInstrumentWindow;
import sample.windows.preferencesWindow.Preferences;

import java.io.IOException;
import java.nio.file.Paths;
import java.util.*;

public class OrchestraConductor extends MusicianMaster {

    private Map<String,Instrument> instruments;

    public OrchestraConductor(Instrument instrument, boolean isActive, String name, double money){
        super(isActive, name, money , true);
        this.type = "Orchestra";
        this.playVariable = 2;

        this.shopperImage = new ImageView(new Image("assets/orchestra.png"));

        this.shopperImage.setPreserveRatio(true);
        this.shopperImage.setFitHeight(190);
        this.shopperImage.setEffect(new DropShadow(30,0,0, Color.GRAY));

        this.instruments = new HashMap<>();

```

```

if (instrument != null) {
    this.instruments.put(instrument.getType(),instrument);
    this.instrument = instrument;
} else this.instrument = null;

this.shopperText = new Label(this.name + ", money: "+ this.money + "\nNumber of instruments: " +this.instruments.size());
this.shopperText.setFont(new Font("Segoe UI Black Italic", 13));

this.shadow = new Ellipse(95, 20);
this.shadow.setFill(Color.BLACK);
this.shadow.setOpacity(0.9);
this.shadow.getTransforms().add(new Rotate(20));
this.shadow.setEffect(new GaussianBlur(40));

this.triangleAct = new Polygon();
triangleAct.getPoints().addAll(0.0, 0.0,
    30.0, 0.0,
    15.0, 25.0);
this.triangleAct.setFill(Color.BLACK);

if (this.instrument != null){
    this.shopperPicture = new Group(shopperImage, shopperText, shadow, triangleAct, this.instrument.getInstrumentImage());
} else {
    this.shopperPicture = new Group(shopperImage, shopperText, shadow, triangleAct);
}
}

public OrchestraConductor(boolean isActive, String name) {
    this(null, isActive, name, 0);
}

public OrchestraConductor(boolean isActive, String name, double money){
    this( null, isActive, name, money);
}

public OrchestraConductor(String name){
    this( null, true, name, 0);
}

public OrchestraConductor(){
    this(null, false,null,0);
}

@Override
public void setShopperInChords() {
    this.shopperImage.setX(this.xChord-40);
    this.shopperImage.setY(this.yChord);

    this.shopperText.setLayoutX(this.xChord-27);
    this.shopperText.setLayoutY(this.yChord -40);
    this.shopperText.setText(this.name + ", money: "+ this.money + "\nNumber of instruments: " +this.instruments.size());

    this.shadow.setLayoutX(this.xChord -10);
    this.shadow.setLayoutY(this.yChord + 170);

    this.triangleAct.setLayoutX(this.xChord+20);

```

```

this.triangleAct.setLayoutY(this.yChord-65);

if (this.isActive) {
    this.triangleAct.setOpacity(1);
} else {
    this.triangleAct.setOpacity(0);
}
if (this.instrument != null) {
    this.instrument.update(this.xChord, this.yChord);
}
}

public void chooseAnInstrument(double x, double y){
    if (this.shopperImage.getBoundsInParent().contains(new Point2D(x, y)) && !this.stay) {
        try {
            new ChooseAnInstrumentWindow().display( this);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

public void changeAnInstrument(String currentInstrument){

    if (currentInstrument.equals("Nothing")){
        if (this.instrument != null)
            this.shopperPicture.getChildren().remove(this.instrument.getInstrumentImage());
        this.instrument= null;
    } else {
        if (this.instrument != null)
            this.shopperPicture.getChildren().remove(this.instrument.getInstrumentImage());
        this.instrument = instruments.get(currentInstrument);
        this.shopperPicture.getChildren().add(instrument.getInstrumentImage());
    }
}

@Override
public void buyAnInstrument(String typeOfInstrument) {
    if (this.money >= Instrument.getInstrument(typeOfInstrument).getPrice() && !this.instruments.containsKey(typeOfInstrument)){
        this.instruments.put(typeOfInstrument, Instrument.getInstrument(typeOfInstrument));
        this.money -= Instrument.getInstrument(typeOfInstrument).getPrice();
    } else {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle("Увага");
        alert.setHeaderText("Повертайтеся пізніше");
        alert.setContentText("Недостатньо коштів для покупки даного інструмента, або ви вже його маєте");
        alert.showAndWait();
    }
}

@Override
public void sellAnInstrument() {

```

```

if (this.instrument != null) {
    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
    alert.setContentText("Ви дійсно хочете продати поточний інструмент?");
    Optional<ButtonType> option = alert.showAndWait();
    if (option.get() == ButtonType.OK) {
        this.shopperPicture.getChildren().remove(this.instrument.getInstrumentImage());
        this.money += this.instrument.getPrice() * 0.5;
        this.instruments.remove(this.instrument.getType());
        this.instrument = null;
    }
} else {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setContentText("Візьміть у руки інструмент, який хочете продати");
    alert.showAndWait();
}
}

@Override
public void playAnInstrument() {
    this.shopperImage.setOpacity(0);
    ImageView orchestraPlaySprite = new ImageView(new Image("assets/orchestraPlaySprite.png"));
    orchestraPlaySprite.setX(this.shopperImage.getX());
    orchestraPlaySprite.setY(this.shopperImage.getY());
    orchestraPlaySprite.setPreserveRatio(true);
    orchestraPlaySprite.setFitHeight(this.shopperImage.getFitHeight());

    this.getShopperPicture().getChildren().add(4, orchestraPlaySprite);
    this.setStay(true);

    Animation playAnimation = new Sprite(orchestraPlaySprite, Duration.millis(2000), 4, 4, 561, 683);
    playAnimation.setCycleCount(10); // як довго буде грати

    String musicPath = "";
    if (this.instrument != null) {
        switch (this.instrument.getType()) {
            case "Guitar":
                musicPath = "src/assets/music/guitarBest.mp3";
                break;
            case "Bayan":
                musicPath = "src/assets/music/bayanBest.mp3";
                break;
            case "Drums":
                musicPath = "src/assets/music/drumsBest.mp3";
                break;
            case "Piano":
                musicPath = "src/assets/music/experience.mp3";
                break;
            case "Trembita":
                musicPath = "src/assets/music/trembitaBest.mp3";
                break;
            case "Violin":
                musicPath = "src/assets/music/violinBest.mp3";

```

```

        break;
    }
} else musicPath = "src/assets/music/guitarBest.mp3"; //чутьи все одно не буде

Media hit = new Media(Paths.get(musicPath).toUri().toString());
AudioClip mediaPlayer = new AudioClip(hit.getSource());
if (this.instrument != null)
    mediaPlayer.setVolume(Preferences.getVOLUME());
else mediaPlayer.setVolume(0);
playAnimation.play();
mediaPlayer.play();

playAnimation.setOnFinished(event -> {
    this.shopperImage.setOpacity(1);
    this.getShopperPicture().getChildren().remove(orchestraPlaySprite);
    this.setStay(false);
    mediaPlayer.stop();
});
}

@Override
public void freeInteract() {
    if (Main.getCity().getShoppers().size() >= 2) {
        Shopper closerShopper = Main.getCity().getShoppers().get(0);
        double closerDistance = Double.POSITIVE_INFINITY;
        for (Shopper shopper : Main.getCity().getShoppers()) {
            if (!(shopper instanceof OrchestraConductor) && !shopper.isStay() && shopper.getInstrument() != null) {
                double distance = Math.sqrt(Math.pow(this.getXChord() - shopper.getXChord(), 2) + Math.pow(this.getYChord() - shopper.getYChord(), 2));
                if (distance < closerDistance) {
                    closerShopper = shopper;
                    closerDistance = distance;
                }
            }
            if (this.shopperImage.getBoundsInParent().intersects(shopper.getShopperImage().getBoundsInParent())) {
                this.changeAnInstrument("Nothing");
                shopper.setXChord(this.getXChord() - 300);
                shopper.setYChord(this.getYChord());
                shopper.setShopperInChords();
                shopper.setStartDirection((byte) 6);
                this.playAnInstrument();
                shopper.playAnInstrument();
            }
        }
    }
}

if (closerShopper.getXChord() < this.getXChord()) {
    this.xChord -= Preferences.getSPEED() / 5;
} else {
    this.xChord += Preferences.getSPEED() / 5;
}

if (closerShopper.getYChord() < this.getYChord()) {
    this.yChord -= Preferences.getSPEED() / 5;
} else {
    this.yChord += Preferences.getSPEED() / 5;
}
}

```

```
    } else {  
        super.freeInteract();  
    }  
}  
  
public Map<String, Instrument> getInstruments() {  
    return instruments;  
}  
  
public void setInstruments(Map<String, Instrument> instruments) {  
    this.instruments = instruments;  
}  
  
}
```

## Лістинг модуля Building.java

```
package objects.secondMacro;

import javafx.scene.Group;
import javafx.scene.control.Label;
import javafx.scene.image.ImageView;
import javafx.scene.shape.Ellipse;
import objects.micro.Shopper;

public abstract class Building {
    protected double xChord;
    protected double yChord;

    protected ImageView buildingImage;
    protected Label buildingText;
    protected Ellipse shadow;

    protected Group buildingPicture;
    protected static int numberOfBuildings = 0;

    protected String buildingType = "Nothing";

    public Group getBuildingPicture() {
        return this.buildingPicture;
    }

    public void setXChord(double xChord) {
        this.xChord = xChord;
    }

    public void setYChord(double yChord) {
        this.yChord = yChord;
    }

    public double getXChord() {
        return xChord;
    }

    public double getYChord() {
        return yChord;
    }

    public static int getNumberOfBuildings() {
        return numberOfBuildings;
    }

    public static void setNumberOfBuildings(int numberOfBuildings) {
```

```
        Building.numberOfBuildings = numberOfBuildings;
    }

    public String getType() {
        return this.buildingType;
    }

    public ImageView getBuildingImage() {
        return buildingImage;
    }

    public abstract void setBuildingInChords();
    public abstract boolean interact(Shopper shopper, boolean isShiftDown);
}
```



## Лістинг модуля Factory.java

```

package objects.secondMacro;

import javafx.animation.Animation;
import javafx.geometry.Rectangle2D;
import javafx.scene.Group;
import javafx.scene.control.Alert;
import javafx.scene.control.Label;
import javafx.scene.effect.GaussianBlur;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.*;
import javafx.scene.media.AudioClip;
import javafx.scene.media.Media;
import javafx.scene.media.MediaException;
import javafx.scene.paint.Color;
import javafx.scene.shape.Ellipse;
import javafx.scene.text.Font;
import javafx.scene.transform.Rotate;
import javafx.util.Duration;
import objects.micro.Shopper;
import sample.Main;
import sample.Sprite;
import sample.windows.preferencesWindow.Preferences;

import java.nio.file.Paths;

public class Factory extends Building{

    private ImageView smokeSprite = new ImageView(new Image("assets/smokeSprite.png"));

    public Factory(){
        this.xChord = Main.random.nextInt(4000);
        this.yChord = Main.random.nextInt(2000);
        Building.numberOfBuildings++;
        this.buildingType = "Factory";
        this.buildingImage = new ImageView(new Image("assets/factory.png"));
        this.buildingImage.setPreserveRatio(true);
        this.buildingImage.setFitHeight(270);

        this.buildingText = new Label("Фабрика для заробітку, \псурове місце, не для жінок!");
        this.buildingText.setFont(new Font("Segoe UI Black Italic", 18));
        this.buildingText.setBorder(new Border(new BorderStroke(Color.YELLOW, BorderStrokeStyle.DASHED, new CornerRadii(10), new
        BorderWidths(3))));

        this.shadow = new Ellipse(175, 40);
        this.shadow.setFill(Color.BLACK);
        this.shadow.setOpacity(0.8);
    }

```

```

this.shadow.getTransforms().add(new Rotate(10));
this.shadow.setEffect(new GaussianBlur(40));
this.smokeSprite.setViewport(new Rectangle2D(0,100,100,100)); // це погане рішення для того щоб зробити спрайт диму невидимий, але воно працює!
this.smokeSprite.setPreserveRatio(true);
this.smokeSprite.setFitHeight(60);
Animation smokeAnimation = new Sprite(smokeSprite, Duration.millis(1000), 10, 10, 100, 100);
smokeAnimation.setCycleCount(Animation.INDEFINITE);

this.buildingPicture = new Group(shadow, buildingImage, buildingText, smokeSprite);
smokeAnimation.play();
}
@Override
public void setBuildingInChords() {
    double x = this.xChord;
    double y = this.yChord;
    this.buildingImage.setX(x);
    this.buildingImage.setY(y);
    this.buildingText.setLayoutX(x+115);
    this.buildingText.setLayoutY(y+280);
    this.shadow.setLayoutX(x+90);
    this.shadow.setLayoutY(y+230);

    this.smokeSprite.setX(x+190);
    this.smokeSprite.setY(y-70);
}

@Override
public boolean interact(Shopper shopper, boolean isShiftDown) {
    if (this.getBuildingImage().getLayoutBounds().contains(shopper.getShopperImage().getLayoutBounds())) {
        if (shopper.isMan()) {

            shopper.getShopperImage().setOpacity(0);

            ImageView shopperWorkSprite = new ImageView(new Image("assets/shopperWorkSprite.png"));
            shopperWorkSprite.setX(shopper.getShopperImage().getX());
            shopperWorkSprite.setY(shopper.getShopperImage().getY());
            shopperWorkSprite.setPreserveRatio(true);
            shopperWorkSprite.setFitHeight(shopper.getShopperImage().getFitHeight());
            shopper.getShopperPicture().getChildren().add(shopperWorkSprite);

            shopper.setStay(true);
            if (shopper.getInstrument() != null) {
                shopper.getInstrument().getInstrumentImage().setOpacity(0);
            }
            Animation workAnimation = new Sprite(shopperWorkSprite, Duration.millis(1500), 8, 8, 498, 684);
            workAnimation.setCycleCount(20); // як довго буде працювати

            AudioClip mediaPlayer = null;

```

```

try {
    Media hit = new Media(Paths.get("src/assets/music/factory.mp3").toUri().toString());
    mediaPlayer = new AudioClip(hit.getSource());
    mediaPlayer.setVolume(Preferences.getVOLUME()*0.5);
    mediaPlayer.play();
} catch (MediaException m){
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setHeaderText("Помилка загрузки аудіо!!");
    alert.setContentText(m.toString());
    alert.show();
}
AudioClip finalMediaPlayer = mediaPlayer;

workAnimation.play();

workAnimation.setOnFinished(event -> {

    shopper.setMoney(shopper.getMoney() +Preferences.getCOMPLEXITY().getSalary());
    shopper.getShopperImage().setOpacity(1);
    shopper.getShopperPicture().getChildren().remove(shopperWorkSprite);
    shopper.setStay(false);
    finalMediaPlayer.stop();
    if (shopper.getInstrument() != null) {
        shopper.getInstrument().getInstrumentImage().setOpacity(1);
    }
});
} else {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setContentText("Жінки не можуть працювати на важкій роботі");
    alert.showAndWait();
}
return true;
}
return false;
}
}

```

## Лістинг модуля School.java

```

package objects.secondMacro;

import javafx.scene.Group;
import javafx.scene.control.Alert;
import javafx.scene.control.Label;
import javafx.scene.effect.GaussianBlur;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.shape.Ellipse;
import javafx.scene.text.Font;
import javafx.scene.transform.Rotate;
import objects.micro.OrchestraConductor;
import objects.micro.Shopper;
import sample.Main;
import sample.windows.buyAnInstrumentWindow.BuyAnInstrumentWindow;
import sample.windows.chooseAnEducationWindow.ChooseAnEducationWindow;
import sample.windows.chooseAnInstrumentWindow.ChooseAnInstrumentWindow;

import java.awt.*;
import java.io.IOException;

public class School extends Building {
    public School() {
        this.xChord = Main.random.nextInt(4000);
        this.yChord = Main.random.nextInt(2000);
        this.buildingType = "School";
        Building.numberOfBuildings++;
        this.buildingImage = new ImageView(new Image("assets/school.png"));
        this.buildingImage.setPreserveRatio(true);
        this.buildingImage.setFitHeight(360);

        this.buildingText = new Label("Музична школа, де вас навчать!!!");
        this.buildingText.setFont(new Font("Segoe UI Black Italic", 17));
        this.buildingText.setBorder(new Border(new BorderStroke(Color.valueOf("#C64706"), BorderStrokeStyle.SOLID, new CornerRadii(0), new
        BorderWidths(3))));

        this.shadow = new Ellipse(175, 40);
        this.shadow.setFill(Color.BLACK);
        this.shadow.setOpacity(0.8);
        this.shadow.getTransforms().add(new Rotate(10));
        this.shadow.setEffect(new GaussianBlur(40));

        this.buildingPicture = new Group(shadow, buildingImage, buildingText);
    }

    @Override

```

```

public boolean interact(Shopper shopper, boolean isShiftDown) {
    if (this.getBuildingImage().getLayoutBounds().contains(shopper.getShopperImage().getLayoutBounds())) {
        if (!(shopper instanceof OrchestraConductor) && shopper.getInstrument() != null) {
            try {
                new ChooseAnEducationWindow().display(shopper);
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
        else {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);
            alert.setContentText("Ви уже максимально навчені, якщо ж ні - перевірте наявність інструменту");
            alert.showAndWait();
        }
        return true;
    }
    return false;
}

```

```

@Override
public void setBuildingInChords() {
    double x = this.xChord;
    double y = this.yChord;
    this.buildingImage.setX(x);
    this.buildingImage.setY(y);
    this.buildingText.setLayoutX(x+33 );
    this.buildingText.setLayoutY(y + 323);
    this.shadow.setLayoutX(x + 90);
    this.shadow.setLayoutY(y + 270);
}
}

```

## Лістинг модуля Shop.java

```

package objects.secondMacro;

import javafx.scene.Group;
import javafx.scene.control.Alert;
import javafx.scene.control.Label;
import javafx.scene.effect.GaussianBlur;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.shape.Ellipse;
import javafx.scene.text.Font;
import javafx.scene.transform.Rotate;
import objects.micro.OrchestraConductor;
import objects.micro.Shopper;
import sample.Main;
import sample.windows.buyAnInstrumentWindow.BuyAnInstrumentWindow;

import java.io.IOException;

public class Shop extends Building {
    public Shop() {
        this.xChord = Main.random.nextInt(4000);
        this.yChord = Main.random.nextInt(2000);
        this.buildingType = "Shop";
        Building.numberOfBuildings++;
        this.buildingImage = new ImageView(new Image("assets/shop.png"));
        this.buildingImage.setPreserveRatio(true);
        this.buildingImage.setFitHeight(270);
        this.buildingText = new Label("Магазин музичних інструментів");
        this.buildingText.setFont(new Font("Segoe UI Black Italic", 18));
        this.buildingText.setBorder(new Border(new BorderStroke(Color.valueOf("#462311"), BorderStrokeStyle.SOLID, new CornerRadii(20), new
BorderWidths(5))));
        this.shadow = new Ellipse(175, 40);
        this.shadow.setFill(Color.BLACK);
        this.shadow.setOpacity(0.8);
        this.shadow.getTransforms().add(new Rotate(10));
        this.shadow.setEffect(new GaussianBlur(40));

        this.buildingPicture = new Group(shadow, buildingImage, buildingText);
    }
    @Override
    public boolean interact(Shopper shopper, boolean isShiftDown) {
        if (this.getBuildingImage().getLayoutBounds().contains(shopper.getShopperImage().getLayoutBounds())) {
            if (!isShiftDown) {
                if (shopper instanceof OrchestraConductor || shopper.getInstrument() == null) {
                    try {
                        new BuyAnInstrumentWindow().display(shopper);
                    }
                }
            }
        }
    }
}

```

```

    } catch (IOException e) {
        e.printStackTrace();

    }
} else {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);

    alert.setTitle("Увага!");
    alert.setHeaderText("Повертайтеся пізніше");
    alert.setContentText("Неможливо купити: даний музикант не може володіти більше, ніж одним інструментом!");
    alert.showAndWait();
}
} else {
    shopper.sellAnInstrument();
}
return true;
}
return false;
}

@Override
public void setBuildingInChords() {
    double x = this.xChord;
    double y = this.yChord;
    this.buildingImage.setX(x);
    this.buildingImage.setY(y);
    this.buildingText.setLayoutX(x + 10);
    this.buildingText.setLayoutY(y + 280);
    this.shadow.setLayoutX(x + 90);
    this.shadow.setLayoutY(y + 230);
}
}

```

## Лістинг модуля Underpass.java

```

package objects.secondMacro;

import javafx.animation.Animation;
import javafx.scene.Group;
import javafx.scene.control.Alert;
import javafx.scene.control.Label;
import javafx.scene.effect.GaussianBlur;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.*;
import javafx.scene.media.AudioClip;
import javafx.scene.media.Media;
import javafx.scene.media.MediaException;
import javafx.scene.paint.Color;
import javafx.scene.shape.Ellipse;
import javafx.scene.text.Font;
import javafx.scene.transform.Rotate;
import javafx.util.Duration;
import objects.micro.Shopper;
import sample.Main;
import sample.Sprite;
import sample.windows.preferencesWindow.Preferences;

import java.nio.file.Paths;
import java.util.ArrayList;

public class Underpass extends Building {

    private ArrayList<Shopper> shoppersInside;
    private ImageView currentAnimation;

    public Underpass() {
        this.shoppersInside = new ArrayList<>(5);
        this.xChord = Main.random.nextInt(4000);
        this.yChord = Main.random.nextInt(2000);
        this.buildingType = "Underpass";
        Building.numberOfBuildings++;
        this.buildingImage = new ImageView(new Image("assets/underpass.png"));
        this.buildingImage.setPreserveRatio(true);
        this.buildingImage.setFitHeight(270);
        this.buildingText = new Label("Піздечний перехід. Всередині грають: "+shoppersInside.size());
        this.buildingText.setFont(new Font("Segoe UI Black Italic", 19));
        this.buildingText.setBorder(new Border(new BorderStroke(Color.valueOf("#B69D90"), BorderStrokeStyle.SOLID, new CornerRadii(0), new
BorderWidths(3))));
        this.shadow = new Ellipse(175, 25);
        this.shadow.setFill(Color.BLACK);
        this.shadow.setOpacity(0.8);
        this.shadow.getTransforms().add(new Rotate(10));
    }

```



```

this.shadow.setEffect(new GaussianBlur(40));

this.buildingPicture = new Group(shadow, buildingImage, buildingText);
}
@Override
public void setBuildingInChords() {
    double x = this.xChord;
    double y = this.yChord;
    this.buildingImage.setX(x);
    this.buildingImage.setY(y);
    this.buildingText.setText("Пізде́мний пере́хід. Всере́дині грають: "+shoppersInside.size());
    this.buildingText.setLayoutX(x );
    this.buildingText.setLayoutY(y + 270);
    this.shadow.setLayoutX(x + 90);
    this.shadow.setLayoutY(y + 250);
}

@Override
public boolean interact(Shopper shopper, boolean isShiftDown) {
    if (this.getBuildingImage().getLayoutBounds().contains(shopper.getShopperImage().getLayoutBounds())) {
        if (shopper.getInstrument() != null) {
            this.buildingImage.setOpacity(0);
            ImageView underpassSprite = new ImageView(new Image("assets/underpassSprite.png"));
            underpassSprite.setPreserveRatio(true);
            underpassSprite.setFitHeight(this.buildingImage.getFitHeight() + 60);
            underpassSprite.setX(this.buildingImage.getX() - 40);
            underpassSprite.setY(this.buildingImage.getY() - 50);
            if (this.shoppersInside.size() != 0) {
                currentAnimation.setOpacity(0);
            }
            currentAnimation = underpassSprite;
            this.buildingPicture.getChildren().add(underpassSprite);
            Animation dance = new Sprite(underpassSprite, Duration.millis(700), 13, 13, 485, 326);
            dance.setCycleCount(10);

            this.shoppersInside.add(shopper);
            shopper.getShopperPicture().setOpacity(0);
            shopper.setStay(true);
            this.setBuildingInChords();
            dance.play();

            dance.setOnFinished(event -> {
                shopper.getShopperPicture().setOpacity(1);
                shopper.setStay(false);
                shopper.setMoney(shopper.getMoney() + Preferences.getCOMPLEXITY().getUnderpassSalary() /
shoppersInside.size()*shopper.getPlayVariable());
                shopper.setStartDirection((byte)6);
                shopper.setXChord(this.xChord-200);
                this.shoppersInside.remove(shopper);
                this.buildingPicture.getChildren().remove(underpassSprite);
                if (this.shoppersInside.size() == 0)
                    this.buildingImage.setOpacity(1);
            });
        }
    }
}

```

```
        this.setBuildingInChords();
    });

    } else {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setContentText("Візьміть в руки інструмент");
        alert.showAndWait();
    }
    return true;
}
return false;
}

public int getShoppersInsideSize() {
    return shoppersInside.size();
}
}
```

## Лістинг модуля City.java

```

package objects.thirdMacro;

import javafx.scene.Group;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.input.MouseButton;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import objects.micro.MusicianMaster;
import objects.micro.OrchestraConductor;
import objects.micro.Shopper;
import objects.secondMacro.*;
import sample.Main;
import sample.MiniMap;
import sample.windows.preferencesWindow.Preferences;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.function.Consumer;

public class City {
    final public static int ROOT_WIDTH = 4800;
    final public static int ROOT_HEIGHT = 2700;
    private boolean interactWithPlayerMode; //характер руху

    private Label informationLabel;
    private Rectangle informationRectangle;
    private Group informationGroup;
    private Button informationButton;

    private Pane root;
    private ArrayList<Shopper> shoppers;
    private ArrayList<Building> buildings;
    private HashMap<String,ArrayList> objectsHashMap = new HashMap<>(); // для більш оптимізованої взаємодії мікрооб'єктів з макро
    private MiniMap miniMap;

    public City(){
        this.root = new Pane();
        this.root.setMinWidth(City.ROOT_WIDTH);
        this.root.setMinHeight(City.ROOT_HEIGHT);
        this.shoppers = new ArrayList<>(5);
        this.buildings = new ArrayList<>(5);
        root.setOnMousePressed(event -> {
            for (Shopper shopper : this.shoppers) {
                if (event.getButton().equals(MouseButton.PRIMARY)) {
                    shopper.mouseClick(event.getX(), event.getY());
                }
            }
        });
    }

```

```

    } else if (event.getButton().equals(MouseButton.SECONDARY)) {
        if (shopper instanceof OrchestraConductor) {
            ((OrchestraConductor) shopper).chooseAnInstrument(event.getX(), event.getY());
        }
    }
}

});

this.interactWithPlayerMode = true;
this.informationLabel = new Label();
this.informationLabel.setFont(new Font("Segoe UI Black Italic", 15));
this.informationLabel.setLayoutX(20);
this.informationLabel.setLayoutY(80);

this.informationRectangle = new Rectangle(480,250);
this.informationRectangle.setFill(Color.LIGHTGREY);
this.informationRectangle.setStroke(Color.BLACK);
this.informationRectangle.setLayoutX(10);
this.informationRectangle.setLayoutY(70);

this.informationButton = new Button("Ігрова інформація ▼");
this.informationButton.setFont(new Font("Segoe UI Black Italic", 15));
this.informationButton.setMinWidth(480);
this.informationButton.setMinHeight(15);
this.informationButton.setLayoutX(10);
this.informationButton.setLayoutY(35);
this.informationButton.setOnAction(event -> {
    if (this.informationRectangle.getOpacity()==1){
        this.informationRectangle.setOpacity(0);
        this.informationLabel.setOpacity(0);
        this.informationButton.setText("Ігрова інформація ▲");
    }else {
        this.informationRectangle.setOpacity(1);
        this.informationLabel.setOpacity(1);
        this.informationButton.setText("Ігрова інформація ▼");
    }
});

this.informationGroup = new Group(informationRectangle,informationLabel, informationButton);
this.root.getChildren().addAll(informationGroup);

this.miniMap = new MiniMap();
this.root.getChildren().addAll(miniMap.getPane());

//опасно так робити)))
objectsHashMap.put("Factory", new ArrayList<Factory>());
objectsHashMap.put("Shop", new ArrayList<Shop>());
objectsHashMap.put("School", new ArrayList<School>());
objectsHashMap.put("Underpass", new ArrayList<Underpass>());
objectsHashMap.put("Shopper", new ArrayList<Shopper>());
objectsHashMap.put("Master", new ArrayList<MusicianMaster>());
objectsHashMap.put("Orchestra", new ArrayList<OrchestraConductor>());
}

```

```

public void addNewShopper(Shopper shopper, boolean rand) {
    this.shoppers.add(shopper);
    if (rand) {
        shopper.setXChord(Main.random.nextInt((int) (Main.getScene().getWidth() + Main.getScrollX())));
        shopper.setYChord(Main.random.nextInt((int) (Main.getScene().getHeight() + Main.getScrollY())));
    }
    shopper.setShopperInChords();
    this.root.getChildren().add(Building.getNumberOfBuildings(),shopper.getShopperPicture());
    this.miniMap.addShopper(shopper);

    this.objectsHashMap.get(shopper.getType()).add(shopper);
}

public void deleteAShopper(Shopper shopper) {
    this.miniMap.deleteAShopper(shopper);
    this.root.getChildren().remove(shopper.getShopperPicture());
    this.shoppers.remove(shopper);
    Shopper.setNumberOfShoppers(Shopper.getNumberOfShoppers() - 1);
    this.objectsHashMap.get(shopper.getType()).remove(shopper);
}

public void addNewBuilding(Building building) {
    this.buildings.add(building);
    building.setBuildingInChords();
    this.root.getChildren().add(0,building.getBuildingPicture());
    this.miniMap.addBuilding(building);

    this.objectsHashMap.get(building.getType()).add(building);
}

public void deleteABuilding(Building building) {
    this.miniMap.deleteABuilding(building);
    this.root.getChildren().remove(building.getBuildingPicture());
    this.buildings.remove(building);
    Building.setNumberOfBuildings(Building.getNumberOfBuildings() - 1);

    this.objectsHashMap.get(building.getType()).remove(building);
}

public void updateInformationLabel(){
    if (Preferences.isINFORMATION()){
        this.getInformationGroup().setOpacity(1);
    } else {
        this.getInformationGroup().setOpacity(0);
    }
    String mode = interactWithPlayerMode ? "взаємодія з користувачем (N)" : "вільна взаємодія (N)";
    int numberOfActive = 0;
    int numberWithoutInstrument = 0;

```

```

        int numberOfShoppersInsideUnderpass = objectsHashMap.get("Underpass").stream().mapToInt(o -> ((Underpass)
o).getShoppersInsideSize()).sum();
        for (Shopper shopper: shoppers){
            if (shopper.isActive()){
                numberOfActive++;
            }
            if (shopper instanceof OrchestraConductor){
                if (((OrchestraConductor) shopper).getInstruments().size()==0)
                    numberWithoutInstrument++;
            } else if (shopper.getInstrument()==null){
                numberWithoutInstrument++;
            }
        }

        this.informationLabel.setText("-режим руху: "+mode+"\n-кількість вуличних музикантів: "+objectsHashMap.get("Shopper").size()+"\n-кількість
випускників муз. школи: "+objectsHashMap.get("Master").size()+"\n-кількість диригентів: "+objectsHashMap.get("Orchestra").size()+"\n-активних
мікрооб'єктів: "+numberOfActive+"\n-кількість мікрооб'єктів без інструменту: "+numberWithoutInstrument+"\n-кількість мікрооб'єктів всередині
підземних переходів: "+numberOfShoppersInsideUnderpass+"\n-кількість фабрик: "+objectsHashMap.get("Factory").size()+"\n-кількість музичних
шкіл: "+objectsHashMap.get("School").size()+"\n-кількість магазинів: "+objectsHashMap.get("Shop").size()+"\n-кількість підземних переходів:
"+objectsHashMap.get("Underpass").size());
    }

    public Pane getRoot() {
        return root;
    }

    public ArrayList<Shopper> getShoppers() {
        return shoppers;
    }

    public ArrayList<Building> getBuildings() {
        return buildings;
    }

    public MiniMap getMiniMap() {
        return miniMap;
    }

    public boolean isInteractWithPlayerMode() {
        return interactWithPlayerMode;
    }

    public void setInteractWithPlayerMode(boolean interactWithPlayerMode) {
        this.interactWithPlayerMode = interactWithPlayerMode;
    }

    public HashMap<String, ArrayList> getObjectsHashMap() {
        return objectsHashMap;
    }

    public Group getInformationGroup() {
        return informationGroup;
    }

```

}

## Лістинг модуля Instrument.java

```
package objects.firstMacro;

import javafx.scene.image.ImageView;

public abstract class Instrument implements Cloneable{
    protected double price;
    protected ImageView instrumentImage = null;
    protected String type = "Nothing";

    public abstract void update(double x, double y);

    public static Instrument getInstrument(String type){
        switch (type) {
            case "Guitar":
                return new Guitar();
            case "Drums":
                return new Drums();
            case "Bayan":
                return new Accordion();
            case "Piano":
                return new Piano();
            case "Trembita":
                return new Trembita();
            case "Violin":
                return new Violin();
            case "Nothing":
                return null;
        }
        return null;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public ImageView getInstrumentImage() {
        return instrumentImage;
    }

    public void setInstrumentImage(ImageView instrumentImage) {
        this.instrumentImage = instrumentImage;
    }

    public String getType() {
```



```
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    @Override
    public Instrument clone() throws CloneNotSupportedException {
        return (Instrument) super.clone();
    }
}
```

## Лістинг модуля Accordion.java

```
package objects.firstMacro;

import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

public class Accordion extends Instrument {
    public Accordion(){
        this.instrumentImage= new ImageView(new Image("assets/bayan.png"));
        this.instrumentImage.setPreserveRatio(true);
        this.instrumentImage.setFitHeight(80);
        this.type = "Bayan";
        this.price = 400;
    }

    @Override
    public void update(double x, double y) {
        this.instrumentImage.setX(x+10);
        this.instrumentImage.setY(y+60);
    }

    @Override
    public Accordion clone() throws CloneNotSupportedException {
        Accordion temp = (Accordion)super.clone();
        temp.instrumentImage = new ImageView();
        temp.instrumentImage.setImage(this.instrumentImage.getImage());
        temp.instrumentImage.setPreserveRatio(this.instrumentImage.isPreserveRatio());
        temp.instrumentImage.setFitHeight(this.instrumentImage.getFitHeight());
        return temp;
    }
}
```

## Лістинг модуля Drums.java

```
package objects.firstMacro;

import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

public class Drums extends Instrument{
    public Drums(){
        this.instrumentImage= new ImageView(new Image("assets/bombom.png"));
        this.instrumentImage.setPreserveRatio(true);
        this.instrumentImage.setFitHeight(180);
        this.type = "Drums";
        this.price = 1000;
    }
    @Override
    public void update(double x, double y) {
        this.instrumentImage.setX(x-10);
        this.instrumentImage.setY(y+60);
    }
    @Override
    public Drums clone() throws CloneNotSupportedException {
        Drums temp = (Drums) super.clone();
        temp.instrumentImage = new ImageView();
        temp.instrumentImage.setImage(this.instrumentImage.getImage());
        temp.instrumentImage.setPreserveRatio(this.instrumentImage.isPreserveRatio());
        temp.instrumentImage.setFitHeight(this.instrumentImage.getFitHeight());
        return temp;
    }
}
```

## Лістинг модуля Guitar.java

```
package objects.firstMacro;

import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

public class Guitar extends Instrument {

    public Guitar(){
        this.instrumentImage= new ImageView(new Image("assets/guitar.png"));
        this.instrumentImage.setPreserveRatio(true);
        this.instrumentImage.setFitHeight(80);
        this.type = "Guitar";
        this.price = 100;
    }

    @Override
    public void update(double x, double y) {
        this.instrumentImage.setX(x+10);
        this.instrumentImage.setY(y+70);
    }

    @Override
    public Guitar clone() throws CloneNotSupportedException {
        Guitar temp = (Guitar) super.clone();
        temp.instrumentImage = new ImageView();
        temp.instrumentImage.setImage(this.instrumentImage.getImage());
        temp.instrumentImage.setPreserveRatio(this.instrumentImage.isPreserveRatio());
        temp.instrumentImage.setFitHeight(this.instrumentImage.getFitHeight());
        return temp;
    }
}
```

## Лістинг модуля Piano.java

```
package objects.firstMacro;

import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

public class Piano extends Instrument{
    public Piano(){
        this.instrumentImage= new ImageView(new Image("assets/piano.png"));
        this.instrumentImage.setPreserveRatio(true);
        this.instrumentImage.setFitHeight(170);
        this.type = "Piano";
        this.price = 600;
    }
    @Override
    public void update(double x, double y) {
        this.instrumentImage.setX(x+50);
        this.instrumentImage.setY(y+60);
    }
    @Override
    public Piano clone() throws CloneNotSupportedException {
        Piano temp = (Piano) super.clone();
        temp.instrumentImage = new ImageView();
        temp.instrumentImage.setImage(this.instrumentImage.getImage());
        temp.instrumentImage.setPreserveRatio(this.instrumentImage.isPreserveRatio());
        temp.instrumentImage.setFitHeight(this.instrumentImage.getFitHeight());
        return temp;
    }
}
```

## Лістинг модуля Trembita.java

```
package objects.firstMacro;

import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

public class Trembita extends Instrument {
    public Trembita() {
        this.instrumentImage= new ImageView(new Image("assets/tremb.png"));
        this.instrumentImage.setPreserveRatio(true);
        this.instrumentImage.setFitHeight(80);
        this.type = "Trembita";
        this.price = 17;
    }
    @Override
    public void update(double x, double y) {
        this.instrumentImage.setX(x+35);
        this.instrumentImage.setY(y-20);
    }
    @Override
    public Trembita clone() throws CloneNotSupportedException {
        Trembita temp = (Trembita) super.clone();
        temp.instrumentImage = new ImageView();
        temp.instrumentImage.setImage(this.instrumentImage.getImage());
        temp.instrumentImage.setPreserveRatio(this.instrumentImage.isPreserveRatio());
        temp.instrumentImage.setFitHeight(this.instrumentImage.getFitHeight());
        return temp;
    }
}
```

## Лістинг модуля Violin.java

```
package objects.firstMacro;

import javafx.scene.image.Image;
import javafx.scene.image.ImageView;

public class Violin extends Instrument{
    public Violin(){
        this.instrumentImage= new ImageView(new Image("assets/violin.png"));
        this.instrumentImage.setPreserveRatio(true);
        this.instrumentImage.setFitHeight(140);
        this.type="Violin";
        this.price = 450;
    }
    @Override
    public void update(double x, double y) {
        this.instrumentImage.setX(x-40);
        this.instrumentImage.setY(y+20);
    }
    @Override
    public Violin clone() throws CloneNotSupportedException {
        Violin temp = (Violin) super.clone();
        temp.instrumentImage = new ImageView();
        temp.instrumentImage.setImage(this.instrumentImage.getImage());
        temp.instrumentImage.setPreserveRatio(this.instrumentImage.isPreserveRatio());
        temp.instrumentImage.setFitHeight(this.instrumentImage.getFitHeight());
        return temp;
    }
}
```

## Лістинг модуля AboutWindow.java

```
package sample.windows.aboutWindow;

import javafx.event.EventHandler;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.scene.input.KeyEvent;
import javafx.stage.Stage;

import java.io.IOException;

import static javafx.stage.Modality.APPLICATION_MODAL;

public class AboutWindow {

    private static Stage window;
    private static Scene scene;

    public static Scene getScene() {
        return scene;
    }
    public static Stage getWindow() {
        return window;
    }
    public AboutWindow(){
        window = new Stage();
        window.initModality(APPLICATION_MODAL);
        window.setResizable(false);

    }

    public boolean display() throws IOException {

        window.setTitle("Про програму");
        Parent alert = FXMLLoader.load(AboutWindow.class.getResource("aboutWindow.fxml"));
        scene = new Scene(alert);
        window.setScene(scene);
        window.getIcons().add(new Image("assets/aboutIcon.png"));
        window.show();

        return true;
    }
}
```



## Лістинг модуля aboutWindow.fxml

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.text.*?>

<AnchorPane prefHeight="360.0" prefWidth="599.0" style="-fx-background-color: e5dcb8; -fx-border-color: #611d1d; -fx-border-width: 8;"
xmlns="http://javafx.com/javafx/10.0.2-internal" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="sample.windows.aboutWindow.AboutWindowController">
    <children>
        <Label alignment="TOP_CENTER" layoutX="176.0" layoutY="29.0" prefHeight="53.0" prefWidth="246.0" text="Про програму!"
textAlignment="CENTER" textFill="#611d1d">
            <font>
                <Font name="Arial Black" size="22.0" />
            </font>
        </Label>
        <Label alignment="TOP_CENTER" layoutX="20.0" layoutY="71.0" prefHeight="256.0" prefWidth="559.0" text="
        &#9;Дана програма
        відрізняється своєю простотою &#10;та універсальністю відносно існуючих реалізацій.&#10;Написана програма, в порівняння з існуючими, не
        &#10;має таких переваг, як високорівнева 3D-графіка,&#10;чи велика функціональність з точки зору створення &#10;музики, але вона значно
        меншого розміру, не &#10;вимагатиме особливих системних вимог. І найголовніше &#10;– вона показує те, що «твориться за кулісами», а
        саме:&#10; початковий розвиток музиканта і покупку його першого &#10;інструмента." textFill="#611d1d">
            <font>
                <Font name="Segoe UI Black" size="18.0" />
            </font>
        </Label>
    </children>
</AnchorPane>

```

## Лістинг модуля BuyAnInstrumentWindow.java

```

package sample.windows.buyAnInstrumentWindow;

import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.stage.Stage;
import objects.micro.OrchestraConductor;
import objects.micro.Shopper;
import sample.windows.chooseAnInstrumentWindow.ChooseAnInstrumentWindow;
import sample.windows.createShopperWindow.CreateShopperWindow;

import java.io.IOException;

import static javafx.stage.Modality.APPLICATION_MODAL;

public class BuyAnInstrumentWindow {
    private static Stage window;
    private static Shopper shopper;

    public static Shopper getShopper() {
        return shopper;
    }

    public BuyAnInstrumentWindow(){
        window = new Stage();
        window.initModality(APPLICATION_MODAL);
        window.setResizable(false);
    }

    public boolean display(Shopper shopper) throws IOException {

        BuyAnInstrumentWindow.shopper = shopper;
        window.setTitle("Магазин інструментів");
        Parent alert = FXMLLoader.load(BuyAnInstrumentWindow.class.getResource("buyAnInstrumentWindow.fxml"));
        Scene scene = new Scene(alert);
        window.setScene(scene);
        window.getIcons().add(new Image("assets/createIcon.png"));
        window.show();

        return true;
    }

    public static Stage getWindow() {
        return window;
    }
}

```

## ЛІСТИНГ модуля BuyAnInstrumentWindowController.java

```
package sample.windows.buyAnInstrumentWindow;

import javafx.fxml.FXML;
import javafx.scene.control.*;
import objects.firstMacro.*;
import objects.firstMacro.Accordion;

public class BuyAnInstrumentWindowController {

    private String typeOfInstrument;

    @FXML
    private Button chooseButton;

    @FXML
    private RadioButton radioGuitar;

    @FXML
    private RadioButton radioViolin;

    @FXML
    private RadioButton radioPiano;

    @FXML
    private RadioButton radioBayan;

    @FXML
    private RadioButton radioDrums;

    @FXML
    private RadioButton radioTremb;

    @FXML
    private Label guitarLabel;

    @FXML
    private Label pianoLabel;

    @FXML
    private Label bayanLabel;

    @FXML
    private Label violinLabel;

    @FXML
    private Label drumsLabel;

    @FXML
```

```
private Label tremblLabel;
```

```
@FXML
```

```
void initialize() {
```

```
    ToggleGroup toggleGroup = new ToggleGroup();
```

```
    radioBayan.setToggleGroup(toggleGroup);
```

```
    radioDrums.setToggleGroup(toggleGroup);
```

```
    radioGuitar.setToggleGroup(toggleGroup);
```

```
    radioPiano.setToggleGroup(toggleGroup);
```

```
    radioTremb.setToggleGroup(toggleGroup);
```

```
    radioViolin.setToggleGroup(toggleGroup);
```

```
    radioBayan.setOnAction(event -> {
```

```
        typeOfInstrument = "Bayan";
```

```
    });
```

```
    radioDrums.setOnAction(event -> {
```

```
        typeOfInstrument = "Drums";
```

```
    });
```

```
    radioGuitar.setOnAction(event -> {
```

```
        typeOfInstrument = "Guitar";
```

```
    });
```

```
    radioPiano.setOnAction(event -> {
```

```
        typeOfInstrument = "Piano";
```

```
    });
```

```
    radioTremb.setOnAction(event -> {
```

```
        typeOfInstrument = "Trembita";
```

```
    });
```

```
    radioViolin.setOnAction(event -> {
```

```
        typeOfInstrument = "Violin";
```

```
    });
```

```
    radioGuitar.fire();
```

```
    guitarLabel.setText(new Guitar().getType() + " --" + new Guitar().getPrice());
```

```
    pianoLabel.setText(new Piano().getType() + " --" + new Piano().getPrice());
```

```
    bayanLabel.setText(new Accordion().getType() + " --" + new Accordion().getPrice());
```

```
    violinLabel.setText(new Violin().getType() + " --" + new Violin().getPrice());
```

```
    drumsLabel.setText(new Drums().getType() + " --" + new Drums().getPrice());
```

```
    tremblLabel.setText(new Trembita().getType() + " --" + new Trembita().getPrice());
```

```
    chooseButton.setOnAction(event -> {
```

```
        BuyAnInstrumentWindow.getShopper().buyAnInstrument(typeOfInstrument);
```

```
        BuyAnInstrumentWindow.getWindow().close();
```

```
    });
```

```
}
```

```
}
```

## Лістинг модуля buyAnInstrumentWindow.fxml

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.RadioButton?>
<?import javafx.scene.image.Image?>
<?import javafx.scene.image.ImageView?>
<?import javafx.scene.layout.AnchorPane?>
<?import javafx.scene.text.Font?>

<AnchorPane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="350.0" prefWidth="573.0"
xmlns="http://javafx.com/javafx/11.0.1" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="sample.windows.buyAnInstrumentWindow.BuyAnInstrumentWindowController">
    <children>
        <Button fx:id="chooseButton" layoutX="238.0" layoutY="300.0" mnemonicParsing="false" prefHeight="35.0" prefWidth="96.0" text="Вибрати"
/>
    </children>
    <ImageView fitHeight="55.0" fitWidth="131.0" layoutX="53.0" layoutY="66.0" pickOnBounds="true" preserveRatio="true">
        <image>
            <Image url="@../assets/guitar.png" />
        </image>
    </ImageView>
    <ImageView fitHeight="55.0" fitWidth="86.0" layoutX="46.0" layoutY="222.0" pickOnBounds="true" preserveRatio="true">
        <image>
            <Image url="@../assets/bayan.png" />
        </image>
    </ImageView>
    <ImageView fitHeight="96.0" fitWidth="134.0" layoutX="206.0" layoutY="127.0" pickOnBounds="true" preserveRatio="true">
        <image>
            <Image url="@../assets/bombom.png" />
        </image>
    </ImageView>
    <ImageView fitHeight="96.0" fitWidth="97.0" layoutX="36.0" layoutY="126.0" pickOnBounds="true" preserveRatio="true">
        <image>
            <Image url="@../assets/piano.png" />
        </image>
    </ImageView>
    <ImageView fitHeight="75.0" fitWidth="77.0" layoutX="214.0" layoutY="56.0" pickOnBounds="true" preserveRatio="true">
        <image>
            <Image url="@../assets/violin.png" />
        </image>
    </ImageView>
    <ImageView fitHeight="88.0" fitWidth="133.0" layoutX="200.0" layoutY="232.0" pickOnBounds="true" preserveRatio="true">
        <image>
            <Image url="@../assets/tremb.png" />
        </image>
    </ImageView>
    <RadioButton fx:id="radioGuitar" layoutX="13.0" layoutY="79.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="134.0">
        <font>
            <Font name="Segoe UI Black" size="12.0" />
        </font>
    </RadioButton>

```

```

</font>
</RadioButton>
<RadioButton fx:id="radioViolin" layoutX="183.0" layoutY="79.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="12.0" />
  </font>
</RadioButton>
<RadioButton fx:id="radioPiano" layoutX="13.0" layoutY="160.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="12.0" />
  </font>
</RadioButton>
<RadioButton fx:id="radioBayan" layoutX="13.0" layoutY="235.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="12.0" />
  </font>
</RadioButton>
<RadioButton fx:id="radioDrums" layoutX="180.0" layoutY="161.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="12.0" />
  </font>
</RadioButton>
<RadioButton fx:id="radioTremb" layoutX="181.0" layoutY="235.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="12.0" />
  </font>
</RadioButton>
<Label fx:id="guitarLabel" layoutX="397.0" layoutY="76.0" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="14.0" />
  </font>
</Label>
<Label fx:id="pianoLabel" layoutX="397.0" layoutY="104.0" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="14.0" />
  </font>
</Label>
<Label layoutX="220.0" layoutY="14.0" prefHeight="55.0" prefWidth="133.0" text="Магазин">
  <font>
    <Font name="Segoe UI Black" size="28.0" />
  </font>
</Label>
<Label fx:id="bayanLabel" layoutX="397.0" layoutY="138.0" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="14.0" />
  </font>
</Label>
<Label fx:id="violinLabel" layoutX="397.0" layoutY="172.0" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="14.0" />
  </font>
</Label>
<Label fx:id="drumsLabel" layoutX="397.0" layoutY="205.0" prefHeight="34.0" prefWidth="134.0">

```

```
</font>
  <Font name="Segoe UI Black" size="14.0" />
</font>
</Label>
<Label fx:id="trembLabel" layoutX="397.0" layoutY="235.0" prefHeight="34.0" prefWidth="134.0">
  <font>
    <Font name="Segoe UI Black" size="14.0" />
  </font>
</Label>
</children>
</AnchorPane>
```

## Лістинг модуля ChooseAnEducationWindow.java

```

package sample.windows.chooseAnEducationWindow;

import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.stage.Stage;
import objects.micro.OrchestraConductor;
import objects.micro.Shopper;
import sample.windows.chooseAnInstrumentWindow.ChooseAnInstrumentWindow;
import sample.windows.createShopperWindow.CreateShopperWindow;

import java.io.IOException;

import static javafx.stage.Modality.APPLICATION_MODAL;

public class ChooseAnEducationWindow {
    private static Stage window;
    private static Shopper shopper;

    public static Shopper getShopper() {
        return shopper;
    }

    public ChooseAnEducationWindow(){
        window = new Stage();
        window.initModality(APPLICATION_MODAL);
        window.setResizable(false);
    }

    public boolean display(Shopper shopper) throws IOException {

        ChooseAnEducationWindow.shopper = shopper;
        window.setTitle("Музична школа");
        Parent alert = FXMLLoader.load(ChooseAnEducationWindow.class.getResource("chooseAnEducationWindow.fxml"));
        Scene scene = new Scene(alert);
        window.setScene(scene);
        window.getIcons().add(new Image("assets/createIcon.png"));
        window.show();

        return true;
    }

    public static Stage getWindow() {
        return window;
    }
}

```



## Лістинг модуля ChooseAnEducationWindowController.java

```

package sample.windows.chooseAnEducationWindow;

import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Label;
import objects.micro.Shopper;
import sample.Main;
import sample.windows.preferencesWindow.Preferences;

public class ChooseAnEducationWindowController {

    private boolean firstEd = false;
    private boolean secondEd = false;

    private double allPr = 0;

    @FXML
    private Button chooseButton;

    @FXML
    private CheckBox firstEducation;

    @FXML
    private CheckBox secondEducation;

    @FXML
    private Label allPrice;

    @FXML
    void initialize() {
        double FIRST_PRICE = Preferences.getCOMPLEXITY().getFirstEducationPrice();
        double SECOND_PRICE = Preferences.getCOMPLEXITY().getSecondEducationPrice();
        allPrice.setText(Double.toString(0));
        firstEducation.setOnAction(event -> {
            firstEd = !firstEd;
            if (firstEd && secondEd)
                allPr = FIRST_PRICE+SECOND_PRICE;
            else if (firstEd)
                allPr = FIRST_PRICE;
            else if (secondEd)
                allPr = SECOND_PRICE;
            else allPr = 0;
            allPrice.setText(Double.toString(allPr));
        });
        secondEducation.setOnAction(event -> {
            secondEd = !secondEd;
            if (firstEd && secondEd)

```

```

        allPr = FIRST_PRICE+SECOND_PRICE;
    else if (firstEd)
        allPr = FIRST_PRICE;
    else if (secondEd)
        allPr = SECOND_PRICE;
    else allPr = 0;
    allPrice.setText(Double.toString(allPr));
});
chooseButton.setOnAction(event -> {
    if (firstEd || secondEd) {
        Shopper old = ChooseAnEducationWindow.getShopper();
        Shopper temp = old.education(firstEd, secondEd, allPr);
        if (old != temp){
            temp.setYChord(old.getYChord());
            temp.setXChord(old.getXChord());
            Main.getCity().deleteAShopper(old);
            Main.getCity().addNewShopper(temp,false);
        }

        ChooseAnEducationWindow.getWindow().close();
    }
});
}
}

```

## Лістинг модуля chooseAnEducationWindow.fxml

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.CheckBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.AnchorPane?>
<?import javafx.scene.text.Font?>

<AnchorPane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="200.0" prefWidth="269.0"
xmlns="http://javafx.com/javafx/11.0.1" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="sample.windows.chooseAnEducationWindow.ChooseAnEducationWindowController">
    <children>
        <Button fx:id="chooseButton" layoutX="86.0" layoutY="150.0" mnemonicParsing="false" prefHeight="35.0" prefWidth="96.0" text="Вибрати" />
        <CheckBox fx:id="firstEducation" layoutX="68.0" layoutY="14.0" mnemonicParsing="false" text="Початкова освіта ">
            <font>
                <Font name="Segoe UI Black" size="12.0" />
            </font>
        </CheckBox>
        <CheckBox fx:id="secondEducation" layoutX="70.0" layoutY="50.0" mnemonicParsing="false" text="Освіта дирижора">
            <font>
                <Font name="Segoe UI Black" size="12.0" />
            </font>
        </CheckBox>
        <Label fx:id="allPrice" layoutX="92.0" layoutY="100.0" prefHeight="35.0" prefWidth="84.0">
            <font>
                <Font name="Segoe UI Black Italic" size="12.0" />
            </font>
        </Label>
        <Label layoutX="89.0" layoutY="82.0" text="Загальна ціна:">
            <font>
                <Font name="Segoe UI Black" size="12.0" />
            </font>
        </Label>
    </children>
</AnchorPane>

```

## Лістинг модуля ChooseAnInstrumentWindow.java

```

package sample.windows.chooseAnInstrumentWindow;

import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.stage.Stage;
import objects.micro.OrchestraConductor;
import sample.windows.createShopperWindow.CreateShopperWindow;

import java.io.IOException;

import static javafx.stage.Modality.APPLICATION_MODAL;

public class ChooseAnInstrumentWindow {
    private static Stage window;
    private static OrchestraConductor orchestra;

    public static OrchestraConductor getOrchestraConductor() {
        return orchestra;
    }

    public ChooseAnInstrumentWindow() {
        window = new Stage();
        window.initModality(APPLICATION_MODAL);
        window.setResizable(false);
    }

    public boolean display( OrchestraConductor orchestraConductor) throws IOException {

        orchestra = orchestraConductor;
        window.setTitle("Виберіть інструмент");
        Parent alert = FXMLLoader.load(ChooseAnInstrumentWindow.class.getResource("chooseAnInstrumentWindow.fxml"));
        Scene scene = new Scene(alert);
        window.setScene(scene);
        window.getIcons().add(new Image("assets/aboutIcon.png"));
        window.show();

        return true;
    }

    public static Stage getWindow() {
        return window;
    }
}

```

## Лістинг модуля ChooseAnInstrumentWindowController.java

```

package sample.windows.chooseAnInstrumentWindow;

import javafx.beans.value.ChangeListener;
import javafx.beans.value.ObservableValue;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.ListView;

public class ChooseAnInstrumentWindowController {
    private String typeOfInstrument ;
    @FXML
    private ListView<String> instrumentsListShopper;

    @FXML
    private Button chooseButton;

    @FXML
    void initialize() {

        instrumentsListShopper.getItems().add("Nothing");
        ChooseAnInstrumentWindow.getOrchestraConductor().getInstruments().keySet().forEach((key)->{
            instrumentsListShopper.getItems().add(key);
        });
        instrumentsListShopper.getSelectionModel().selectedItemProperty().addListener(new ChangeListener<String>() {
            @Override
            public void changed(ObservableValue<? extends String> observable, String oldValue, String newValue) {
                typeOfInstrument = newValue;
                System.out.println(typeOfInstrument);
            }
        });
        chooseButton.setOnAction(event -> {
            ChooseAnInstrumentWindow.getOrchestraConductor().changeAnInstrument(typeOfInstrument);
            ChooseAnInstrumentWindow.getWindow().close();
        });
    }
}

```

## Лістинг модуля chooseAnInstrumentWindow.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.ListView?>
<?import javafx.scene.layout.AnchorPane?>

<AnchorPane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="200.0" prefWidth="207.0"
xmlns="http://javafx.com/javafx/11.0.1" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="sample.windows.chooseAnInstrumentWindow.ChooseAnInstrumentWindowController">
    <children>
        <ListView fx:id="instrumentsListShopper" prefHeight="134.0" prefWidth="219.0" />
        <Button fx:id="chooseButton" layoutX="65.0" layoutY="150.0" mnemonicParsing="false" prefHeight="35.0" prefWidth="96.0" text="Вибрати" />
    </children>
</AnchorPane>
```

## Лістинг модуля CreateShopperWindow.java

```
package sample.windows.createShopperWindow;

import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.stage.Stage;
import java.io.IOException;

import static javafx.stage.Modality.APPLICATION_MODAL;

public class CreateShopperWindow {

    private static Stage window;

    public CreateShopperWindow(){
        window = new Stage();
        window.initModality(APPLICATION_MODAL);
        window.setResizable(false);
    }

    public boolean display() throws IOException {

        window.setTitle("Створення нового персонажа");
        Parent alert = FXMLLoader.load(CreateShopperWindow.class.getResource("createShopperWindow.fxml"));
        Scene scene = new Scene(alert);
        window.setScene(scene);
        window.getIcons().add(new Image("assets/createIcon.png"));
        window.show();

        return true;
    }

    public static Stage getWindow() {
        return window;
    }
}
```

## Лістинг модуля CreateShopperWindowController.java

```
package sample.windows.createShopperWindow;
```

```
import javafx.beans.value.ChangeListener;
import javafx.beans.value.ObservableValue;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.*;
import objects.firstMacro.Instrument;
import objects.micro.MusicianMaster;
import objects.micro.OrchestraConductor;
import objects.micro.Shopper;
import sample.Main;
```

```
public class CreateShopperWindowController {
```

```
    private boolean isMan ;
    private String tempChange = "shopper";
    private String typeOfInstrument = "Nothing";
    @FXML
    private TextField fieldName;
```

```
    @FXML
    private TextField fieldMoney;
```

```
    @FXML
    private Button yesButton;
```

```
    @FXML
    private RadioButton radioMan;
```

```
    @FXML
    private RadioButton radioWoman;
```

```
    @FXML
    private ListView<String> instrumentsList;
```

```
    @FXML
    private RadioButton radioShopper;
```

```
    @FXML
    private RadioButton radioMaster;
```

```
    @FXML
    private RadioButton orchestra;
```



@FXML

private Label labelUnderInstruments;

@FXML

private CheckBox checkIsActive;

@FXML

void initialize() {

ToggleGroup toggleGroup = new ToggleGroup();

radioMan.setToggleGroup(toggleGroup);

radioWoman.setToggleGroup(toggleGroup);

radioMan.setOnAction((event -> {

isMan = true;

}));

radioWoman.setOnAction((event -> {

isMan = false;

}));

ToggleGroup toggleGroup1 = new ToggleGroup();

radioShopper.setToggleGroup(toggleGroup1);

radioMaster.setToggleGroup(toggleGroup1);

orchestra.setToggleGroup(toggleGroup1);

radioShopper.setOnAction((event -> {

radioWoman.setLayoutX(93);

tempChange = "shopper";

instrumentsList.setOpacity(0);

labelUnderInstruments.setLayoutX(128);

}));

radioMaster.setOnAction((event -> {

radioWoman.setLayoutX(93);

tempChange = "master";

instrumentsList.setOpacity(1);

labelUnderInstruments.setLayoutX(2000);

}));

orchestra.setOnAction((event -> {

radioWoman.setLayoutX(2000);

instrumentsList.setOpacity(1);

labelUnderInstruments.setLayoutX(2000);

tempChange = "orchestra";

}));

ObservableList<String> instruments = FXCollections.observableArrayList("Nothing", "Guitar", "Drums", "Bayan", "Piano", "Violin", "Trembita");

instrumentsList.getItems().addAll(instruments);

instrumentsList.getSelectionModel().selectedItemProperty().addListener(new ChangeListener<String>() {

@Override

public void changed(ObservableValue<? extends String> observable, String oldValue, String newValue) {

typeOfInstrument = newValue;

```

    }
});

radioMan.fire();
radioShopper.fire();

yesButton.setOnAction(event -> {
    if (!fieldName.getText().isEmpty() && !fieldMoney.getText().isEmpty()){
        Shopper temp = null;
        if (tempChange.equals("shopper")){
            temp = new Shopper(null,checkIsActive.isSelected(), fieldName.getText(), Integer.parseInt(fieldMoney.getText()),isMan);
        } else if (tempChange.equals("master")){
            temp = new MusicianMaster(Instrument.getInstrument(typeOfInstrument),checkIsActive.isSelected(), fieldName.getText(),
Integer.parseInt(fieldMoney.getText()),isMan);
        } else if (tempChange.equals("orchestra")){
            temp = new OrchestraConductor(Instrument.getInstrument(typeOfInstrument),checkIsActive.isSelected(), fieldName.getText(),
Integer.parseInt(fieldMoney.getText()));

        }
        Main.getCity().addNewShopper(temp, true);
        CreateShopperWindow.getWindow().close();
    }
});
}
}

```

## Лістинг модуля createShopperWindow.fxml

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.CheckBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.ListView?>
<?import javafx.scene.control.RadioButton?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.AnchorPane?>
<?import javafx.scene.text.Font?>

<AnchorPane prefHeight="371.0" prefWidth="569.0" style="-fx-background-color: e5dcb8; -fx-border-color: #611d1d; -fx-border-width: 8;"
xmlns="http://javafx.com/javafx/11.0.1" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="sample.windows.createShopperWindow.CreateShopperWindowController">
    <children>
        <Label alignment="CENTER" layoutX="109.0" prefHeight="91.0" prefWidth="350.0" text="Створення нового покупця." textFill="#611d1d">
            <font>
                <Font name="Segoe UI Black Italic" size="23.0" />
            </font>
        </Label>
        <TextField fx:id="fieldName" layoutX="72.0" layoutY="95.0" prefHeight="34.0" prefWidth="191.0" promptText="Ім'я" style="-fx-background-
radius: 10;" />
        <TextField fx:id="fieldMoney" layoutX="72.0" layoutY="143.0" prefHeight="34.0" prefWidth="191.0" promptText="Кількість грошей" style="-fx-
background-radius: 10;" />
        <Button fx:id="yesButton" layoutX="365.0" layoutY="265.0" mnemonicParsing="false" prefHeight="55.0" prefWidth="150.0" style="-fx-
background-radius: 10;" text="Створити" textAlignment="CENTER" textFill="#611d1d">
            <font>
                <Font name="Segoe UI Black" size="14.0" />
            </font>
        </Button>
        <RadioButton fx:id="radioMan" layoutX="94.0" layoutY="196.0" mnemonicParsing="false" prefHeight="20.0" prefWidth="98.0" text="Чоловік">
            <font>
                <Font name="Segoe UI Black" size="12.0" />
            </font></RadioButton>
        <RadioButton fx:id="radioWoman" layoutX="93.0" layoutY="224.0" mnemonicParsing="false" prefHeight="18.0" prefWidth="98.0" text="Жінка"
textFill="#c800ff">
            <font>
                <Font name="Segoe UI Black" size="12.0" />
            </font></RadioButton>
        <ListView fx:id="instrumentsList" layoutX="114.0" layoutY="252.0" prefHeight="82.0" prefWidth="200.0" style="-fx-background-radius: 10;" />
        <Label layoutX="21.0" layoutY="102.0" prefHeight="18.0" prefWidth="50.0" text="Ім'я:">
            <font>
                <Font name="Segoe UI Black Italic" size="14.0" />
            </font>
        </Label>
        <Label layoutX="18.0" layoutY="150.0" prefHeight="18.0" prefWidth="50.0" text="Гроші:">
            <font>
                <Font name="Segoe UI Black Italic" size="14.0" />
            </font>
        </Label>
    </children>

```

```

</Label>
<Label layoutX="10.0" layoutY="284.0" text="Інструмент:">
  <font>
    <Font name="Segoe UI Black Italic" size="14.0" />
  </font>
</Label>
<Label layoutX="19.0" layoutY="208.0" text="Стать:">
  <font>
    <Font name="Segoe UI Black Italic" size="14.0" />
  </font>
</Label>
<RadioButton fx:id="radioShopper" layoutX="318.0" layoutY="86.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="200.0"
text="Вуличний &quot;профі&quot;">
  <font>
    <Font name="Segoe UI Black" size="16.0" />
  </font>
</RadioButton>
<RadioButton fx:id="radioMaster" layoutX="318.0" layoutY="126.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="229.0"
text="Випускник муз. школи">
  <font>
    <Font name="Segoe UI Black" size="16.0" />
  </font>
</RadioButton>
<RadioButton fx:id="orchestra" layoutX="318.0" layoutY="168.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="200.0" text="Дирижор
оркестру">
  <font>
    <Font name="Segoe UI Black" size="16.0" />
  </font>
</RadioButton>
<Label fx:id="labelUnderInstruments" layoutX="118.0" layoutY="262.0" prefHeight="63.0" prefWidth="191.0" text="Сурові початківці
можуть&#10;бути музикантами навіть&#10;без інструменту!">
  <font>
    <Font name="Segoe UI Black" size="14.0" />
  </font>
</Label>
<CheckBox fx:id="checkIsActive" layoutX="193.0" layoutY="208.0" mnemonicParsing="false" prefHeight="20.0" prefWidth="97.0"
text="Активувати">
  <font>
    <Font name="Segoe UI Black" size="12.0" />
  </font>
</CheckBox>
</children>
</AnchorPane>

```

## Лістинг модуля Complexity.java

```

package sample.windows.preferencesWindow
import sample.Main;
public enum Complexity {
    LIGHT{
        public int getSalary(){
            return 600 + Main.random.nextInt(600);
        }
        public int getFirstEducationPrice(){
            return 500;
        }
        public int getSecondEducationPrice(){
            return 1500;
        }
        public int getUnderpassSalary(){
            return 300;
        }
    },
    MEDIUM{
        public int getSalary(){
            return 200 +Main.random.nextInt(500);
        }
        public int getFirstEducationPrice(){
            return 2000;
        }
        public int getSecondEducationPrice(){
            return 5000;
        }
        public int getUnderpassSalary(){
            return 200;
        }
    },
    HARD{
        public int getSalary(){
            return Main.random.nextInt(100);
        }
        public int getFirstEducationPrice(){
            return 7000;
        }
        public int getSecondEducationPrice(){
            return 14000;
        }
        public int getUnderpassSalary(){
            return 30;
        }
    };
    public abstract int getSalary();
    public abstract int getFirstEducationPrice();
    public abstract int getSecondEducationPrice();
    public abstract int getUnderpassSalary();

```

}

## Лістинг модуля Preferences.java

```
package sample.windows.preferencesWindow;
import sample.Main;
public class Preferences {
    private static boolean MAP = true;
    private static boolean INFORMATION = true;
    private static double VOLUME = 0.4;
    private static double SPEED = 7;
    private static Complexity COMPLEXITY = Complexity.LIGHT;

    public static boolean isMAP() {
        return MAP;
    }

    public static boolean isINFORMATION() {
        return INFORMATION;
    }

    public static double getVOLUME() {
        return VOLUME;
    }

    public static double getSPEED() {
        return SPEED;
    }

    public static void setMAP(boolean MAP) {
        Preferences.MAP = MAP;
    }

    public static void setINFORMATION(boolean INFORMATION) {
        Preferences.INFORMATION = INFORMATION;
    }

    public static void setVOLUME(double VOLUME) {
        Preferences.VOLUME = VOLUME;
    }

    public static void setSPEED(double SPEED) {
        Preferences.SPEED = SPEED;
    }

    public static void setCOMPLEXITY(Complexity complexity)
    {
        COMPLEXITY = complexity;
    }

    public static Complexity getCOMPLEXITY() {
        return COMPLEXITY;
    }
}
```

}



## Лістинг модуля PreferencesWindow.java

```
package sample.windows.preferencesWindow;

import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.stage.Stage;

import java.io.IOException;

import static javafx.stage.Modality.APPLICATION_MODAL;

public class PreferencesWindow {
    private static Stage window;

    public PreferencesWindow(){
        window = new Stage();
        window.initModality(APPLICATION_MODAL);
        window.setResizable(false);
    }

    public boolean display() throws IOException {

        window.setTitle("Налаштування");
        Parent alert = FXMLLoader.load(PreferencesWindow.class.getResource("preferencesWindow.fxml"));
        Scene scene = new Scene(alert);
        window.setScene(scene);
        window.getIcons().add(new Image("assets/prefIcon.png"));
        window.show();

        return true;
    }

    public static Stage getWindow() {
        return window;
    }
}
```

## Лістинг модуля PreferencesWindowController.java

```

package sample.windows.preferencesWindow;
import javafx.fxml.FXML;
import javafx.scene.control.*;

public class PreferencesWindowController {

    private boolean isMap = Preferences.isMAP();
    private boolean isInformation = Preferences.isINFORMATION();
    private double speed = Preferences.getSPEED();
    private double volume = Preferences.getVOLUME();
    private Complexity complexity = Preferences.getCOMPLEXITY();
    @FXML
    private CheckBox checkMiniMap;
    @FXML
    private CheckBox checkInformation;
    @FXML
    private RadioButton radioLight;
    @FXML
    private RadioButton radioMedium;
    @FXML
    private RadioButton radioHard;
    @FXML
    private ScrollBar scrollSpeed;
    @FXML
    private ScrollBar scrollVolume;
    @FXML
    private Button chooseButton;
    @FXML
    private Button cancelButton;
    @FXML

    void initialize() {

        checkMiniMap.setOnAction(event -> {
            isMap = checkMiniMap.isSelected();
        });
        checkMiniMap.setSelected(Preferences.isMAP());

        checkInformation.setOnAction(event -> {
            isInformation = checkInformation.isSelected();
        });
        checkInformation.setSelected(Preferences.isINFORMATION());

        scrollSpeed.setMin(1);
        scrollSpeed.setMax(80);
        scrollSpeed.setValue(speed);
        scrollSpeed.valueProperty().addListener(event -> {
            speed = scrollSpeed.getValue();
        });
        scrollVolume.setMin(0);
        scrollVolume.setMax(1.5);
    }
}

```

```

scrollVolume.setValue(volume);
scrollVolume.valueProperty().addListener(event -> {
    volume = scrollVolume.getValue();
});
ToggleGroup toggleGroup = new ToggleGroup();
radioLight.setToggleGroup(toggleGroup);
radioMedium.setToggleGroup(toggleGroup);
radioHard.setToggleGroup(toggleGroup);

radioLight.setOnAction(event -> {
    complexity = Complexity.LIGHT;
});
radioMedium.setOnAction(event -> {
    complexity = Complexity.MEDIUM;

});
radioHard.setOnAction(event -> {
    complexity = Complexity.HARD;
});
switch (complexity) {
    case LIGHT:
        radioLight.fire();
        break;
    case MEDIUM:
        radioMedium.fire();
        break;
    case HARD:
        radioHard.fire();
        break;
}

chooseButton.setOnAction(event -> {
    Preferences.setMAP(isMap);
    Preferences.setINFORMATION(isInformation);
    Preferences.setSPEED(speed);
    Preferences.setVOLUME(volume);
    Preferences.setCOMPLEXITY(complexity);
    PreferencesWindow.getWindow().close();
});
cancelButton.setOnAction(event -> {
    PreferencesWindow.getWindow().close();
});

}
}

```

## Лістинг модуля preferencesWindow.fxml

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.CheckBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.RadioButton?>
<?import javafx.scene.control.ScrollBar?>
<?import javafx.scene.layout.AnchorPane?>
<?import javafx.scene.text.Font?>

<AnchorPane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="261.0" prefWidth="430.0" style="-fx-background-color: e5dcb8; -fx-border-color: #611d1d; -fx-border-width: 8;" xmlns="http://javafx.com/javafx/11.0.1"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="sample.windows.preferencesWindow.PreferencesWindowController">
    <children>
        <CheckBox fx:id="checkMiniMap" layoutX="27.0" layoutY="178.0" mnemonicParsing="false" text="Відображати міні-карту">
            <font>
                <Font name="Segoe UI Black" size="12.0" />
            </font></CheckBox>
        <RadioButton fx:id="radioLight" layoutX="275.0" layoutY="68.0" mnemonicParsing="false" prefHeight="20.0" prefWidth="82.0" text="Легкий">
            <font>
                <Font name="Segoe UI Black" size="14.0" />
            </font></RadioButton>
        <RadioButton fx:id="radioMedium" layoutX="275.0" layoutY="102.0" mnemonicParsing="false" text="Середній">
            <font>
                <Font name="Segoe UI Black" size="14.0" />
            </font></RadioButton>
        <RadioButton fx:id="radioHard" layoutX="275.0" layoutY="137.0" mnemonicParsing="false" prefHeight="20.0" prefWidth="91.0" text="Важкий">
            <font>
                <Font name="Segoe UI Black" size="14.0" />
            </font></RadioButton>
        <ScrollBar fx:id="scrollSpeed" layoutX="22.0" layoutY="98.0" prefHeight="14.0" prefWidth="137.0" />
        <ScrollBar fx:id="scrollVolume" layoutX="21.0" layoutY="147.0" prefHeight="14.0" prefWidth="138.0" />
        <Label layoutX="27.0" layoutY="68.0" text="Швидкість:">
            <font>
                <Font name="Segoe UI Black" size="14.0" />
            </font></Label>
        <Label layoutX="27.0" layoutY="120.0" text="Гучність:">
            <font>
                <Font name="Segoe UI Black" size="14.0" />
            </font></Label>
        <Button fx:id="chooseButton" layoutX="311.0" layoutY="204.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="91.0" text="Зберегти">
            <font>
                <Font name="Segoe UI Black" size="12.0" />
            </font></Button>
        <Label layoutX="147.0" layoutY="25.0" text="Налаштування">
            <font>
                <Font name="Segoe UI Black" size="18.0" />
            </font>
        </Label>
    </children>

```

```
<Button fx:id="cancelButton" layoutX="215.0" layoutY="204.0" mnemonicParsing="false" prefHeight="34.0" prefWidth="91.0" text="Відмінити">
  <font>
    <Font name="Segoe UI Black" size="12.0" />
  </font>
</Button>
<CheckBox fx:id="checkInformation" layoutX="27.0" layoutY="204.0" mnemonicParsing="false" text="Відображати інформацію">
  <font>
    <Font name="Segoe UI Black" size="12.0" />
  </font>
</CheckBox>
</children>
</AnchorPane>
```

